ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN

Đánh giá Hiệu năng Hệ thống (CO3007)

ĐÁNH GIÁ HIỆU NĂNG HỆ THỐNG BẤT ĐỒNG BỘ DỰA TRÊN LÝ THUYẾT HÀNG ĐỢI VÀ MÔ PHỎNG SỰ KIỆN RỜI RẠC

Lớp: L02 – Nhóm: SPE-L0x-008

Ngày nộp: 27/10/2026

GVHD: Thầy Nguyễn Phương Duy

STT	Họ và Tên Sinh Viên	MSSV
1	Lê Kim Ngân	2312218
2	Lê Trần Bảo Nhi	2312499
3	Nguyễn Anh Thư	2313385
4	Phan Ngọc Phương Trinh	2313582
5	Phạm Công Võ	2313946
6	Lưu Nguyên Vũ	2314058
7	Nguyễn Thị Thảo Vân	2420010
8	Nguyễn Trung Kiên	2311738
9	Trần Quang Vinh	2313932

TP. HỒ CHÍ MINH, THÁNG 12/2026

BẢNG PHÂN CÔNG NHIỆM VỤ THỰC HIỆN ĐỀ TÀI

(Đính kèm Báo cáo Bài tập lớn – Đánh giá Hiệu năng Hệ thống (CO3007) – Nhóm SPE-L0x-008)

STT	Họ và tên	MSSV	Vai trò chính	Nhiệm vụ được phân công chi tiết	Đánh giá
KHỐI VAI TRÒ 1: DESIGNER (THIẾT KẾ HỆ THỐNG, MÔ HÌNH HÓA & KIẾN TRÚC PHẦN MỀM)					
1	—	—	**Designer** *(System & DES Arch)*	• Xác định bối cảnh, ranh giới hệ thống và các dịch vụ đánh giá; soạn thảo **Mục 1.1, 1.2**. • Thiết kế kiến trúc tổng thể bộ máy mô phỏng DES (Event Loop, PDES, LVT); soạn thảo **Mục 3.1**.	—
2	—	—	**Designer** *(Metrics & Async Arch)*	• Thiết kế hệ thống chỉ số hiệu năng (Time-weighted vs. Sample-averaged); soạn thảo **Mục 1.3, 1.4, 1.5**. • Thiết kế kiến trúc bất đồng bộ phi nghẽn luồng và phân tách dòng thời gian ảo; soạn thảo **Mục 3.2, 3.3**.	—
3	—	—	**Designer** *(Models & UML Design)*	• Mô hình hóa lý thuyết hàng đợi đơn ($M/M/1, M/M/1/K, M/M/\infty$) và đa server/mạng; soạn thảo **Phần 2 (Mục 2.1 – 2.3)**. • Thiết kế hướng đối tượng phần mềm, đặc tả quan hệ và xây dựng biểu đồ UML; soạn thảo **Mục 3.4**.	—
KHỐI VAI TRÒ 2: DEV / DEVELOPER (PHÁT TRIỂN & HIỆN THỰC TOÀN BỘ MÃ NGUỒN)					
4	—	—	**Dev** *(Lead DES Engine)*	• Lập trình hạt nhân điều phối DES (`SequentialEventDispatcher`), cấu trúc `Event` và đồng hồ ảo LVT trong gói `bkitevsim/core/` và `bkitevsim/dispatchers/`. • Hiện thực hàng đợi phân tầng `LadderQueue` $O(1)$ và `MinHeapPriorityQueue` $O(\log N)$.	—
5	—	—	**Dev** *(Async Coordination)*	• Lập trình tầng bất đồng bộ `bkitevsim/async_engine/`: bộ quản lý lời hứa `AsyncFutureManager` (Promise/Future) và coroutine `BkSysCoLogicalProcess`. • Hiện thực bộ điều phối `AsyncTaskCoordinator` hỗ trợ `TaskGroup`, `gather`, `as_completed`, `Runner`.	—
6	—	—	**Dev** *(LP Models & Manual)*	• Lập trình các trạm logic `bkitevsim/models/`: `SourceLP`, `QueueServerLP` (Kendall hợp nhất), `BranchingServerLP`, `SinkLP`. • Lập trình bộ tích lũy `MetricsCollector`; viết hướng dẫn cài đặt và cấu trúc mã nguồn (**Mục 5.3**).	—
KHỐI VAI TRÒ 3: EXPERIMENT / THỰC NGHIỆM (KIỂM CHỨNG & ĐÁNH GIÁ HIỆU NĂNG)					
7	—	—	**Experiment** *(Workload & Scenarios)*	• Thiết kế bộ sinh tải (Poisson Workload Generator, Bursty Traffic) và ma trận kịch bản thực nghiệm từ tải nhẹ đến bão hòa ($\rho = 0.1 \to 0.95$). • Soạn thảo **Mục 4.1 và Mục 4.2** (Workload & Thiết kế kịch bản).	—
8	—	—	**Experiment** *(Verification & Valid)*	• Xây dựng bộ kiểm thử tự động 31 bài test (`pytest`), kiểm chứng tính bảo toàn Job, tính đơn điệu nhân quả (Trace log). • Thẩm định độ chính xác mô hình ($M/M/1$, Erlang loss $M/M/n/n$, Jackson Network) với sai số $< 5\%$; soạn thảo **Mục 4.3, 4.4**.	—
9	—	—	**Experiment** *(Sensitivity Analysis)*	• Thực hiện phân tích độ nhạy dung lượng bộ đệm K (đánh đổi Drop rate vs. Latency), nhận diện nút thắt cổ chai, xuất biểu đồ 300 DPI. • Phân tích tối ưu hóa hệ thống; soạn thảo **Mục 4.5 và Mục 5.1, 5.2** (Tổng kết, bài học kinh nghiệm).	—

TÓM TẮT ĐỀ TÀI

(ĐÁNH GIÁ HIỆU NĂNG HỆ THỐNG BẤT ĐỒNG BỘ BẰNG LÝ THUYẾT HÀNG ĐỢI VÀ MÔ PHỎNG SỰ KIỆN RỜI RẠC)

Báo cáo Bài tập lớn này tập trung vào việc mô hình hóa, thiết kế kiến trúc và đánh giá hiệu năng toàn diện một hệ thống tính toán phân tán với cơ chế truyền thông bất đồng bộ phi nghẽn luồng (*asynchronous non-blocking communication*). Dựa trên phương pháp luận 10 bước của GS. Raj Jain, nhóm nghiên cứu đã kết hợp chặt chẽ giữa **Lý thuyết hàng đợi giải tích** (*Analytical Queuing Theory*) và kỹ thuật **Mô phỏng sự kiện rời rạc** (*Discrete-Event Simulation – DES*).

Về mặt lý thuyết, hệ thống được mô hình hóa thành mạng hàng đợi mở nhiều chặng theo Định lý Jackson, bao gồm thiết bị I/O, máy chủ trung tâm CPU và các thiết bị ngoại vi (Máy in, Ổ đĩa) rẽ nhánh xác suất, cùng hệ thống các mô hình hàng đợi đơn chuẩn Kendall: $M/M/1$, $M/M/1/K$, $M/M/\infty$, $M/M/c$ và hệ thống tổn thất Erlang $M/M/n/n$.

Về mặt hiện thực, nhóm đã xây dựng thành công một bộ máy mô phỏng DES hướng đối tượng tuân thủ mô hình tiến trình logic PDES của Fujimoto. Hệ thống tích hợp đồng hồ thời gian ảo LVT đơn điệu, cấu trúc hàng đợi phân tầng hai cấp `LadderQueue` đạt độ phức tạp $O(1)$, và ứng dụng các kỹ thuật đồng quy hiện đại trong Python 3.11+ (`asyncio.TaskGroup`, `AsyncFutureManager`, `Runner`) để phối hợp các tác vụ bất đồng bộ.

Kết quả kiểm chứng (*Verification*) thông qua bộ kiểm thử tự động 31 ca kiểm thử chuyên sâu (`pytest`) xác nhận hệ thống bảo toàn tuyệt đối số lượng thực thể và tính nhân quả thời gian. Quá trình thẩm định (*Validation*) với 5 lượt lặp độc lập (*replications*) và ước lượng Khoảng tin cậy 95% Student's t cho thấy kết quả mô phỏng hội tụ xuất sắc với nghiệm giải tích, đạt sai số tương đối $\varepsilon < 5\%$ trên toàn dải tải trọng ($\rho = 0.1 \to 0.95$). Cuối cùng, phân tích độ nhạy đã làm rõ sự đánh đổi giữa tỉ lệ rơi gói và thời gian trễ theo dung lượng bộ đệm K, đồng thời định vị chính xác nút thắt cổ chai (*Bottleneck*) phục vụ mục tiêu tối ưu hóa tài nguyên.

Từ khóa: Mô phỏng sự kiện rời rạc (DES), Lý thuyết hàng đợi, Ký hiệu Kendall, Python asyncio, TaskGroup, LadderQueue, Định lý Jackson, Khoảng tin cậy 95%, Phân tích độ nhạy.

MỤC LỤC

BẢNG PHÂN CÔNG NHIỆM VỤ i

TÓM TẮT ĐỀ TÀI ii

MỤC LỤC iii

DANH MỤC HÌNH VẼ v

DANH MỤC BẢNG BIỂU vi

1 Tổng quan đề tài và Mục tiêu đánh giá 1

1.1 Bối cảnh bài toán và Mục tiêu hệ thống . . . 1

1.1.1 Bối cảnh kỹ thuật: Chuyển dịch sang kiến trúc bất đồng bộ hướng sự kiện 1

1.1.2 Mục tiêu và phạm vi đánh giá hiệu năng . . . 2

1.2 Dịch vụ hệ thống và Kết quả khả dĩ . . . 2

1.2.1 Đặc tả 4 dịch vụ cốt lõi của hệ thống . . . 2

1.2.2 Bốn trạng thái kết quả khả dĩ (Possible Outcomes) . . . 3

1.3 Hệ thống chỉ số hiệu năng chính (KPIs) . . . 3

1.4 Bảng quy ước ký hiệu toán học và thuật ngữ kỹ thuật . . . 4

1.5 Tham số hệ thống, Tải làm việc và Kế hoạch thực nghiệm . . . 6

1.5.1 Tham số hệ thống (System Parameters) . . . 6

1.5.2 Tham số tải làm việc (Workload Parameters) . . . 7

1.5.3 Thiết kế Kế hoạch thực nghiệm và 5 Kịch bản khảo sát cốt lõi . . . 8

1.6 Lựa chọn kỹ thuật đánh giá hiệu năng . . . 9

1.6.1 So sánh ba phương pháp luận đánh giá . . . 9

1.6.2 Chiến lược phương pháp luận kép (Dual-Methodology Strategy) . . . 11

2 Cơ sở lý thuyết và Mô hình hóa hàng đợi 12

2.1 Tổng quan lý thuyết hàng đợi và Ký hiệu Kendall chuẩn hóa . . . 12

2.1.1 Cấu trúc tổng thể của một hệ thống hàng đợi . . . 12

2.1.2 Hệ thống ký hiệu Kendall mở rộng . . . 13

2.1.3 Tính chất không nhớ (Memoryless Property) của phân phối Mũ . . . 14

2.2 Tiến trình ngẫu nhiên và Thuộc tính của dòng Poisson . . . 15

2.2.1 Khái niệm tiến trình ngẫu nhiên và Chuỗi Markov . . . 15

2.2.2 Đặc trưng và 5 thuộc tính cốt lõi của Dòng Poisson . . . 15

2.3 Tiến trình Sinh-Tử (Birth-Death Processes) và Cân bằng dừng . . . 16

2.3.1 Mô hình toán học của Tiến trình Sinh-Tử . . . 16

2.3.2 Phương trình cân bằng dòng toàn cục và Nghiệm xác suất dừng . . . 16

2.4 Các nguyên lý phổ quát cho mọi hệ thống hàng đợi ($G/G/c$ Rules) . . . 17

2.4.1 Điều kiện ổn định hệ thống (Stability Condition) . . . 17

2.4.2 Quan hệ số lượng trong hệ thống và hàng đợi 18
2.4.3 Định luật Little (Little's Law, Little 1961) 18
2.4.4 Quan hệ thời gian lưu trú và thời gian chờ 18
2.5 Phân tích chi tiết 6 mô hình hàng đợi đơn chuẩn hóa và mô hình $M/D/1$ 19
2.5.1 Mô hình $M/M/1$: Hệ thống đơn máy chủ, bộ đệm vô hạn 19
2.5.2 Mô hình $M/M/1/K$: Hệ thống đơn máy chủ, dung lượng đệm hữu hạn . 20
2.5.3 Mô hình $M/M/\infty$: Hệ thống vô hạn máy chủ (Pure Delay System) 22
2.5.4 Mô hình $M/M/n/n$: Hệ thống Erlang-B tổn thất (Loss System) 23
2.5.5 Mô hình $M/M/c$: Hệ thống đa máy chủ, bộ đệm vô hạn (Erlang-C Delay) 24
2.5.6 Mô hình $M/M/c/K$: Hệ thống đa máy chủ, dung lượng đệm hữu hạn . . 26
2.5.7 Mô hình đối chứng $M/D/1$: Thời gian phục vụ tất định 27
2.6 Mô hình mạng hàng đợi Jackson (Jackson Queueing Networks) 28
2.6.1 Cấu trúc mạng hàng đợi mở (Open Jackson Networks) 28
2.6.2 Hệ phương trình cân bằng lưu lượng (Traffic Equations) 28
2.6.3 Ứng dụng giải tích cho Mô hình mạng tham chiếu của đề tài 29

3 Kiến trúc và Hiện thực bộ mô phỏng DES 31
3.1 Thiết kế phần mềm và Mô hình hóa hướng đối tượng (UML Design) 31
3.1.1 Mô tả tổng quan sơ đồ cấu trúc lớp (UML Class Diagram) 32
3.1.2 Mối quan hệ giữa các lớp trong hệ thống 42
3.1.3 Sơ đồ tuần tự tương tác sự kiện (UML Sequence Diagram) 44
3.2 Kiến trúc điều phối cốt lõi của bộ mô phỏng DES (Core DES Dispatching Architecture) 48
3.2.1 Cấu trúc Lớp Bản ghi Sự kiện (`Event`) và Cơ chế So sánh theo Thời gian ảo (Timestamp Ordering) 48
3.2.2 Cấu trúc hàng đợi sự kiện phân tầng hai cấp (LadderQueue) 51
3.2.3 Vòng lặp điều phối sự kiện trung tâm (Dispatcher Event Loop) 54
3.2.4 Bộ điều khiển mô phỏng Facade (Simulation & SimulationOptions) 56
3.3 Hiện thực mô hình Tiến trình logic và Các trạm hàng đợi (PDES) 59
3.3.1 Lớp cơ sở Tiến trình logic và Cơ chế giao tiếp hướng thông điệp 59
3.3.2 Kiến trúc trạm phục vụ hợp nhất 6 mô hình hàng đợi Kendall 61
3.3.3 Hiện thực mạng hàng đợi Jackson và Cơ chế rẽ nhánh xác suất 64
3.4 Cơ chế giao tiếp phi nghẽn và Điều phối bất đồng bộ (Python Asyncio) 66
3.4.1 Cơ chế giao tiếp phi nghẽn luồng theo mô hình Promise / Future 66
3.4.2 Các mẫu điều phối đồng quy có cấu trúc (Structured Concurrency) 67
3.5 Quản lý thời gian ảo và Kiểm chứng tính nhân quả cục bộ 69
3.5.1 Nguyên lý bước nhảy thời gian ảo và Phân tách với thời gian thực 69
3.5.2 Ràng buộc nhân quả cục bộ và Kiểm chứng vết tính toán 70

TÀI LIỆU THAM KHẢO 71

DANH MỤC HÌNH VẼ

1 Sơ đồ chuyển dịch tốc độ trạng thái trong Tiến trình Sinh-Tử tổng quát 16
2 Sơ đồ cấu trúc mạng hàng đợi Jackson 4 trạm tham chiếu của đề tài 29
3 Sơ đồ cấu trúc lớp chi tiết (UML Class Diagram) của hệ thống mô phỏng DES . 32
4 Sơ đồ tuần tự luân chuyển sự kiện (UML Sequence Diagram) trong vòng lặp DES 44
5 Kiến trúc phân tầng hai cấp và cơ chế nạp tầng (Rung Refill) của thuật toán LadderQueue 51
6 Máy trạng thái hữu hạn (FSM) mô hình hóa vòng đời phục vụ của `QueueServerLP` 63

DANH MỤC BẢNG BIỂU

1 Bảng quy ước ký hiệu toán học và thuật ngữ kỹ thuật chuẩn hóa 6
2 Bảng phân loại và cấu hình tham số hệ thống (System Parameters) 7
3 Bảng phân loại tham số tải làm việc và các yếu tố nghiên cứu (Workload Parameters & Study Factors) 8
4 So sánh 3 phương pháp đánh giá hiệu năng hệ thống theo chuẩn Raj Jain 11
5 Bảng chú thích vai trò và cấu trúc lưu trữ của các thực thể tham gia sơ đồ tuần tự 45
6 Ma trận thông điệp sự kiện luân chuyển giữa các thành phần 47
7 So sánh độ phức tạp tính toán giữa Min-Heap và LadderQueue 53
8 Ánh xạ tham số cấu hình hợp nhất cho 6 mô hình hàng đợi Kendall 62
9 So sánh đặc tính kỹ thuật của 4 mẫu điều phối bất đồng bộ 68
10 So sánh bản chất giữa Mô phỏng DES và Mô phỏng theo thời gian thực 69

1 Tổng quan đề tài và Mục tiêu đánh giá

Nghiên cứu tập trung thiết lập bối cảnh kỹ thuật, xác định phạm vi nghiên cứu, các mục tiêu đo lường trọng tâm và phương pháp luận đánh giá hiệu năng hệ thống dựa trên quy trình khoa học của Raj Jain [1]. Trọng tâm nội dung hướng đến việc phân tích bài toán điều phối trong các hệ thống xử lý bất đồng bộ hiện đại, xây dựng và chuẩn hóa hệ thống chỉ số đánh giá hiệu năng (Key Performance Indicators – KPIs), đồng thời hệ thống hóa các ký hiệu và quy ước toán học giải tích làm cơ sở lý thuyết thống nhất cho quá trình phân tích, mô hình hóa và kiểm chứng xuyên suốt đề tài.

1.1 Bối cảnh bài toán và Mục tiêu hệ thống

1.1.1 Bối cảnh kỹ thuật: Chuyển dịch sang kiến trúc bất đồng bộ hướng sự kiện

Trong kỷ nguyên của các hệ thống phân tán, điện toán đám mây và vi dịch vụ (Microservices), khối lượng công việc đặt lên các cổng giao tiếp mạng (API Gateways, Load Balancers) và máy chủ dịch vụ ngày càng gia tăng với tốc độ đột biến. Các mô hình lập trình đồng bộ truyền thống dựa trên cơ chế phân luồng đa luồng (*Multi-threading with Blocking I/O*) bộc lộ những giới hạn nghiêm trọng:

- **Quá tải chi phí chuyển đổi ngữ cảnh (Context Switching Overhead):** Khi số lượng kết nối đồng thời chạm ngưỡng hàng chục nghìn (bài toán C10K/C1000K), hệ điều hành phải dành phần lớn chu kỳ CPU cho việc hoán đổi không gian thanh ghi và ngăn xếp luồng (Thread Stack), làm suy giảm nghiêm trọng thông lượng hữu ích.
- **Cạn kiệt tài nguyên bộ nhớ:** Mỗi luồng hệ điều hành tiêu tốn một vùng bộ nhớ cố định (thường từ 2MB đến 8MB cho stack frame). Việc duy trì một lượng lớn luồng chờ I/O thụ động dẫn đến cạn kiệt tài nguyên bộ nhớ (OOM – Out of Memory).

Để giải quyết triệt để vấn đề này, các nền tảng kỹ thuật hiện đại (Node.js, Go Goroutines, Python Asynchronous I/O – `asyncio`) đã chuyển dịch mạnh mẽ sang mô hình kiến trúc phi nghẽn, hướng sự kiện (*Non-blocking Event-Driven Architecture*) dựa trên Vòng lặp sự kiện (Event Loop) và cơ chế Đa nhiệm cộng tác (Cooperative Multitasking). Trong kiến trúc này, luồng thực thi chính không bao giờ bị phong tỏa (block) khi thực hiện các cuộc gọi I/O; thay vào đó, tác vụ được đóng gói dưới dạng đối tượng ủy thác tương lai (*Promise / Future*) và tiếp tục nhường quyền điều khiển CPU cho các tác vụ khác.

Bài toán thách thức đặt ra: Mặc dù kiến trúc bất đồng bộ giải phóng tài nguyên CPU khỏi việc chờ đợi I/O, nó lại chuyển dịch áp lực sang các hàng đợi nhiệm vụ (Task Queues / In-memory Buffers). Khi tốc độ luồng yêu cầu gửi đến (λ) vượt quá tốc độ xử lý của trạm phục vụ (μ), các tác vụ bất đồng bộ sẽ nhanh chóng tích lũy trong bộ đệm. Nếu không được kiểm

soát thông qua các mô hình điều phối hàng đợi khoa học, hệ thống sẽ đối mặt với tình trạng phình to độ trễ (Bufferbloat), tràn bộ đệm (Queue Overflow) và sụp đổ dịch vụ dây chuyền.

1.1.2 Mục tiêu và phạm vi đánh giá hiệu năng

Xuất phát từ yêu cầu cấp thiết trên, đề tài này được triển khai nhằm đạt được các mục tiêu kỹ thuật cụ thể sau:

1. **Xây dựng bộ mô phỏng sự kiện rời rạc chuẩn công nghiệp (DES Engine):** Thiết kế và hiện thực một khung mô phỏng DES hướng đối tượng (OOP) theo chuẩn PDES của Richard Fujimoto, tích hợp sâu cùng thư viện `asyncio` trên nền tảng Python 3.11+, hỗ trợ đa dạng cấu trúc hàng đợi ưu tiên (Min-Heap, LadderQueue).

2. **Khảo sát và đánh giá toàn diện 6 mô hình hàng đợi đơn Kendall:** Khảo sát định lượng các chỉ số hiệu năng trên 6 trạm hàng đợi cơ bản: $M/M/1$, $M/M/1/K$, $M/M/\infty$, $M/M/n/n$ (Erlang-B Loss System), $M/M/c$ (Erlang-C Delay System) và $M/M/c/K$, đồng thời đối chiếu với mô hình thời gian phục vụ tất định $M/D/1$.

3. **Phân tích và mô phỏng mạng hàng đợi Jackson đa chặng:** Đánh giá mạng xử lý phân tán 4 trạm vật lý gồm luồng tuần tự (Pipeline) và rẽ nhánh xác suất (Branching Network) theo đúng kiến trúc tham chiếu của đề tài: Source $\rightarrow$ I/O $\rightarrow$ CPU $\rightarrow$ $\{\text{Printer}, \text{Disk}\}$ $\rightarrow$ Sink.

4. **Thẩm định chéo (Cross-Validation) giữa Mô phỏng và Giải tích:** Sử dụng các công thức giải tích kinh điển của Lý thuyết Hàng đợi làm giá trị chuẩn (Ground Truth) để kiểm chứng độ chính xác của bộ mô phỏng DES, đo lường sai số tương đối (Relative Error $\leq 5\%$) và độ rộng khoảng tin cậy 95% (Confidence Interval).

1.2 Dịch vụ hệ thống và Kết quả khả dĩ

Theo phương pháp luận Raj Jain, bước xác định ranh giới hệ thống yêu cầu đặc tả rõ các dịch vụ cung cấp và danh mục toàn bộ các kết quả khả dĩ mà một yêu cầu có thể trải qua khi luân chuyển trong hệ thống.

1.2.1 Đặc tả 4 dịch vụ cốt lõi của hệ thống

1. **Dịch vụ Tiếp nhận yêu cầu (Ingestion Service):** Đóng vai trò là cổng giao tiếp ngoại vi, chịu trách nhiệm tiếp nhận các gói tin / yêu cầu gửi đến từ môi trường ngoài theo quy trình ngẫu nhiên Poisson với tốc độ trung bình λ. Dịch vụ thực hiện gán định danh duy nhất (Task ID), đóng dấu thời gian vào hệ thống (t_{arrival}) và chuyển giao cho bộ phận điều phối.

2. **Dịch vụ Quản lý hàng đợi (Queuing / Buffer Management Service):** Quản lý không gian nhớ đệm của trạm phục vụ theo kỷ luật FCFS (First-Come, First-Served). Dịch vụ chịu trách nhiệm kiểm tra dung lượng đệm tối đa K. Nếu còn chỗ trống, yêu cầu được đẩy vào hàng chờ; nếu bộ đệm đã bão hòa, kích hoạt cơ chế hủy gói (Drop).

3. **Dịch vụ Xử lý tính toán và I/O (Processing Service):** Điều phối các tài nguyên vật lý (CPU core, kênh đọc/ghi ổ đĩa, máy in) để giải quyết tác vụ. Thời gian phục vụ của mỗi yêu cầu tuân theo phân phối ngẫu nhiên xác định (phân phối Mũ với tốc độ μ, hoặc phân phối Tất định D). Khi máy chủ bận, các yêu cầu tiếp theo phải xếp hàng.

4. **Dịch vụ Phản hồi và Báo hiệu bất đồng bộ (Notification Service):** Cầu nối giữa động cơ mô phỏng và ứng dụng phía người dùng. Khi một tác vụ hoàn tất tại trạm cuối cùng (Sink), dịch vụ này cập nhật trạng thái của đối tượng `asyncio.Future`, đánh thức các tiến trình đang chờ đợi (`await`) và ghi nhận số liệu thống kê vào Trace Log.

1.2.2 Bốn trạng thái kết quả khả dĩ (Possible Outcomes)

Đối với mỗi yêu cầu được bơm vào hệ thống, kết quả thực thi được phân loại chính xác vào một trong 4 trạng thái sau:

1. **Xử lý thành công (Success):** Yêu cầu đi qua trọn vẹn các chặng phục vụ cần thiết, hoàn thành xử lý và rời khỏi hệ thống đúng quy trình kỹ thuật. Toàn bộ thời gian lưu trú trong hệ thống (W) được ghi nhận đầy đủ.

2. **Đang chờ phục vụ (Queued):** Yêu cầu đã được tiếp nhận hợp lệ nhưng toàn bộ máy chủ đang bận; tác vụ hiện đang nằm trong hàng đợi chờ tới lượt xử lý theo trật tự FCFS.

3. **Bị chặn / Hủy gói do tràn đệm (Dropped / Blocked):** Yêu cầu gửi đến trạm phục vụ đúng thời điểm hệ thống đã đạt sức chứa tối đa (K khách hàng). Yêu cầu bị loại bỏ ngay lập tức, không được phục vụ và làm tăng biến đếm mất gói (N_{dropped}).

4. **Hết thời hạn chờ (Timeout / Aborted):** Áp dụng cho các kịch bản bất đồng bộ nâng cao có thiết lập giới hạn thời gian (Deadline). Nếu thời gian nằm trong hàng đợi vượt quá ngưỡng quy định, tác vụ bị hủy ngang để bảo toàn tài nguyên cho hệ thống.

1.3 Hệ thống chỉ số hiệu năng chính (KPIs)

Để đánh giá định lượng chất lượng phục vụ của hệ thống, 5 nhóm chỉ số hiệu năng chính được chuẩn hóa toán học như sau:

- **Thông lượng hệ thống (Throughput – X hoặc λ_{out}):** Số lượng yêu cầu hoàn tất xử lý thành công trên một đơn vị thời gian mô phỏng (T_{sim}):

$$X = \frac{N_{\text{completed}}}{T_{\text{sim}}} \quad \text{(yêu cầu/giây hoặc req/s)} \tag{1}$$

  Đối với các hệ thống không mất gói và đạt trạng thái dừng, thông lượng bằng tốc độ đến: $X = \lambda$. Đối với các hệ thống có tổn thất ($M/M/1/K$, $M/M/c/K$), thông lượng hiệu dụng là: $\lambda_{\text{eff}} = \lambda(1 - P_b)$.

- **Thời gian đáp ứng (W - Response Time) và Thời gian chờ (W_q - Waiting Time):**
  - Thời gian chờ trong hàng đợi (W_q): Khoảng thời gian từ khi yêu cầu vào hàng đợi đến thời điểm bắt đầu được phục vụ: $W_q = t_{\text{service_start}} - t_{\text{arrival}}$.
  - Thời gian đáp ứng toàn hệ thống (W): Tổng thời gian một yêu cầu trải qua trong trạm phục vụ, gồm thời gian chờ và thời gian thực thi (S):

$$W = W_q + S = t_{\text{departure}} - t_{\text{arrival}} \implies E[W] = E[W_q] + \frac{1}{\mu} \tag{2}$$

- **Số lượng yêu cầu trung bình trong hệ thống (L) và trong hàng đợi (L_q):**
  - $L_q = E[n_q]$: Số lượng yêu cầu trung bình đang nằm chờ trong bộ đệm.
  - $L = E[n]$: Tổng số lượng yêu cầu đang có mặt tại trạm phục vụ (bao gồm các yêu cầu đang chờ và các yêu cầu đang được máy chủ phục vụ): $L = L_q + L_s$.

- **Hệ số sử dụng máy chủ (Server Utilization – ρ và U):** Tỷ lệ phần trăm thời gian các máy chủ ở trạng thái làm việc bận rộn:

$$\rho = \frac{\lambda}{c \cdot \mu}, \quad U = \frac{\sum_{i=1}^{c} T_{\text{busy}}^{(i)}}{c \cdot T_{\text{sim}}} \tag{3}$$

  Điều kiện bắt buộc để hệ thống hàng đợi có bộ đệm vô hạn đạt trạng thái ổn định (Ergodic / Steady-state) là $\rho < 1$.

- **Xác suất mất gói / Chặn dịch vụ (Blocking / Drop Probability – P_b):** Tỷ lệ các yêu cầu bị loại bỏ do bộ đệm đầy trên tổng số yêu cầu đã gửi đến:

$$P_b = \frac{N_{\text{dropped}}}{N_{\text{arrived}}} \tag{4}$$

1.4 Bảng quy ước ký hiệu toán học và thuật ngữ kỹ thuật

Nhằm đảm bảo tính mạch lạc, chính xác và nhất quán xuyên suốt toàn bộ báo cáo, Bảng 1 tổng hợp đầy đủ các ký hiệu toán học, định nghĩa lý thuyết và đơn vị tính tương ứng.

Ký hiệu	Thuật ngữ Anh – Việt	Định nghĩa & Ý nghĩa vật lý	Đơn vị
τ	Inter-arrival time (Thời gian liên đến)	Khoảng thời gian ngẫu nhiên giữa 2 lần đến liên tiếp: $\tau_j = t_j - t_{j-1}$	giây (s)
λ	Arrival rate (Tốc độ đến)	Số lượng yêu cầu trung bình gửi đến hệ thống trong một đơn vị thời gian: $\lambda = 1/E[\tau]$	yêu cầu/s
s	Service time (Thời gian phục vụ)	Thời gian máy chủ xử lý hoàn tất một yêu cầu; kỳ vọng $E[s] = 1/\mu$	giây (s)

Ký hiệu	Thuật ngữ Anh – Việt	Định nghĩa & Ý nghĩa vật lý	Đơn vị
μ	Service rate (Tốc độ phục vụ)	Số lượng yêu cầu một máy chủ có thể xử lý trong một đơn vị thời gian: $\mu = 1/E[s]$	yêu cầu/s
c (m)	Number of servers (Số máy chủ)	Số lượng bộ xử lý / kênh phục vụ độc lập làm việc song song	máy chủ
K (B)	System capacity (Sức chứa tối đa)	Tổng số yêu cầu tối đa cho phép trong hệ thống (gồm cả đang xử lý và hàng đợi)	yêu cầu
SD	Service discipline (Kỷ luật phục vụ)	Quy tắc lựa chọn yêu cầu từ hàng đợi để phục vụ (FCFS, LCFS, PS, SPT, ...)	quy tắc
ρ	Traffic intensity / Utilization	Hệ số cường độ tải của trạm phục vụ: $\rho = \frac{\lambda}{c\mu}$	hệ số
a	Offered load (Tải đề nghị)	Lượng lưu lượng đến tính theo đơn vị Erlang: $a = \lambda/\mu = c\rho$	Erlang
n	System jobs (Số khách trong hệ thống)	Biến ngẫu nhiên chỉ tổng số khách hàng hiện diện trong trạm phục vụ	yêu cầu
n_q	Queue jobs (Số khách trong hàng đợi)	Biến ngẫu nhiên chỉ số lượng khách hàng đang chờ tại bộ đệm ($n = n_q + n_s$)	yêu cầu
n_s	Active servers (Số khách đang xử lý)	Số lượng máy chủ đang bận xử lý tác vụ tại một thời điểm ($n_s \leq c$)	yêu cầu
L	Mean number in system ($E[n]$)	Kỳ vọng toán học số lượng yêu cầu có mặt trong toàn bộ hệ thống	yêu cầu
L_q	Mean number in queue ($E[n_q]$)	Kỳ vọng toán học số lượng yêu cầu đang chờ trong hàng đợi	yêu cầu
r	Response time (Thời gian đáp ứng)	Biến ngẫu nhiên tổng thời gian lưu lại trong hệ thống: $r = w + s$	giây (s)
w	Waiting time (Thời gian chờ)	Biến ngẫu nhiên thời gian khách hàng phải chờ trong hàng đợi trước khi phục vụ	giây (s)
W	Mean response time ($E[r]$)	Kỳ vọng thời gian lưu trú trong hệ thống (Sojourn time): $W = L/\lambda_{\text{eff}}$	giây (s)
W_q	Mean waiting time ($E[w]$)	Kỳ vọng thời gian chờ đợi trong bộ đệm: $W_q = L_q/\lambda_{\text{eff}} = W - 1/\mu$	giây (s)
p_k	State probability ($P\{n = k\}$)	Xác suất dừng của hệ thống khi có chính xác k khách hàng có mặt	xác suất
p_0	Idle probability ($P\{n = 0\}$)	Xác suất hệ thống hoàn toàn rỗng (tất cả máy chủ đều rảnh rỗi)	xác suất

Ký hiệu	Thuật ngữ Anh – Việt	Định nghĩa & Ý nghĩa vật lý	Đơn vị
P_b	Blocking probability (Xác suất chặn)	Xác suất một yêu cầu mới đến bị từ chối do bộ đệm đầy (p_K hoặc $B(n,a)$)	xác suất
λ_{eff}	Effective arrival rate (Tốc độ hữu hiệu)	Tốc độ luồng yêu cầu thực tế được tiếp nhận vào phục vụ: $\lambda(1-P_b)$	yêu cầu/s
γ_i	External arrival rate (Tốc độ đến ngoài)	Tốc độ dòng Poisson từ nguồn ngoại vi đưa trực tiếp vào trạm i trong mạng Jackson	yêu cầu/s
r_{ij}	Routing probability (Xác suất rẽ nhánh)	Xác suất gói tin sau khi hoàn tất tại trạm i được chuyển tiếp tới trạm j	xác suất

Bảng 1: *Bảng quy ước ký hiệu toán học và thuật ngữ kỹ thuật chuẩn hóa*

1.5 Tham số hệ thống, Tải làm việc và Kế hoạch thực nghiệm

Theo bước 4 (List Parameters) và bước 5 (Select Factors and Levels) trong phương pháp luận Raj Jain [1], việc thiết kế thực nghiệm đòi hỏi sự phân tách tường minh giữa các đại lượng cấu hình hệ thống, đặc trưng của luồng tải làm việc, và các yếu tố chủ động biến thiên trong từng kịch bản đo đạc.

1.5.1 Tham số hệ thống (System Parameters)

Tham số hệ thống bao gồm các thuộc tính phần cứng và cấu hình phần mềm được duy trì cố định làm môi trường cơ sở (baseline) trong mỗi đợt đo đạc:

- **Số lượng máy chủ song song (c):** $c \in \{1, 2, 4, 8\}$ máy chủ. Đại diện cho sự tiến hóa kiến trúc từ các vi xử lý đơn lõi (Single-core), vi xử lý đa lõi (Multi-core) đến các cụm máy chủ dịch vụ phân tán (Server Cluster) cùng khai thác chung một hàng đợi nhiệm vụ.
- **Kích thước bộ đệm lưu trữ tối đa (K):** $K \in \{3, 5, 10, 20, 50, \infty\}$ yêu cầu. Mô phỏng các ngưỡng giới hạn tài nguyên bộ nhớ khả dụng (RAM / Queue buffer limit) trong hệ thống thực tế. Tham số này kích hoạt cơ chế phòng ngự tràn bộ đệm (Backpressure / Drop tail) khi hệ thống chạm ngưỡng tới hạn.
- **Tốc độ phục vụ chuẩn hóa của máy chủ (μ):** Được cấu hình ở hai mức tải chuẩn: $\mu = 10.0$ yêu cầu/giây (thời gian phục vụ trung bình $E[s] = 1/\mu = 100\,\text{ms}$) cho các bài đo độ trễ chi tiết, và $\mu = 2.0$ yêu cầu/giây ($E[s] = 500\,\text{ms}$) cho các kịch bản đo đạc lưu lượng Erlang.
- **Kỷ luật phục vụ hàng đợi (Service Discipline – SD):** Kỷ luật *First-Come, First-Served* (FCFS / FIFO) không ưu tiên (Non-preemptive), đảm bảo tính công bằng tuyệt đối cho các tiến trình logic xếp hàng.

Bảng 2 tổng hợp các tham số hệ thống then chốt, dải cấu hình và ý nghĩa kỹ thuật trong mô hình.

Tham số hệ thống	Ký hiệu	Phân loại	Giá trị cấu hình	Ý nghĩa kỹ thuật
Số máy chủ	c	System Param	$\{1, 2, 4, 8\}$	Cấu hình lõi CPU / Trạm xử lý song song
Dung lượng bộ đệm	K	System / Factor	$\{3, 5, 8, 12, 20, 50, \infty\}$	Ngưỡng dung lượng bộ nhớ RAM giới hạn
Tốc độ phục vụ	μ	System Param	$\{2.0, 10.0\}$ req/s	Năng lực tính toán của mỗi máy chủ
Xác suất rẽ nhánh	p	System Param	$p_{\text{printer}} = 0.3$, $p_{\text{disk}} = 0.7$	Cơ chế định tuyến ngẫu nhiên mạng Jackson
Kỷ luật phục vụ	SD	System Param	FCFS / FIFO	Kỷ luật phục vụ không ưu tiên

Bảng 2: *Bảng phân loại và cấu hình tham số hệ thống (System Parameters)*

1.5.2 Tham số tải làm việc (Workload Parameters)

Tham số tải làm việc mô tả các thuộc tính thống kê ngẫu nhiên của luồng yêu cầu đầu vào:

- **Tốc độ yêu cầu đến (λ):** Được điều chỉnh biến thiên liên tục theo công thức $\lambda = \rho \cdot c \cdot \mu$ nhằm quét toàn diện dải cường độ tải từ trạng thái rảnh rỗi đến tiệm cận bão hòa: $\rho \in [0.10, 0.95]$.
- **Quy luật phân phối dòng đến (Arrival Process):** Dòng Poisson thuần nhất (Homogeneous Poisson Process), khoảng cách thời gian giữa hai lần đến liên tiếp tuân theo phân phối Mũ: $\tau \sim \text{Exponential}(\lambda)$. Thuộc tính này đảm bảo tính chất không nhớ (Memoryless) và tính chất PASTA (*Poisson Arrivals See Time Averages*).
- **Quy luật thời gian phục vụ (Service Process):**
  - *Phân phối Mũ ngẫu nhiên (M):* Thời gian xử lý $s \sim \text{Exponential}(\mu)$ cho 6 mô hình hàng đợi chuẩn Kendall ($M/M/1$, $M/M/1/K$, $M/M/\infty$, $M/M/n/n$, $M/M/c$, $M/M/c/K$).
  - *Phân phối Tất định hằng số (D):* Thời gian xử lý cố định $s = 1/\mu$ cho mô hình đối chứng $M/D/1$, phục vụ việc đánh giá tác động của độ biến thiên thời gian phục vụ lên độ trễ hàng đợi theo định lý Pollaczek-Khinchine.

Bảng 3 tổng kết các tham số tải làm việc, yếu tố nghiên cứu và điều kiện thống kê cho toàn bộ kế hoạch thực nghiệm.

Tham số tải	Ký hiệu	Phân loại	Dải giá trị khảo sát	Ý nghĩa kỹ thuật
Tốc độ đến	λ	Workload / Factor	$[1.0, 9.5]$ req/s	Áp lực lưu lượng gửi đến từ môi trường ngoài
Hệ số sử dụng tải	ρ	Study Factor	$\{0.1, 0.3, 0.5, 0.7, 0.8, 0.9, 0.95\}$	Mức độ làm việc của hệ thống (tải rảnh $\rightarrow$ bão hòa)
Tải đề nghị Erlang	a	Study Factor	$[1.0, 20.0]$ Erlang	Khảo sát hệ thống mất gói $M/M/n/n$ (Erlang-B)
Thời gian mô phỏng	T_{sim}	Execution Param	$3000 - 10000$ giây ảo	Đảm bảo hệ thống đạt trạng thái dừng ổn định
Số lượt lặp độc lập	R	Statistical Param	$R \geq 30$ lượt chạy	Đảm bảo khoảng tin cậy 95% theo chuẩn Student-t

Bảng 3: *Bảng phân loại tham số tải làm việc và các yếu tố nghiên cứu (Workload Parameters & Study Factors)*

1.5.3 Thiết kế Kế hoạch thực nghiệm và 5 Kịch bản khảo sát cốt lõi

Để chứng minh độ tin cậy của bộ máy mô phỏng DES và đánh giá toàn diện các hiện tượng tắc nghẽn, kế hoạch thực nghiệm được xây dựng thành 5 kịch bản phân tầng chặt chẽ:

1. **Kịch bản 1 - Khảo sát hành vi phi tuyến tiệm cận bão hòa ($\rho \to 1$) trên $M/M/1$:**
   - **Mục tiêu:** Làm rõ hiện tượng trễ bùng nổ phi tuyến và sự suy sụp hiệu năng khi hệ số tải vượt ngưỡng an toàn ($\rho > 0.80$).
   - **Phương pháp:** Cố định năng lực máy chủ $\mu = 10.0$ req/s; quét biến thiên $\lambda \in \{1.0, 3.0, 5.0, 7.0, 8.0, 8.5, 9.0, 9.5\}$ req/s (tương ứng $\rho \in [0.10, 0.95]$).
   - **Chỉ số theo dõi:** Thời gian đáp ứng trung bình W và số khách trung bình L, đối chiếu trực tiếp với công thức giải tích $W = \frac{1}{\mu(1-\rho)}$ và $L = \frac{\rho}{1-\rho}$.
2. **Kịch bản 2 - Phân tích độ nhạy của dung lượng bộ đệm ($M/M/1/K$ và $M/M/c/K$):**
   - **Mục tiêu:** Giải quyết bài toán đánh đổi kỹ thuật (*Trade-off*) giữa Xác suất mất gói (P_b) và Thời gian trễ hàng đợi (W_q), nhận diện hiện tượng tràn đệm (*Bufferbloat*).
   - **Phương pháp:** Khảo sát dưới hai chế độ: Tải nặng ($\rho = 0.9$) và Quá tải ($\rho = 1.2$); quét biến thiên kích thước đệm $K \in \{3, 5, 8, 12, 20, 30, 50\}$ yêu cầu.
   - **Chỉ số theo dõi:** Xác suất rớt gói thực nghiệm $P_b = N_{\text{dropped}}/N_{\text{arrived}}$, thông lượng hữu hiệu $\lambda_{\text{eff}} = \lambda(1 - P_b)$ và thời gian chờ W_q.

3. **Kịch bản 3 - Tối ưu hóa kiến trúc máy chủ đa luồng ($M/M/c$ vs. $M/M/1$):**
   - **Mục tiêu:** Giải quyết bài toán thiết kế: *Nên trang bị 1 máy chủ siêu nhanh tốc độ $c\mu$ hay c máy chủ song song tốc độ μ chia sẻ chung một hàng đợi?*
   - **Phương pháp:** Cố định tổng năng lực phục vụ $C_{\text{total}} = c\mu = 20.0\,\text{req/s}$; so sánh cấu hình 1 máy chủ $(c = 1, \mu = 20)$ với các cụm đa máy chủ $(c = 2, \mu = 10)$, $(c = 4, \mu = 5)$.
   - **Chỉ số theo dõi:** Xác suất phải chờ xếp hàng $P(\text{Wait}) = P_C(c, a)$ (công thức Erlang-C) và thời gian lưu trú toàn trình W.

4. **Kịch bản 4 - Phân tích mạng hàng đợi Jackson và Nhận diện điểm nghẽn (Bottleneck):**
   - **Mục tiêu:** Phân tích luồng cân bằng mạng hàng đợi mở, định vị chính xác vị trí thắt cổ chai (*bottleneck*) giới hạn hiệu năng của toàn hệ thống.
   - **Phương pháp:** Cấu hình kiến trúc 4 trạm Source $\to$ I/O $\to$ CPU $\to$ {Printer, Disk} $\to$ Sink với $\mu_{\text{IO}} = 8.0, \mu_{\text{CPU}} = 10.0, \mu_{\text{Printer}} = 4.0, \mu_{\text{Disk}} = 6.0\,\text{req/s}$; xác suất rẽ nhánh sau CPU là $p_{\text{printer}} = 0.3, p_{\text{disk}} = 0.7$.
   - **Chỉ số theo dõi:** Hệ số tải cục bộ $\rho_i = \lambda_i/\mu_i$ tại từng trạm và thời gian trễ toàn trình đầu–cuối (*End-to-End Latency*).

5. **Kịch bản 5 - Thẩm định ảnh hưởng của phương sai thời gian phục vụ ($M/M/1$ vs. $M/D/1$):**
   - **Mục tiêu:** Kiểm chứng định lý Pollaczek–Khinchine (P–K) về tác động tiêu cực của độ biến thiên phục vụ lên độ trễ hàng đợi.
   - **Phương pháp:** So sánh mô hình phục vụ ngẫu nhiên phân phối Mũ ($\sigma_s^2 = 1/\mu^2$) với mô hình thời gian phục vụ tất định ($\sigma_s^2 = 0$) tại cùng tốc độ $\mu = 10.0\,\text{req/s}$.
   - **Chỉ số theo dõi:** Chứng minh thực nghiệm: khi triệt tiêu hoàn toàn phương sai, thời gian chờ trong hàng đợi giảm chính xác một nửa: $W_q(M/D/1) = \frac{1}{2}W_q(M/M/1)$.

1.6 Lựa chọn kỹ thuật đánh giá hiệu năng

Theo giáo trình Raj Jain [1], bước 6 trong quy trình đánh giá hiệu năng đòi hỏi sự lựa chọn khoa học giữa ba phương pháp luận: **Mô hình hóa giải tích (*Analytical Modeling*)**, **Đo đạc thực tế (*Direct Measurement*)** và **Mô phỏng sự kiện rời rạc (*Discrete-Event Simulation – DES*)**.

1.6.1 So sánh ba phương pháp luận đánh giá

- **Mô hình hóa giải tích (*Analytical Modeling*):**
  - **Bản chất kỹ thuật:** Giải hệ phương trình cân bằng dừng (*Global Balance Equations*) của chuỗi Markov (CTMC), công thức Pollaczek–Khinchine và mạng Jackson.

  - **Ưu điểm vượt trội:** Cho nghiệm đóng (*closed-form solution*) chính xác tuyệt đối (100%) với thời gian tính toán tức thì.
  - **Hạn chế cốt lõi:** Bị ràng buộc bởi các giả thiết ngẫu nhiên lý tưởng (Poisson, phân phối Mũ); bất lực trước các hành vi phi tuyến vi mô (vòng lặp sự kiện bất đồng bộ, cơ chế drop-tail khi tràn bộ đệm).
  - **Vai trò trong đề tài:** Đóng vai trò là hệ quy chiếu chuẩn (*Ground Truth*) để thẩm định chéo độ chính xác của bộ mô phỏng DES.

- **Đo đạc thực tế (*Direct Measurement*):**
  - **Bản chất kỹ thuật:** Triển khai tải thực nghiệm trên phần cứng thật và thu thập số liệu qua các công cụ đo chuyên dụng (Prometheus, eBPF, JMeter).
  - **Ưu điểm vượt trội:** Phản ánh chính xác tác động vật lý của CPU đa lõi, phân cấp bộ nhớ đệm (Cache) và chi phí chuyển đổi ngữ cảnh (*context switch*) của hệ điều hành.
  - **Hạn chế cốt lõi:** Chi phí thiết bị đắt đỏ, không khả thi khi hệ thống chưa được chế tạo; khó kiểm soát nhiễu mạng và tiềm ẩn nguy cơ gây sập hệ thống khi đẩy tải tới hạn ($P_b \to 1$).
  - **Vai trò trong đề tài:** Cung cấp các thông số kiểm chuẩn thực tế về thời gian xử lý và trễ vật lý cho mô hình.

- **Mô phỏng sự kiện rời rạc (*Discrete-Event Simulation – DES*):**
  - **Bản chất kỹ thuật:** Vận hành bằng cách duy trì hàng đợi sự kiện tương lai (`EventQueue`) và bước nhảy thời gian ảo, chuyển trạng thái hệ thống theo từng mốc biến cố ngẫu nhiên.
  - **Ưu điểm vượt trội:** Cho phép mô hình hóa chi tiết mọi tiến trình bất đồng bộ phức tạp của `asyncio` (Coroutines, TaskGroup); linh hoạt tùy biến thông số với chi phí thấp và an toàn tuyệt đối.
  - **Thách thức kỹ thuật:** Tồn tại sai số phương sai do lấy mẫu ngẫu nhiên; đòi hỏi kỹ thuật loại bỏ pha quá độ (*Warm-up*) và quy trình lặp thống kê ($R \geq 30$) nghiêm ngặt.
  - **Vai trò trong đề tài:** Phương pháp khảo sát thực nghiệm trọng tâm xuyên suốt 5 kịch bản phân tích hiệu năng.

Bảng 4 tổng hợp so sánh đa tiêu chí giữa ba phương pháp luận theo chuẩn Raj Jain.

Tiêu chí đánh giá	Mô hình Giải tích	Đo đạc Thực tế	Mô phỏng DES
Giai đoạn áp dụng	Mọi giai đoạn thiết kế	Hệ thống đã tồn tại vật lý	Mọi giai đoạn thiết kế
Thời gian & Chi phí	Nhanh; Chi phí rất thấp	Rất chậm; Chi phí rất cao	Trung bình; Chi phí thấp
Khả năng mở rộng	Giới hạn bởi độ phức tạp toán	Giới hạn bởi phần cứng vật lý	Rất cao, linh hoạt mọi tham số
Độ chính xác	Tuyệt đối (với giả thiết lý tưởng)	Cao nhất trên môi trường thực	Rất cao (tiệm cận lý thuyết)
Độ phức tạp toán	Rất cao khi mạng mở rộng	Không yêu cầu	Thấp đến trung bình
Mô hình hóa bất đồng bộ	Kém (không biểu diễn Event Loop)	Tốt nhưng khó kiểm soát vi mô	Tối ưu (khớp 100% coroutine)

Bảng 4: *So sánh 3 phương pháp đánh giá hiệu năng hệ thống theo chuẩn Raj Jain*

1.6.2 Chiến lược phương pháp luận kép (Dual-Methodology Strategy)

Nhận thức rõ ưu nhược điểm trên, đề tài áp dụng **Chiến lược phương pháp luận kép**: lấy **Mô phỏng DES** làm công cụ khảo sát thực nghiệm chủ đạo, kết hợp **Mô hình giải tích** làm hệ quy chiếu thẩm định chuẩn (*Ground Truth*):

- **Hệ quy chiếu chuẩn (Ground Truth):** Sử dụng công thức giải tích đóng để thẩm định chéo bộ mô phỏng DES trên 6 mô hình Kendall chuẩn, bảo đảm sai số tương đối không vượt quá ngưỡng quy định ($\leq 3\% - 5\%$).
- **Không gian thực nghiệm mở rộng (DES Extensibility):** Sau khi kiểm chứng độ chính xác, bộ mô phỏng DES được dùng để phân tích các bài toán phi tuyến vượt ngoài khả năng giải tích: điều phối đồng quy `TaskGroup`, `as_completed`, phân tích tải đột biến (*Bursty Traffic*) và phân vị trễ đuôi (P_{95}, P_{99}).
- **Kiểm chuẩn tính dừng và Độ tin cậy thống kê:**
  - *Loại bỏ pha quá độ (Warm-up Period):* Loại bỏ 500 yêu cầu đầu tiên để triệt tiêu ảnh hưởng khởi động từ trạng thái rỗng, chỉ thu thập số liệu khi hệ thống đã đạt trạng thái dừng (*Steady-state*).
  - *Chạy lặp độc lập (Independent Replications):* Thực hiện $R \geq 30$ lượt chạy với seed ngẫu nhiên độc lập để triệt tiêu phương sai lấy mẫu, tính toán Khoảng tin cậy 95% (*Confidence Interval*) theo phân phối Student-t, đảm bảo tính chuẩn xác và tái lập của kết quả.

2 Cơ sở lý thuyết và Mô hình hóa hàng đợi

Nền tảng toán học giải tích của Lý thuyết Hàng đợi (Queueing Theory) giữ vai trò là chuẩn đối chiếu tuyệt đối (Ground Truth) nhằm kiểm chứng và thẩm định độ chính xác của bộ mô phỏng biến cố rời rạc (DES). Khung lý thuyết được xây dựng tuần tự và chặt chẽ qua các cấu phần kỹ thuật cốt lõi:

- **Ký hiệu Kendall ($A/S/c/K/N/SD$):** Chuẩn hóa và phân loại cấu trúc hệ thống hàng đợi, dung lượng, nguồn phát và kỷ luật phục vụ.
- **Tiến trình Poisson và Sinh–Tử:** Vận dụng tính không nhớ (Memoryless), xây dựng phương trình cân bằng Markov và xác định phân phối trạng thái dừng p_0, p_n.
- **Các định luật tổng quát ($G/G/c$):** Thiết lập điều kiện ổn định $\rho < 1$, Định luật Little ($L = \lambda_{\text{eff}} W$) và quan hệ phân rã $L = L_q + L_s$.
- **Các mô hình hàng đợi giải tích:** Dẫn xuất công thức dạng đóng cho $M/M/1$, $M/M/1/K$, $M/M/\infty$, Erlang-B, Erlang-C, $M/M/c/K$ và đối chứng $M/D/1$.
- **Mạng hàng đợi Jackson 4 trạm:** Giải hệ phương trình lưu lượng, phân tích nút thắt cổ chai (Bottleneck) và áp dụng nghiệm dạng tích độc lập (Product-Form Solution).

2.1 Tổng quan lý thuyết hàng đợi và Ký hiệu Kendall chuẩn hóa

2.1.1 Cấu trúc tổng thể của một hệ thống hàng đợi

Một hệ thống hàng đợi điển hình trong công nghệ máy tính và truyền thông được hợp thành từ 3 thành phần vật lý cơ bản:

1. **Dòng khách hàng (Customer Stream / Arrival Process):** Tập hợp các thực thể (gói tin mạng, luồng I/O, tiến trình tính toán) đến hệ thống theo một quy trình ngẫu nhiên. Thời điểm đến của khách thứ j là t_j. Khoảng cách giữa 2 lần đến liên tiếp được xác định bởi biến ngẫu nhiên $\tau_j = t_j - t_{j-1}$. Chuỗi $\{\tau_j\}$ thường được giả định là các biến ngẫu nhiên độc lập và cùng phân phối (IID – Independent and Identically Distributed).
2. **Hàng đợi và Kỷ luật phục vụ (Queue & Service Discipline):** Vùng nhớ đệm lưu trữ các yêu cầu chưa thể phục vụ ngay. Kỷ luật phục vụ quy định trật tự lựa chọn khách hàng từ hàng đợi:
    - **FCFS (First-Come, First-Served):** Đến trước được phục vụ trước (kỷ luật mặc định trong hầu hết các bộ đệm mạng).
    - **LCFS (Last-Come, First-Served):** Đến sau được phục vụ trước (ứng dụng trong cấu trúc ngăn xếp / stack).
    - **LCFS-PR (LCFS with Preemption and Resume):** Đến sau ngắt quyền phục vụ của khách hiện tại và chiếm dụng máy chủ ngay lập tức.

- **RR (Round-Robin) & PS (Processor Sharing):** Phân bổ lát cắt thời gian (Quantum). Khi kích thước quantum tiến về 0, mô hình trở thành chia sẻ bộ vi xử lý đồng thời (PS).
- **IS (Infinite Server):** Vô hạn máy chủ, mỗi khách đến đều có riêng một máy chủ phục vụ, không bao giờ phải chờ đệm ($W_q = 0$).
- **SPT / SRPT (Shortest Processing / Remaining Processing Time):** Ưu tiên tác vụ có thời gian thực thi ngắn nhất nhằm tối thiểu hóa độ trễ trung bình.

3. **Cơ sở phục vụ (Service Facility):** Bao gồm c máy chủ ($c \geq 1$) làm việc song song. Thời gian phục vụ của khách hàng là biến ngẫu nhiên s với kỳ vọng $E[s] = 1/\mu$, tương ứng tốc độ phục vụ μ trên mỗi máy chủ. Tổng năng lực phục vụ tối đa của trạm là $c\mu$.

2.1.2 Hệ thống ký hiệu Kendall mở rộng

Để chuẩn hóa việc phân loại các mô hình hàng đợi, D. G. Kendall đã đề xuất ký pháp chuẩn $A/S/c/B/K/SD$ với quy ước ý nghĩa như sau:

- A **(Arrival Process):** Phân phối xác suất của khoảng thời gian giữa 2 lần đến liên tiếp (τ).
- S **(Service Time Distribution):** Phân phối xác suất thời gian phục vụ một yêu cầu (s).
- c **(m – Number of Servers):** Số lượng máy chủ phục vụ làm việc đồng thời ($c \in \{1, 2, \ldots, \infty\}$).
- B **(System Capacity / Buffer Limit):** Sức chứa tối đa của hệ thống (bao gồm các chỗ đang xử lý trên máy chủ và các vị trí chờ trong bộ đệm). Mặc định $B = \infty$ nếu lược bỏ.
- K **(Population Size):** Tổng quy mô khách hàng của nguồn phát. Mặc định $K = \infty$ nếu lược bỏ.
- SD **(Service Discipline):** Kỷ luật phục vụ hàng đợi. Mặc định là FCFS nếu lược bỏ.

Các ký hiệu phân phối xác suất chuẩn bao gồm:

- M **(Markovian / Exponential):** Phân phối mũ, tương ứng dòng đến là dòng Poisson có tính chất không nhớ.
- D **(Deterministic):** Phân phối tất định, thời gian là một hằng số cố định không đổi.
- E_k **(Erlang-k):** Phân phối Erlang bậc k, mô hình hóa chuỗi k công đoạn phục vụ nối tiếp có phân phối mũ.
- H_k **(Hyper-exponential-k):** Phân phối siêu mũ, hỗn hợp xác suất của k kênh phân phối mũ độc lập.
- G **(General):** Phân phối tổng quát bất kỳ (áp dụng đúng cho mọi loại hàm phân phối).

Ví dụ: Ký hiệu $M/M/3/20/1500/\text{FCFS}$ đại diện cho hệ thống có thời gian đến và thời gian phục vụ phân phối mũ, gồm 3 máy chủ, tổng sức chứa 20 khách (3 máy chủ +17 ghế chờ), quy mô nguồn phát tối đa 1500 khách và kỷ luật FCFS. Khi lược bỏ các tham số mặc định, ký hiệu $M/M/1$ biểu diễn hệ thống $M/M/1/\infty/\infty/\text{FCFS}$.

2.1.3 Tính chất không nhớ (Memoryless Property) của phân phối Mũ

Để chuẩn hóa việc phân loại các mô hình hàng đợi, D. G. Kendall đã đề xuất ký pháp chuẩn $A/S/c/B/K/SD$ với quy ước ý nghĩa như sau: Phân phối mũ là nền tảng toán học duy nhất cho phép mô hình hóa hệ thống bằng chuỗi Markov thời gian liên tục. Một biến ngẫu nhiên $X \geq 0$ tuân theo phân phối mũ với tham số λ có hàm phân phối tích lũy (CDF) và hàm mật độ xác suất (PDF) lần lượt là:

$$F(t) = P(X \leq t) = 1 - e^{-\lambda t}, \quad f(t) = \lambda e^{-\lambda t} \quad (t \geq 0) \tag{5}$$

Định lý: Tính chất không nhớ của phân phối Mũ

Biến ngẫu nhiên X có tính chất không nhớ nếu và chỉ nếu với mọi $s, t \geq 0$:

$$P(X > s + t \mid X > t) = P(X > s) \tag{6}$$

Chứng minh:
Theo công thức xác suất có điều kiện:

$$P(X > s + t \mid X > t) = \frac{P(\{X > s + t\} \cap \{X > t\})}{P(X > t)} = \frac{P(X > s + t)}{P(X > t)} \tag{7}$$

Thay biểu thức hàm sống sót $P(X > x) = 1 - F(x) = e^{-\lambda x}$:

$$P(X > s + t \mid X > t) = \frac{e^{-\lambda(s+t)}}{e^{-\lambda t}} = \frac{e^{-\lambda s} \cdot e^{-\lambda t}}{e^{-\lambda t}} = e^{-\lambda s} = P(X > s) \quad \text{(đpcm)} \tag{8}$$

Ý nghĩa kỹ thuật:

Tính chất không nhớ khẳng định rằng khoảng thời gian còn lại cho đến khi sự kiện tiếp theo xảy ra hoàn toàn độc lập với lượng thời gian đã trôi qua trong quá khứ. Việc ghi nhớ lịch sử hoạt động không đem lại bất kỳ thông tin nào để dự đoán thời điểm kết thúc phục vụ hay thời điểm khách kế tiếp đến, giúp đơn giản hóa trạng thái hệ thống thành một chuỗi Markov không có biến trễ.

2.2 Tiến trình ngẫu nhiên và Thuộc tính của dòng Poisson

2.2.1 Khái niệm tiến trình ngẫu nhiên và Chuỗi Markov

Một tiến trình ngẫu nhiên (Stochastic Process) là một họ các biến ngẫu nhiên $\{X(t), t \in T\}$ biểu diễn sự tiến hóa trạng thái của hệ thống theo trục thời gian t.

- Nếu không gian trạng thái là rời rạc, ký hiệu $n(t) \in \{0, 1, 2, \dots\}$ biểu diễn số lượng khách hàng trong hệ thống tại thời điểm $t \rightarrow$ **Tiến trình trạng thái rời rạc (Discrete-state process)**.
- Nếu không gian trạng thái là liên tục, ký hiệu $w(t) \in [0, \infty)$ biểu diễn thời gian chờ $\rightarrow$ **Tiến trình trạng thái liên tục (Continuous-state process)**.
- **Tiến trình Markov (Markov Process):** Là tiến trình mà trạng thái tương lai chỉ phụ thuộc duy nhất vào trạng thái hiện tại, hoàn toàn độc lập với toàn bộ quỹ đạo quá khứ:

$$P(X(t_{k+1}) = x_{k+1} \mid X(t_k) = x_k, \dots, X(t_0) = x_0) = P(X(t_{k+1}) = x_{k+1} \mid X(t_k) = x_k) \quad (9)$$

  Một tiến trình Markov có không gian trạng thái rời rạc được gọi là **Chuỗi Markov (Markov Chain)**.

2.2.2 Đặc trưng và 5 thuộc tính cốt lõi của Dòng Poisson

Khi khoảng thời gian liên đến τ là các biến ngẫu nhiên IID phân phối mũ tham số λ, số lượng yêu cầu k xuất hiện trong khoảng thời gian có độ dài t tuân theo phân phối Poisson:

$$P\{N(t) = k\} = \frac{(\lambda t)^k}{k!} e^{-\lambda t}, \quad k = 0, 1, 2, \dots \quad (10)$$

Trong lý thuyết hàng đợi và mô hình hóa mạng, dòng Poisson sở hữu 5 thuộc tính toán học then chốt:

1. **Thuộc tính gộp dòng (Merging Property):** Hợp của m dòng Poisson độc lập có tốc độ lần lượt là $\lambda_1, \lambda_2, \dots, \lambda_m$ tạo thành một dòng Poisson duy nhất với tốc độ tổng hợp bằng tổng các tốc độ thành phần:
$$\lambda = \sum_{i=1}^{m} \lambda_i \quad (11)$$
2. **Thuộc tính tách dòng (Splitting Property):** Nếu một dòng Poisson tốc độ λ được phân chia ngẫu nhiên vào m nhánh con với xác suất tương ứng $p_1, p_2, \dots, p_m$ ($\sum p_i = 1$), thì mỗi dòng con i cũng là một dòng Poisson độc lập với tốc độ:
$$\lambda_i = p_i \cdot \lambda \quad (12)$$

3. **Thuộc tính co giãn thời gian (Scaling Property):** Nếu khoảng thời gian τ được nhân với hệ số co giãn $c > 0$, biến ngẫu nhiên $c\tau$ vẫn tuân theo phân phối mũ với tốc độ điều chỉnh bằng λ/c.

4. **Định lý Burke (Burke's Theorem):** Đối với một trạm hàng đợi $M/M/c$ ở trạng thái dừng với tốc độ đến $\lambda < c\mu$, luồng các yêu cầu rời khỏi trạm (Departure Process) cũng chính là một dòng Poisson thuần nhất có tốc độ đúng bằng λ. Hơn nữa, tại bất kỳ thời điểm t, số lượng khách đang có mặt trong hệ thống độc lập với chuỗi các thời điểm rời đi trước thời điểm t.

5. **Dòng ra của hệ thống đa máy chủ:** Khi dòng vào một cụm m máy chủ là Poisson tốc độ λ và các máy chủ có thời gian phục vụ phân phối mũ tốc độ μ_i, dòng ra tổng hợp vẫn là dòng Poisson với tốc độ λ, với điều kiện ổn định $\lambda < \sum_{i=1}^{m} \mu_i$.

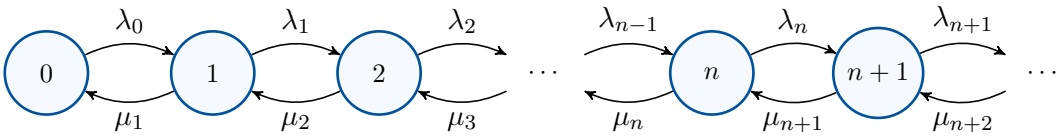
2.3 Tiến trình Sinh-Tử (Birth-Death Processes) và Cân bằng dừng

2.3.1 Mô hình toán học của Tiến trình Sinh-Tử

Tiến trình Sinh-Tử (Birth-Death Process) là trường hợp đặc biệt của chuỗi Markov thời gian liên tục, trong đó các bước chuyển trạng thái chỉ giới hạn giữa các trạng thái lân cận:

- **Sự kiện Sinh (Birth / Arrival):** Làm số lượng khách trong hệ thống tăng thêm 1 đơn vị ($n \to n+1$) với tốc độ chuyển dịch λ_n.
- **Sự kiện Tử (Death / Departure):** Khách hoàn tất dịch vụ rời khỏi hệ thống, làm giảm số lượng khách đi 1 đơn vị ($n \to n-1$) với tốc độ chuyển dịch μ_n.

Sơ đồ chuyển dịch trạng thái toàn cục của tiến trình Sinh-Tử được mô tả trong Hình 1.



Hình 1: *Sơ đồ chuyển dịch tốc độ trạng thái trong Tiến trình Sinh-Tử tổng quát*

2.3.2 Phương trình cân bằng dòng toàn cục và Nghiệm xác suất dừng

Ở trạng thái cân bằng thống kê (Steady State), nguyên lý bảo toàn lưu lượng yêu cầu: *Tốc độ chuyển dịch rời khỏi một tập trạng thái phải bằng tốc độ chuyển dịch đi vào tập trạng thái đó.*

Thiết lập phương trình cân bằng trên vết cắt giữa hai trạng thái lân cận $n-1$ và n:

$$\lambda_{n-1} p_{n-1} = \mu_n p_n \implies p_n = \frac{\lambda_{n-1}}{\mu_n} p_{n-1} \tag{13}$$

Bằng phương pháp truy hồi liên tiếp từ trạng thái rỗng p_0:

$$p_1 = \frac{\lambda_0}{\mu_1} p_0 \tag{14}$$

$$p_2 = \frac{\lambda_1}{\mu_2} p_1 = \frac{\lambda_0 \lambda_1}{\mu_1 \mu_2} p_0 \tag{15}$$

$$p_n = \left(\prod_{i=0}^{n-1} \frac{\lambda_i}{\mu_{i+1}} \right) p_0, \quad n = 1, 2, \ldots, \infty \tag{16}$$

Để tìm xác suất hệ thống rỗng p_0, áp dụng điều kiện chuẩn hóa tổng xác suất trên toàn không gian trạng thái bằng 1:

$$\sum_{n=0}^{\infty} p_n = 1 \implies p_0 \left[1 + \sum_{n=1}^{\infty} \left(\prod_{i=0}^{n-1} \frac{\lambda_i}{\mu_{i+1}} \right) \right] = 1 \tag{17}$$

Từ đó suy ra biểu thức tổng quát của p_0:

$$p_0 = \left[1 + \sum_{n=1}^{\infty} \left(\prod_{i=0}^{n-1} \frac{\lambda_i}{\mu_{i+1}} \right) \right]^{-1} \tag{18}$$

Hệ thống tồn tại phân phối dừng ổn định khi và chỉ khi chuỗi vô hạn trong mẫu số của (18) hội tụ.

2.4 Các nguyên lý phổ quát cho mọi hệ thống hàng đợi ($G/G/c$ Rules)

Các định luật sau đây có giá trị phổ quát cho mọi hệ thống hàng đợi Kendall dạng $G/G/c$, độc lập với loại hàm phân phối của thời gian đến và thời gian phục vụ:

2.4.1 Điều kiện ổn định hệ thống (Stability Condition)

Đối với hệ thống có bộ đệm vô hạn ($B = \infty$), điều kiện cần và đủ để hệ thống không bị tràn vô hạn là tốc độ đến phải nhỏ hơn tổng tốc độ phục vụ tối đa của các máy chủ:

$$\lambda < c \cdot \mu \iff \rho = \frac{\lambda}{c\mu} < 1 \tag{19}$$

Các hệ thống có kích thước nguồn phát hữu hạn ($K < \infty$) hoặc có dung lượng đệm hữu hạn ($B < \infty$) luôn luôn ổn định ở mọi giá trị λ vì các yêu cầu vượt ngưỡng sẽ bị chặn hoặc nguồn phát bị cạn kiệt.

2.4.2 Quan hệ số lượng trong hệ thống và hàng đợi

Tại bất kỳ thời điểm nào, tổng số khách hàng trong hệ thống (n) bằng số khách đang chờ trong hàng đợi (n_q) cộng số khách đang được phục vụ tại các máy chủ (n_s):

$$n = n_q + n_s \implies E[n] = E[n_q] + E[n_s] \iff L = L_q + L_s \tag{20}$$

Nếu tốc độ phục vụ của máy chủ độc lập với số lượng yêu cầu đang chờ trong hàng đợi, thì hiệp phương sai $\text{Cov}(n_q, n_s) = 0$, dẫn đến quan hệ cộng phương sai:

$$\text{Var}[n] = \text{Var}[n_q] + \text{Var}[n_s] \tag{21}$$

2.4.3 Định luật Little (Little's Law, Little 1961)

Định luật Little

Đối với bất kỳ hệ thống hàng đợi nào đạt trạng thái dừng với tốc độ đến trung bình λ:

$$L = \lambda \cdot W \iff E[n] = \lambda \cdot E[r] \tag{22}$$

Định luật Little áp dụng được cho toàn bộ hệ thống hoặc cho bất kỳ phân hệ con độc lập nào:

- Áp dụng cho vùng hàng đợi: $L_q = \lambda \cdot W_q \iff E[n_q] = \lambda \cdot E[w]$.
- Áp dụng cho cụm máy chủ phục vụ: $L_s = \lambda \cdot E[s] = \frac{\lambda}{\mu} = c\rho$.

Lưu ý quan trọng:

Trong các hệ thống có hiện tượng mất gói ($M/M/1/K$, $M/M/c/K$), biến λ trong định luật Little phải được thay thế bằng thông lượng hiệu dụng thực tế đi vào hệ thống: $\lambda_{\text{eff}} = \lambda(1 - P_b)$.

2.4.4 Quan hệ thời gian lưu trú và thời gian chờ

Thời gian một khách hàng lưu lại trong trạm phục vụ (r) bằng thời gian chờ trong hàng đợi (w) cộng thời gian được máy chủ phục vụ (s):

$$r = w + s \implies E[r] = E[w] + E[s] \iff W = W_q + \frac{1}{\mu} \tag{23}$$

Nếu thời gian phục vụ s độc lập với thời gian chờ w ($\text{Cov}(w, s) = 0$):

$$\text{Var}[r] = \text{Var}[w] + \text{Var}[s] \tag{24}$$

2.5 Phân tích chi tiết 6 mô hình hàng đợi đơn chuẩn hóa và mô hình $M/D/1$

2.5.1 Mô hình $M/M/1$: Hệ thống đơn máy chủ, bộ đệm vô hạn

Mô hình $M/M/1$ giả định dòng đến là dòng Poisson tốc độ λ, thời gian phục vụ phân phối mũ với tham số μ, 1 máy chủ duy nhất ($c = 1$), bộ đệm và nguồn phát vô hạn ($B = \infty, K = \infty$), kỷ luật FCFS.

Phương trình cân bằng dừng:

Với mọi trạng thái n, tốc độ sinh và tử là hằng số: $\lambda_n = \lambda$ $(n \geq 0)$ và $\mu_n = \mu$ $(n \geq 1)$. Đặt hệ số tải trọng $\rho = \lambda/\mu$. Áp dụng (16):

$$p_n = \left(\frac{\lambda}{\mu}\right)^n p_0 = \rho^n p_0 \tag{25}$$

Sử dụng điều kiện chuẩn hóa với chuỗi cấp số nhân khi $\rho < 1$:

$$\sum_{n=0}^{\infty} p_n = p_0 \sum_{n=0}^{\infty} \rho^n = \frac{p_0}{1-\rho} = 1 \implies p_0 = 1 - \rho \tag{26}$$

Do đó, phân phối xác suất số lượng khách trong hệ thống có dạng:

$$p_n = (1-\rho)\rho^n, \quad n = 0, 1, 2, \ldots \tag{27}$$

Dẫn xuất các chỉ số hiệu năng:

- Số lượng khách trung bình trong hệ thống ($L = E[n]$):

$$L = \sum_{n=0}^{\infty} n p_n = (1-\rho) \sum_{n=0}^{\infty} n\rho^n = (1-\rho)\frac{\rho}{(1-\rho)^2} = \frac{\rho}{1-\rho} = \frac{\lambda}{\mu - \lambda} \tag{28}$$

- Phương sai số lượng khách trong hệ thống:

$$\mathrm{Var}[n] = E[n^2] - (E[n])^2 = \frac{\rho}{(1-\rho)^2} \tag{29}$$

- Thời gian đáp ứng trung bình ($W = E[r]$): Theo định luật Little $L = \lambda W$:

$$W = \frac{L}{\lambda} = \frac{1}{\mu(1-\rho)} = \frac{1}{\mu - \lambda} \tag{30}$$

- Thời gian chờ trung bình trong hàng đợi ($W_q = E[w]$):

$$W_q = W - \frac{1}{\mu} = \frac{1}{\mu - \lambda} - \frac{1}{\mu} = \frac{\lambda}{\mu(\mu - \lambda)} = \frac{\rho}{\mu(1-\rho)} \tag{31}$$

- Số lượng khách trung bình trong hàng đợi ($L_q = E[n_q]$):

$$L_q = \lambda W_q = \frac{\lambda^2}{\mu(\mu-\lambda)} = \frac{\rho^2}{1-\rho} = L - \rho \tag{32}$$

- Hàm phân phối tích lũy của thời gian đáp ứng r và thời gian chờ w:

$$F(r) = 1 - e^{-r\mu(1-\rho)}, \quad r \geq 0 \tag{33}$$
$$F(w) = 1 - \rho e^{-w\mu(1-\rho)}, \quad w \geq 0 \tag{34}$$

- Bách phân vị thứ q (q-percentile, ví dụ $q = 90$ hoặc $q = 95$):

$$r_q = \frac{1}{\mu(1-\rho)} \ln\left(\frac{100}{100-q}\right), \quad w_q = \max\left\{0,\ \frac{1}{\mu(1-\rho)} \ln\left(\frac{100\rho}{100-q}\right)\right\} \tag{35}$$

Bộ công thức giải tích chuẩn cho Mô hình $M/M/1$

Điều kiện ổn định:	$\rho = \frac{\lambda}{\mu} < 1, \quad c = 1, \quad B = \infty$
Phân phối trạng thái dừng:	$p_0 = 1 - \rho, \qquad p_n = (1-\rho)\rho^n$
Số khách trong hệ thống:	$L = \frac{\rho}{1-\rho} = \frac{\lambda}{\mu-\lambda}, \qquad L_q = \frac{\rho^2}{1-\rho}$
Thời gian đáp ứng & chờ:	$W = \frac{1}{\mu-\lambda}, \qquad W_q = \frac{\rho}{\mu-\lambda} = \frac{\lambda}{\mu(\mu-\lambda)}$
Xác suất có $\geq k$ khách:	$P(n \geq k) = \sum_{j=k}^{\infty} p_j = \rho^k$

2.5.2 Mô hình $M/M/1/K$: Hệ thống đơn máy chủ, dung lượng đệm hữu hạn

Hệ thống có 1 máy chủ, tổng dung lượng tối đa là K khách (1 khách đang được phục vụ và $K-1$ chỗ chờ trong bộ đệm). Khi hệ thống đạt K khách, mọi yêu cầu tiếp theo đến sẽ bị từ chối và hủy bỏ ngay lập tức (Drop / Loss).

Phương trình cân bằng dừng:

Không gian trạng thái hữu hạn $\{0, 1, 2, \ldots, K\}$. Tốc độ chuyển dịch:

$$\lambda_n = \begin{cases} \lambda & \text{khi } 0 \leq n < K \\ 0 & \text{khi } n \geq K \end{cases}, \qquad \mu_n = \mu \quad (1 \leq n \leq K) \tag{36}$$

Xác suất trạng thái có dạng: $p_n = \rho^n p_0$ $(0 \le n \le K)$. Chuẩn hóa tổng xác suất:

$$\sum_{n=0}^{K} p_n = p_0 \sum_{n=0}^{K} \rho^n = p_0 \frac{1-\rho^{K+1}}{1-\rho} = 1 \implies p_0 = \frac{1-\rho}{1-\rho^{K+1}} \quad (\text{với } \rho \neq 1) \tag{37}$$

Khi $\rho = 1$, phân phối trở thành đều: $p_n = \frac{1}{K+1}$.

Dẫn xuất các chỉ số hiệu năng:

- Xác suất chặn / Mất gói do đầy bộ đệm (P_b):

$$P_b = p_K = \frac{(1-\rho)\rho^K}{1-\rho^{K+1}} \tag{38}$$

- Thông lượng hiệu dụng thực tế (λ_{eff}):

$$\lambda_{\text{eff}} = \lambda(1-P_b) = \lambda(1-p_K) \tag{39}$$

- Số lượng khách trung bình trong hệ thống (L):

$$L = \sum_{n=0}^{K} n p_n = \frac{\rho}{1-\rho} - \frac{(K+1)\rho^{K+1}}{1-\rho^{K+1}} \tag{40}$$

- Thời gian đáp ứng trung bình (W) và thời gian chờ (W_q):

$$W = \frac{L}{\lambda_{\text{eff}}} = \frac{L}{\lambda(1-P_b)}, \quad W_q = W - \frac{1}{\mu}, \quad L_q = \lambda_{\text{eff}} W_q = L - (1-p_0) \tag{41}$$

Bộ công thức giải tích chuẩn cho Mô hình $M/M/1/K$

Tham số:	$c = 1$, Sức chứa: K, Không gian trạng thái: $n \in \{0, 1, \ldots, K\}$
Xác suất rỗng p_0:	$p_0 = \frac{1-\rho}{1-\rho^{K+1}}$ (nếu $\rho = 1$ thì $p_0 = \frac{1}{K+1}$)
Phân phối trạng thái dừng:	$p_n = \frac{(1-\rho)\rho^n}{1-\rho^{K+1}}, \quad 0 \le n \le K$
Xác suất mất gói (Chặn):	$P_b = p_K = \frac{(1-\rho)\rho^K}{1-\rho^{K+1}}$
Thông lượng hiệu dụng:	$\lambda_{\text{eff}} = \lambda(1-P_b)$
Số khách trong hệ thống:	$L = \frac{\rho}{1-\rho} - \frac{(K+1)\rho^{K+1}}{1-\rho^{K+1}}$
Chỉ số thời gian (Little):	$W = \frac{L}{\lambda_{\text{eff}}}, \quad W_q = W - \frac{1}{\mu}, \quad L_q = \lambda_{\text{eff}} W_q$

2.5.3 Mô hình $M/M/\infty$: Hệ thống vô hạn máy chủ (Pure Delay System)

Mô hình $M/M/\infty$ đại diện cho trạm phục vụ tự do (*Self-service facility*) hoặc trạm có độ trễ cố định (Fixed delay network link). Mỗi khi có một yêu cầu đến, hệ thống luôn sẵn sàng cấp phát một máy chủ mới để phục vụ ngay lập tức. Khách hàng không bao giờ phải chờ trong hàng đợi ($W_q = 0, L_q = 0$).

Phương trình cân bằng dừng:

Tốc độ sinh là hằng số $\lambda_n = \lambda$ $(n \geq 0)$. Vì có n khách đang được phục vụ đồng thời bởi n máy chủ độc lập, tốc độ kết thúc phục vụ tổng hợp tỷ lệ thuận với số khách: $\mu_n = n\mu$ $(n \geq 1)$. Áp dụng (16) với tải đề nghị $a = \lambda/\mu$:

$$p_n = p_0 \prod_{i=0}^{n-1} \frac{\lambda}{(i+1)\mu} = p_0 \frac{(\lambda/\mu)^n}{n!} = p_0 \frac{a^n}{n!} \tag{42}$$

Chuẩn hóa tổng xác suất:

$$\sum_{n=0}^{\infty} p_n = p_0 \sum_{n=0}^{\infty} \frac{a^n}{n!} = p_0 e^a = 1 \implies p_0 = e^{-a} = e^{-\lambda/\mu} \tag{43}$$

Phân phối số lượng khách trong hệ thống chính là **phân phối Poisson** tham số a:

$$p_n = \frac{a^n}{n!} e^{-a} = \frac{(\lambda/\mu)^n}{n!} e^{-\lambda/\mu}, \quad n = 0, 1, 2, \ldots \tag{44}$$

Bộ công thức giải tích chuẩn cho Mô hình $M/M/\infty$

Tham số:	$c = \infty, \quad B = \infty, \quad$ Tải đề nghị: $a = \dfrac{\lambda}{\mu}$
Phân phối trạng thái dừng:	$p_n = \dfrac{a^n}{n!} e^{-a}, \qquad p_0 = e^{-a}$
Số khách trong hệ thống:	$L = E[n] = a = \dfrac{\lambda}{\mu}, \qquad \mathrm{Var}[n] = a$
Chỉ số hàng đợi:	$L_q = 0, \qquad W_q = 0$
Thời gian đáp ứng:	$W = E[s] = \dfrac{1}{\mu}$

2.5.4 Mô hình $M/M/n/n$: Hệ thống Erlang-B tổn thất (Loss System)

Hệ thống gồm n máy chủ làm việc song song nhưng hoàn toàn **không có bộ đệm chờ** ($B = n$). Khi một khách hàng đến, nếu có ít nhất một máy chủ rảnh rỗi, khách được tiếp nhận phục vụ ngay. Nếu toàn bộ n máy chủ đều bận, yêu cầu bị từ chối và chặn mất gói ngay lập tức. Đây là mô hình kinh điển do A. K. Erlang phát triển cho mạng điện thoại thoại cổ điển.

Phương trình cân bằng dừng:

Không gian trạng thái gồm $n+1$ mức: $\{0, 1, \ldots, n\}$.

$$\lambda_k = \begin{cases} \lambda & \text{khi } 0 \le k < n \\ 0 & \text{khi } k \ge n \end{cases}, \qquad \mu_k = k\mu \quad (1 \le k \le n) \tag{45}$$

Biểu thức xác suất trạng thái:

$$p_k = p_0 \frac{(\lambda/\mu)^k}{k!} = p_0 \frac{a^k}{k!} \quad (0 \le k \le n) \tag{46}$$

Chuẩn hóa tổng xác suất bằng 1:

$$\sum_{k=0}^{n} p_k = p_0 \sum_{k=0}^{n} \frac{a^k}{k!} = 1 \implies p_0 = \left[\sum_{k=0}^{n} \frac{a^k}{k!} \right]^{-1} \tag{47}$$

Từ đó suy ra phân phối xác suất dừng:

$$p_k = \frac{\dfrac{a^k}{k!}}{\displaystyle\sum_{i=0}^{n} \frac{a^i}{i!}}, \quad k = 0, 1, \ldots, n \tag{48}$$

Công thức mất gói Erlang-B (Erlang's Loss Formula):

Xác suất chặn dịch vụ chính là xác suất toàn bộ n máy chủ đều bận (p_n):

$$P_b = B(n, a) = p_n = \frac{\dfrac{a^n}{n!}}{\displaystyle\sum_{i=0}^{n} \frac{a^i}{i!}} \tag{49}$$

Giải thuật tính toán đệ quy ổn định số học (Numerically Stable Recursion):

Khi n lớn hoặc a lớn, việc tính toán trực tiếp $a^n/n!$ sẽ dẫn đến lỗi tràn số dấu phẩy động (*Overflow*). Trong mã nguồn phần mềm mô phỏng, công thức truy hồi sau được áp dụng:

$$B(0, a) = 1, \qquad B(k, a) = \frac{a \cdot B(k-1, a)}{k + a \cdot B(k-1, a)} \quad \text{với } k = 1, 2, \ldots, n \tag{50}$$

Bộ công thức giải tích chuẩn cho Mô hình $M/M/n/n$ (Erlang-B)

Tham số:	$c = n$, Sức chứa: $B = n$ (0 chỗ chờ đệm), $a = \dfrac{\lambda}{\mu}$
Phân phối trạng thái dừng:	$p_k = \dfrac{a^k/k!}{\sum_{i=0}^{n} a^i/i!}, \quad 0 \le k \le n$
Xác suất chặn Erlang-B:	$P_b = B(n,a) = p_n = \dfrac{a^n/n!}{\sum_{i=0}^{n} a^i/i!}$
Thuật toán đệ quy ổn định:	$B(k,a) = \dfrac{aB(k-1,a)}{k + aB(k-1,a)}, \quad B(0,a) = 1$
Tải phục vụ thực tế:	$A_c = a(1 - P_b) = L$
Chỉ số hàng đợi & thời gian:	$L_q = 0, \quad W_q = 0, \quad W = \dfrac{1}{\mu}$

2.5.5 Mô hình $M/M/c$: Hệ thống đa máy chủ, bộ đệm vô hạn (Erlang-C Delay)

Hệ thống có c máy chủ giống nhau ($c \ge 2$) cùng chia sẻ một hàng đợi chung vô hạn ($B = \infty$). Khách đến theo dòng Poisson tốc độ λ. Tốc độ phục vụ của mỗi máy chủ là μ.

Phương trình cân bằng dừng:

Tốc độ sinh $\lambda_n = \lambda$ với mọi $n \ge 0$. Tốc độ kết thúc phục vụ phụ thuộc vào số máy chủ đang hoạt động:

$$\mu_n = \begin{cases} n\mu & \text{khi } 1 \le n < c \\ c\mu & \text{khi } n \ge c \end{cases} \tag{51}$$

Hệ số cường độ tải: $\rho = \frac{\lambda}{c\mu} < 1$. Tải đề nghị: $a = \frac{\lambda}{\mu} = c\rho$. Áp dụng (16), biểu thức xác suất trạng thái là:

$$p_n = \begin{cases} \dfrac{a^n}{n!}p_0 = \dfrac{(c\rho)^n}{n!}p_0 & \text{khi } 0 \le n < c \\ \dfrac{a^n}{c!c^{n-c}}p_0 = \dfrac{c^c\rho^n}{c!}p_0 & \text{khi } n \ge c \end{cases} \tag{52}$$

Xác định xác suất hệ thống rỗng p_0 từ điều kiện $\sum_{n=0}^{\infty} p_n = 1$:

$$p_0 = \left[\sum_{k=0}^{c-1}\frac{(c\rho)^k}{k!} + \sum_{n=c}^{\infty}\frac{(c\rho)^c}{c!}\rho^{n-c}\right]^{-1} = \left[\sum_{k=0}^{c-1}\frac{(c\rho)^k}{k!} + \frac{(c\rho)^c}{c!(1-\rho)}\right]^{-1} \tag{53}$$

Công thức trễ Erlang-C (Erlang's Delay Formula):

Xác suất một yêu cầu mới đến thấy tất cả c máy chủ đều bận và buộc phải xếp hàng chờ

trong bộ đệm ($P(W_q > 0) = P(n \geq c)$):

$$C(c,a) = \sum_{n=c}^{\infty} p_n = \frac{(c\rho)^c}{c!(1-\rho)} p_0 = \frac{\dfrac{(c\rho)^c}{c!(1-\rho)}}{\displaystyle\sum_{k=0}^{c-1} \frac{(c\rho)^k}{k!} + \frac{(c\rho)^c}{c!(1-\rho)}} \tag{54}$$

Các chỉ số hiệu năng:

- Số lượng khách trung bình trong hàng đợi (L_q):

$$L_q = \sum_{n=c}^{\infty} (n-c) p_n = \frac{\rho}{1-\rho} C(c,a) \tag{55}$$

- Thời gian chờ trung bình trong hàng đợi (W_q):

$$W_q = \frac{L_q}{\lambda} = \frac{C(c,a)}{c\mu - \lambda} \tag{56}$$

- Thời gian đáp ứng trung bình (W) và tổng số khách trong hệ thống (L):

$$W = W_q + \frac{1}{\mu} = \frac{C(c,a)}{c\mu - \lambda} + \frac{1}{\mu}, \qquad L = \lambda W = L_q + \frac{\lambda}{\mu} = L_q + c\rho \tag{57}$$

Bộ công thức giải tích chuẩn cho Mô hình $M/M/c$ (Erlang-C)

Điều kiện ổn định:	$\rho = \dfrac{\lambda}{c\mu} < 1, \quad a = \dfrac{\lambda}{\mu} = c\rho, \quad B = \infty$
Xác suất rỗng p_0:	$p_0 = \left[\displaystyle\sum_{k=0}^{c-1} \frac{(c\rho)^k}{k!} + \frac{(c\rho)^c}{c!(1-\rho)}\right]^{-1}$
Xác suất chờ Erlang-C:	$C(c,a) = \dfrac{(c\rho)^c}{c!(1-\rho)} p_0$
Thời gian chờ trong hàng đợi:	$W_q = \dfrac{C(c,a)}{c\mu - \lambda}$
Thời gian đáp ứng:	$W = W_q + \dfrac{1}{\mu} = \dfrac{C(c,a)}{c\mu - \lambda} + \dfrac{1}{\mu}$
Số khách trong hệ thống:	$L_q = \dfrac{\rho}{1-\rho} C(c,a), \qquad L = L_q + c\rho$

2.5.6 Mô hình $M/M/c/K$: Hệ thống đa máy chủ, dung lượng đệm hữu hạn

Mô hình tổng quát gồm c máy chủ phục vụ song song với tổng sức chứa tối đa là K khách ($K \geq c$). Bộ đệm có $K - c$ vị trí chờ. Khi hệ thống có đúng K khách, mọi yêu cầu tiếp theo sẽ bị loại bỏ ($P_b = p_K$).

Phương trình cân bằng dừng:

Không gian trạng thái hữu hạn $\{0, 1, 2, \ldots, K\}$. Tốc độ chuyển dịch:

$$\lambda_n = \begin{cases} \lambda & \text{khi } 0 \leq n < K \\ 0 & \text{khi } n \geq K \end{cases}, \qquad \mu_n = \begin{cases} n\mu & \text{khi } 1 \leq n \leq c \\ c\mu & \text{khi } c < n \leq K \end{cases} \tag{58}$$

Phân phối xác suất trạng thái dừng:

$$p_n = \begin{cases} \dfrac{(c\rho)^n}{n!} p_0 & \text{khi } 0 \leq n \leq c \\ \dfrac{c^c \rho^n}{c!} p_0 & \text{khi } c < n \leq K \end{cases} \tag{59}$$

với $\rho = \frac{\lambda}{c\mu}$. Xác định p_0 từ điều kiện $\sum_{n=0}^{K} p_n = 1$:

$$p_0 = \left[\sum_{k=0}^{c-1} \frac{(c\rho)^k}{k!} + \frac{(c\rho)^c}{c!} \sum_{n=c}^{K} \rho^{n-c} \right]^{-1} = \left[\sum_{k=0}^{c-1} \frac{(c\rho)^k}{k!} + \frac{(c\rho)^c (1 - \rho^{K-c+1})}{c!(1-\rho)} \right]^{-1} \tag{60}$$

(với trường hợp $\rho = 1$, số hạng tổng hình học được thay bằng $K - c + 1$).

Bộ công thức giải tích chuẩn cho Mô hình $M/M/c/K$

Tham số: c máy chủ, $K \geq c$, Ghế chờ: $K - c$, $\rho = \dfrac{\lambda}{c\mu}$

Xác suất rỗng p_0: $p_0 = \left[\sum_{k=0}^{c-1} \dfrac{(c\rho)^k}{k!} + \dfrac{(c\rho)^c (1 - \rho^{K-c+1})}{c!(1-\rho)} \right]^{-1}$

Xác suất mất gói: $P_b = p_K = \dfrac{c^c \rho^K}{c!} p_0, \qquad \lambda_{\text{eff}} = \lambda(1 - P_b)$

Số khách hàng đợi: $L_q = \dfrac{p_0 (c\rho)^c \rho}{c!(1-\rho)^2} \left[1 - \rho^{K-c+1} - (1-\rho)(K-c+1)\rho^{K-c} \right]$

Chỉ số thời gian (Little): $W_q = \dfrac{L_q}{\lambda_{\text{eff}}}, \qquad W = W_q + \dfrac{1}{\mu}, \qquad L = \lambda_{\text{eff}} W$

2.5.7 Mô hình đối chứng $M/D/1$: Thời gian phục vụ tất định

Trong thực tế kỹ thuật, nhiều thiết bị mạng và phần cứng tính toán có thời gian xử lý cố định không đổi (ví dụ: chuyển tiếp khung tin Ethernet kích thước chuẩn qua cổng vật lý cố định). Mô hình $M/D/1$ có dòng đến Poisson nhưng thời gian phục vụ là một hằng số tất định: $s = 1/\mu$, tương ứng phương sai thời gian phục vụ $\sigma_s^2 = 0$.

Công thức Pollaczek-Khinchine (P-K Formula):

Đối với hàng đợi tổng quát $M/G/1$, công thức Pollaczek-Khinchine xác định thời gian chờ trung bình trong hàng đợi là:

$$W_q = \frac{\lambda E[s^2]}{2(1-\rho)} = \frac{\lambda(\sigma_s^2 + (1/\mu)^2)}{2(1-\rho)} \tag{61}$$

- Đối với $M/M/1$: Thời gian phục vụ phân phối mũ nên $\sigma_s^2 = (1/\mu)^2 \implies E[s^2] = 2/\mu^2$. Thay vào công thức P-K thu được:

$$W_q^{(M/M/1)} = \frac{\lambda(2/\mu^2)}{2(1-\rho)} = \frac{\rho}{\mu(1-\rho)} \tag{62}$$

- Đối với $M/D/1$: Thời gian phục vụ tất định nên $\sigma_s^2 = 0 \implies E[s^2] = 1/\mu^2$. Thay vào công thức P-K:

$$W_q^{(M/D/1)} = \frac{\lambda(1/\mu^2)}{2(1-\rho)} = \frac{\rho}{2\mu(1-\rho)} = \frac{1}{2}W_q^{(M/M/1)} \tag{63}$$

So sánh sâu sắc: Nhờ loại bỏ hoàn toàn tính biến động (phương sai) của thời gian phục vụ ($\sigma_s^2 = 0$), *thời gian chờ đợi trong hàng đợi của mô hình $M/D/1$ giảm chính xác 50% so với mô hình $M/M/1$ tại cùng một mức tải trọng ρ*!

Bộ công thức giải tích chuẩn cho Mô hình $M/D/1$ (So sánh $M/M/1$)

Tham số:	$c = 1, \quad s = \frac{1}{\mu}$ (Hằng số), $\quad \sigma_s^2 = 0, \quad \rho = \frac{\lambda}{\mu} < 1$
Thời gian chờ trong hàng đợi:	$W_q = \frac{\rho}{2\mu(1-\rho)} = \frac{1}{2}W_q^{(M/M/1)}$
Thời gian đáp ứng:	$W = \frac{1}{\mu} + \frac{\rho}{2\mu(1-\rho)}$
Số khách trong hàng đợi:	$L_q = \frac{\rho^2}{2(1-\rho)}$
Số khách trong hệ thống:	$L = \rho + \frac{\rho^2}{2(1-\rho)}$

2.6 Mô hình mạng hàng đợi Jackson (Jackson Queueing Networks)

2.6.1 Cấu trúc mạng hàng đợi mở (Open Jackson Networks)

Trong các hệ thống phân tán thực tế, một yêu cầu sau khi được xử lý tại trạm này có thể được định tuyến chuyển tiếp sang các trạm khác trước khi rời khỏi hệ thống. Một mạng hàng đợi mở gồm M trạm phục vụ được gọi là **Mạng Jackson** nếu thỏa mãn 4 điều kiện:

1. Mỗi trạm $i \in \{1, 2, \ldots, M\}$ tiếp nhận một dòng yêu cầu từ nguồn ngoại vi theo quy trình Poisson độc lập với tốc độ $\gamma_i \geq 0$.
2. Mỗi trạm i có c_i máy chủ phục vụ với thời gian phục vụ phân phối mũ tốc độ μ_i, kỷ luật phục vụ FCFS và bộ đệm vô hạn.
3. Một yêu cầu hoàn tất tại trạm i được chuyển tiếp tới trạm j với xác suất chuyển đổi $r_{ij} \geq 0$, hoặc rời khỏi mạng hoàn toàn với xác suất $d_i = 1 - \sum_{j=1}^{M} r_{ij}$. Ma trận $P = [r_{ij}]_{M \times M}$ là ma trận xác suất chuyển đổi tuyến đường.
4. Các quyết định định tuyến là độc lập thống kê với trạng thái quá khứ và hiện tại của các hàng đợi.

2.6.2 Hệ phương trình cân bằng lưu lượng (Traffic Equations)

Để mạng đạt trạng thái dừng ổn định, tổng tốc độ dòng yêu cầu đi vào trạm i (λ_i) phải bằng tổng tốc độ luồng ngoại vi γ_i cộng các luồng chuyển tiếp từ tất cả các trạm j trong mạng:

$$\lambda_i = \gamma_i + \sum_{j=1}^{M} \lambda_j \cdot r_{ji}, \quad i = 1, 2, \ldots, M \tag{64}$$

Viết dưới dạng phương trình ma trận: $\boldsymbol{\lambda}^T = \boldsymbol{\gamma}^T + \boldsymbol{\lambda}^T P \implies \boldsymbol{\lambda}^T = \boldsymbol{\gamma}^T (I - P)^{-1}$.

Định lý Jackson (Jackson's Theorem, 1957)

Nếu mọi trạm trong mạng đều thỏa mãn điều kiện ổn định $\rho_i = \frac{\lambda_i}{c_i \mu_i} < 1$ $(\forall i = 1, \ldots, M)$, thì phân phối xác suất đồng thời của số lượng yêu cầu $(n_1, n_2, \ldots, n_M)$ tại các trạm ở trạng thái dừng có **dạng tích độc lập (Product-Form Solution)**:

$$P(n_1, n_2, \ldots, n_M) = \prod_{i=1}^{M} p_i(n_i) \tag{65}$$

trong đó $p_i(n_i)$ chính là phân phối xác suất dừng của một trạm hàng đợi Kendall $M/M/c_i$ độc lập có tốc độ đến trung bình bằng λ_i và tốc độ phục vụ μ_i!

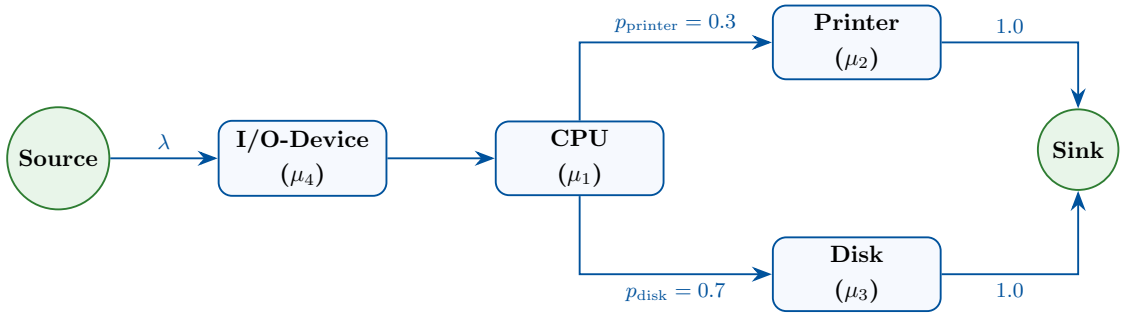
Ý nghĩa đột phá của Định lý Jackson:

Mặc dù trong thực tế các dòng chuyển tiếp giữa các nút trong mạng có thể không còn là dòng Poisson thuần nhất do tác động can thiệp của các trạm trước, Định lý Jackson chứng minh

rằng ở trạng thái cân bằng, *từng trạm trong mạng vẫn hành xử hoàn toàn giống hệt như một hàng đợi Poisson độc lập*!

2.6.3 Ứng dụng giải tích cho Mô hình mạng tham chiếu của đề tài

Kiến trúc mạng hàng đợi tham chiếu trong bài tập lớn được mô tả bởi sơ đồ đồ hình luồng sự kiện trong Hình 2.



Hình 2: *Sơ đồ cấu trúc mạng hàng đợi Jackson 4 trạm tham chiếu của đề tài*

Hệ thống bao gồm 4 trạm phục vụ đơn máy chủ ($c_i = 1$):

- **Trạm 1 - Thiết bị ngoại vi (I/O-Device):** Tốc độ phục vụ μ_4. Luồng đến ngoại vi $\gamma_{\text{IO}} = \lambda$.
- **Trạm 2 - Bộ vi xử lý (CPU Server):** Tốc độ phục vụ μ_1. Toàn bộ dòng từ I/O chuyển tiếp sang CPU: $r_{\text{IO,CPU}} = 1.0$.
- **Trạm 3 - Máy in (Printer):** Tốc độ phục vụ μ_2. CPU rẽ nhánh sang Printer với xác suất $p_{\text{printer}} = 0.3$.
- **Trạm 4 - Ổ đĩa lưu trữ (Disk Server):** Tốc độ phục vụ μ_3. CPU rẽ nhánh sang Disk với xác suất $p_{\text{disk}} = 0.7$.
- **Điểm đích (Sink):** Thu gom toàn bộ các gói tin hoàn tất từ Printer và Disk để ghi nhận kết quả.

Thiết lập hệ phương trình lưu lượng:

$$\lambda_{\text{IO}} = \lambda \tag{66}$$

$$\lambda_{\text{CPU}} = \lambda_{\text{IO}} = \lambda \tag{67}$$

$$\lambda_{\text{Printer}} = p_{\text{printer}} \cdot \lambda_{\text{CPU}} = 0.3 \cdot \lambda \tag{68}$$

$$\lambda_{\text{Disk}} = p_{\text{disk}} \cdot \lambda_{\text{CPU}} = 0.7 \cdot \lambda \tag{69}$$

Điều kiện ổn định và Nút thắt cổ chai (Bottleneck Node): Hệ số tải của từng trạm phục vụ là:

$$\rho_{\text{IO}} = \frac{\lambda}{\mu_4}, \quad \rho_{\text{CPU}} = \frac{\lambda}{\mu_1}, \quad \rho_{\text{Printer}} = \frac{0.3\lambda}{\mu_2}, \quad \rho_{\text{Disk}} = \frac{0.7\lambda}{\mu_3} \tag{70}$$

Điều kiện ổn định toàn mạng yêu cầu: $\max\{\rho_{\text{IO}}, \rho_{\text{CPU}}, \rho_{\text{Printer}}, \rho_{\text{Disk}}\} < 1$. Trạm có hệ số sử dụng lớn nhất $\rho_{\max} = \max_i(\rho_i)$ chính là **nút thắt cổ chai (Bottleneck)**. Khi tốc độ đến λ tăng dần, nút này sẽ chạm ngưỡng bão hòa ($\rho = 1$) đầu tiên, quyết định giới hạn thông lượng tối đa của toàn hệ thống: $\lambda_{\max} = \min\left\{\mu_4, \mu_1, \frac{\mu_2}{0.3}, \frac{\mu_3}{0.7}\right\}$.

Chỉ số hiệu năng toàn mạng (End-to-End Metrics):

Vì mỗi nút i độc lập đóng vai trò là một hàng đợi $M/M/1$, thời gian đáp ứng tại từng nút là $W_i = \frac{1}{\mu_i - \lambda_i}$.

- **Tổng số khách trung bình trong toàn mạng (L_{net}):**

$$L_{\text{net}} = L_{\text{IO}} + L_{\text{CPU}} + L_{\text{Printer}} + L_{\text{Disk}} = \frac{\rho_{\text{IO}}}{1-\rho_{\text{IO}}} + \frac{\rho_{\text{CPU}}}{1-\rho_{\text{CPU}}} + \frac{\rho_{\text{Printer}}}{1-\rho_{\text{Printer}}} + \frac{\rho_{\text{Disk}}}{1-\rho_{\text{Disk}}} \quad (71)$$

- **Thời gian đáp ứng trung bình toàn trình (End-to-End Response Time – W_{net}):** Theo định luật Little mở rộng cho toàn mạng với tốc độ đến ngoại vi tổng $\gamma = \lambda$:

$$W_{\text{net}} = \frac{L_{\text{net}}}{\lambda} = W_{\text{IO}} + W_{\text{CPU}} + 0.3 \cdot W_{\text{Printer}} + 0.7 \cdot W_{\text{Disk}} \quad (72)$$

Bộ công thức giải tích chuẩn cho Mạng Jackson 4 trạm tham chiếu

Tốc độ luân chuyển:	$\lambda_{\text{IO}} = \lambda, \quad \lambda_{\text{CPU}} = \lambda, \quad \lambda_{\text{Printer}} = 0.3\lambda, \quad \lambda_{\text{Disk}} = 0.7\lambda$
Hệ số sử dụng máy chủ:	$\rho_i = \dfrac{\lambda_i}{\mu_i} < 1 \quad (\forall i \in \{\text{IO}, \text{CPU}, \text{Printer}, \text{Disk}\})$
Chỉ số tại trạm i:	$W_i = \dfrac{1}{\mu_i - \lambda_i}, \qquad L_i = \dfrac{\lambda_i}{\mu_i - \lambda_i} = \dfrac{\rho_i}{1-\rho_i}$
Tổng số khách toàn mạng:	$L_{\text{net}} = L_{\text{IO}} + L_{\text{CPU}} + L_{\text{Printer}} + L_{\text{Disk}}$
Thời gian đáp ứng toàn trình:	$W_{\text{net}} = W_{\text{IO}} + W_{\text{CPU}} + 0.3W_{\text{Printer}} + 0.7W_{\text{Disk}} = \dfrac{L_{\text{net}}}{\lambda}$

3 Kiến trúc và Hiện thực bộ mô phỏng DES

Nội dung tập trung làm rõ kiến trúc kỹ thuật và giải pháp hiện thực bộ mô phỏng sự kiện rời rạc (*Discrete-Event Simulation – DES*) trên nền tảng Python 3.11+ với thư viện `asyncio`, bao gồm các khía cạnh trọng tâm sau:

- **Mô hình hóa hệ thống:** Vận dụng phương pháp lập trình hướng đối tượng (OOP), tham khảo các nguyên lý của mô phỏng sự kiện rời rạc phân tán (PDES) theo Richard Fujimoto, kết hợp cơ chế xử lý bất đồng bộ nhằm tổ chức và điều phối các tiến trình mô phỏng.
- **Phạm vi và khả năng mô phỏng:** Triển khai sáu mô hình hàng đợi theo ký hiệu Kendall, từ $M/M/1$ đến $M/M/c/K$, cùng mạng hàng đợi Jackson gồm bốn trạm với cơ chế định tuyến theo xác suất.
- **Thiết kế và kiểm chứng:** Xây dựng hệ thống theo quy trình từ thiết kế kiến trúc bằng sơ đồ UML, đặc tả các thuật toán cốt lõi đến kiểm thử và đánh giá tính đúng đắn của cơ chế xử lý sự kiện, đặc biệt là tính nhân quả trong mô phỏng.

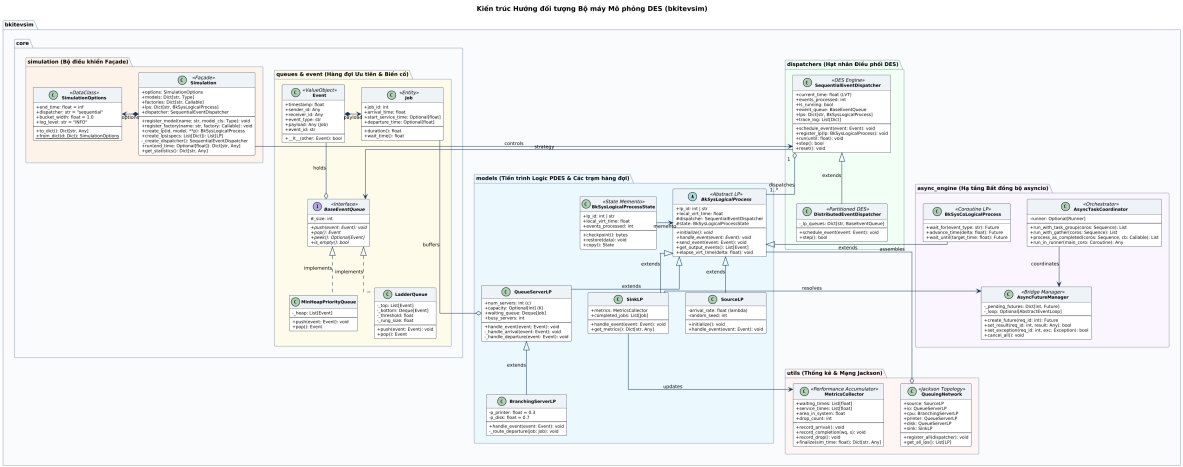
3.1 Thiết kế phần mềm và Mô hình hóa hướng đối tượng (UML Design)

Kiến trúc phần mềm của bộ máy mô phỏng tuân thủ nghiêm ngặt các nguyên lý SOLID, hướng đến tính kết dính cao (*High Cohesion*) và độ phụ thuộc lỏng (*Loose Coupling*), được phân rã thành 4 phân hệ trụ cột gắn kết hữu cơ:

1. **Phân hệ Điều khiển Mô phỏng Facade (*Simulation Controller Subsystem*):**
   - *Vai trò:* Cung cấp điểm tiếp xúc cấp cao duy nhất (*Single Point of Contact*) cho ứng dụng người dùng thông qua lớp mặt tiền `Simulation` và đối tượng cấu hình `SimulationOptions` theo mô hình Hình 4 đề bài.
   - *Trừu tượng hóa vận hành:* Che giấu toàn bộ độ phức tạp của việc cấp phát bộ điều phối `Dispatcher`, cấu hình hàng đợi `LadderQueue`, quản trị vòng đời và nạp tự động các tiến trình logic.
2. **Phân hệ Hạt nhân Điều phối (*Simulation Engine Subsystem*):**
   - *Hạt nhân điều hành trung tâm:* Quản lý vòng lặp DES, duy trì trục thời gian ảo toàn cục (t_{current}) và bộ đếm sự kiện qua `SequentialEventDispatcher` (tuần tự tập trung) và `DistributedEventDispatcher` (tuần tự phân vùng hàng đợi).
   - *Hàng đợi ưu tiên (`BaseEventQueue`):* Thiết kế theo mẫu *Strategy Pattern*, hoán đổi linh hoạt giữa cây `MinHeapPriorityQueue` ($\mathcal{O}(\log N)$) và cấu trúc phân tầng `LadderQueue` ($\boldsymbol{\mathcal{O}(1)}$ khấu hao).
   - *Kiểm chứng vết:* Ghi nhận nhật ký vết thực thi (`trace_log`) phục vụ kiểm chứng tính đơn điệu nhân quả ($t_{k+1} \geq t_k$).

3. **Phân hệ Tiến trình Logic (*Logical Process Subsystem – PDES*):**
   - *Mô hình hóa PDES:* Đóng gói khép kín trạng thái nội tại (`BkSysLogicalProcessState`) và kiểm soát đồng hồ ảo cục bộ *LVT* (*Local Virtual Time*) theo nguyên lý Fujimoto.
   - *Các trạm chức năng:* Kế thừa từ lớp cơ sở `BkSysLogicalProcess`, gồm trạm nguồn `SourceLP`, trạm phục vụ hợp nhất `QueueServerLP`, trạm rẽ nhánh ngẫu nhiên `BranchingServerLP`, trạm thu gom `SinkLP` và `NetworkSinkLP`.
   - *Cơ chế giao tiếp:* Trao đổi dữ liệu thuần túy qua các bản tin sự kiện `Event` mang thực thể `Job`, bảo đảm nguyên tắc phi chia sẻ trạng thái (*Shared-Nothing*).
4. **Phân hệ Cầu nối Bất đồng bộ & Động cơ Coroutine (*Async Integration Subsystem*):**
   - *Giao tiếp phi nghẽn luồng:* Cầu nối giữa thời gian ảo gián đoạn và môi trường bất đồng bộ qua `AsyncFutureManager` (`await` phi phong tỏa).
   - *Mẫu điều phối cấu trúc:* Chuẩn hóa 4 mẫu điều phối trong `AsyncTaskCoordinator`: `TaskGroup`, `gather`, `as_completed` và `Runner`.
   - *Động cơ Coroutine:* Cung cấp khung coroutine bất đồng bộ `BkSysCoLogicalProcess` và `BkSysCoEventLoop` với cơ chế `await` phi nghẽn.

3.1.1 Mô tả tổng quan sơ đồ cấu trúc lớp (UML Class Diagram)



Hình 3: *Sơ đồ cấu trúc lớp chi tiết (UML Class Diagram) của hệ thống mô phỏng DES*

Mô tả tổng quan sơ đồ lớp

Sơ đồ lớp chi tiết tại Hình 3 đặc tả toàn diện kiến trúc hướng đối tượng của bộ máy mô phỏng sự kiện rời rạc kết hợp điều phối tác vụ bất đồng bộ (*Discrete-Event Simulation with Asynchronous Task Coordination*). Hệ thống tổ chức phân tầng rõ rệt giữa bộ điều khiển Facade (`Simulation`, `SimulationOptions`), các dịch vụ điều phối sự kiện (`SequentialEventDispatcher`, `DistributedEventDispatcher`), cấu trúc hàng đợi ưu tiên (`BaseEventQueue`, `LadderQueue`),

các trạm tiến trình logic PDES (`SourceLP`, `QueueServerLP`, `BranchingServerLP`, `SinkLP`) cùng hạ tầng quản lý tác vụ bất đồng bộ (`AsyncFutureManager`, `AsyncTaskCoordinator`) và thu thập số đo hiệu năng thời gian thực (`MetricsCollector`).

Phân hệ Điều khiển Mô phỏng Cấp cao (Simulation Controller Facade)

- **SimulationOptions**: Lớp đóng gói toàn diện các tham số cấu hình môi trường mô phỏng.
  - *Thuộc tính:*
    + `end_time: float` - Mốc thời gian ảo kết thúc mô phỏng (mặc định: 100.0).
    + `dispatcher: str` - Tên loại bộ điều phối (`"sequential"` hoặc `"distributed_sequential"`).
    + `bucket_width: float` - Bề rộng nấc thang Δ cho hàng đợi ưu tiên `LadderQueue`.
    + `log_level: str` - Mức độ ghi nhật ký hệ thống (`"INFO"`, `"DEBUG"`,...).
    + `log_file: Optional[str]` - Đường dẫn tệp lưu trữ nhật ký ghi vết.
    + `stats_file: Optional[str]` - Đường dẫn tệp xuất kết quả thống kê số liệu.
    + `verbose: bool` - Cờ bật/tắt chế độ in chi tiết tiến trình thực thi ra màn hình console.
    + `model_params: Dict[str, Any]` - Từ điển chứa các tham số nghiệp vụ tùy biến bổ sung.
  - *Phương thức:*
    + `to_dict(): Dict[str, Any]` - Tuần tự hóa toàn bộ cấu hình sang đối tượng từ điển Python.
    + `from_dict(data: Dict[str, Any]): SimulationOptions` - Phương thức lớp tái tạo đối tượng cấu hình từ dữ liệu từ điển/JSON.
- **Simulation**: Lớp Facade mặt tiền điều phối cấp cao nhất, đóng vai trò là điểm chạm chính kết nối người dùng với bộ máy mô phỏng.
  - *Thuộc tính:*
    + `options: SimulationOptions` - Đối tượng lưu trữ các cấu hình tham số vận hành.
    + `models: Dict[str, Type[BkSysLogicalProcess]]` - Danh bạ đăng ký các lớp mô hình logic theo tên.
    + `factories: Dict[str, Callable[..., BkSysLogicalProcess]]` - Danh bạ các hàm xưởng khởi tạo LP.
    + `lps: Dict[Any, BkSysLogicalProcess]` - Bảng băm quản lý toàn bộ thực thể tiến trình logic theo `lp_id`.
    + `dispatcher: Optional[SequentialEventDispatcher]` - Con trỏ tham chiếu đến bộ điều phối đang hoạt động.
  - *Phương thức:*
    + `register_model(name, model_cls)`: Đăng ký lớp mô hình `BkSysLogicalProcess` vào danh bạ.

+ `register_factory(name: str, factory: Callable): void` - Đăng ký hàm xưởng khởi tạo LP tương ứng.
+ `create_lp(lp_id, model_name, **params)`: Khởi tạo một thực thể LP theo mô hình đã đăng ký.
+ `create_lps(model_name, count, **params)`: Khởi tạo danh sách các thực thể LP cùng kiểu hàng loạt.
+ `get_lp(lp_id: int | str): Optional[BkSysLogicalProcess]` - Truy xuất thực thể LP theo mã định danh.
+ `get_all_lps(): List[BkSysLogicalProcess]` - Trả về danh sách tất cả các LP đã đăng ký.
+ `_create_dispatcher(): SequentialEventDispatcher` - Tự động khởi tạo bộ điều phối tuần tự hoặc phân vùng tương ứng với cấu hình.
+ `run(): Dict[str, Any]` - Khởi chạy kịch bản mô phỏng: liên kết toàn bộ LP vào Dispatcher, vận hành vòng lặp DES và trả về thống kê.
+ `get_statistics(): Dict[str, Any]` - Thu gom và hợp nhất chỉ số thống kê từ Dispatcher và tất cả các trạm logic `per_lp_stats`.

Phân hệ Hạt nhân & Điều phối sự kiện

- **SequentialEventDispatcher (alias Dispatcher)**: Hạt nhân điều hành trung tâm quản lý vòng đời sự kiện theo thứ tự mốc thời gian ảo đơn điệu.
  - *Thuộc tính:*
    + `end_time: float` - Mốc thời gian ảo chặn trên để kết thúc mô phỏng.
    + `bucket_width: float` - Bề rộng nấc thang của hàng đợi `LadderQueue`.
    + `lps: Dict[Any, BkSysLogicalProcess]` - Bảng tra cứu danh bạ các tiến trình logic theo `lp_id`.
    + `event_queue: BaseEventQueue` - Hàng đợi sự kiện ưu tiên (`LadderQueue` hoặc `MinHeapPriorityQueue`).
    + `current_time: float` - Mốc thời gian ảo hiện tại của hệ thống (LVT toàn cục).
    + `events_processed: int` - Tổng số sự kiện đã được trích xuất và xử lý thành công.
    + `trace_log: List[Dict[str, Any]]` - Nhật ký ghi vết kiểm tra tính đơn điệu nhân quả ($t_{k+1} \geq t_k$).
    + `max_trace_entries: int` - Giới hạn dung lượng bộ nhớ nhật ký vết (mặc định 1000).
    + `stats: Dict[str, Any]` - Báo cáo thống kê thời gian ảo và thời gian thực máy tính.
    + `_initialized: bool` - Cờ trạng thái bảo đảm quy trình mồi sự kiện ban đầu `initialize()` chỉ chạy 1 lần.
  - *Phương thức:*
    + `register_lp(lp: BkSysLogicalProcess): void` - Đăng ký tiến trình logic vào danh bạ và liên kết Dispatcher.

+ `get_lp(lp_id: Any): Optional[BkSysLogicalProcess]` - Tra cứu trạm logic theo định danh.
+ `get_all_lps(): List[BkSysLogicalProcess]` - Lấy toàn bộ danh sách trạm logic đã đăng ký.
+ `schedule_event(event: Event): void` - Đẩy sự kiện mới vào hàng đợi sự kiện ưu tiên.
+ `schedule_initial_events(): void` - Duyệt toàn bộ LPs và kích hoạt phương thức `initialize()` mỗi sự kiện đầu tiên.
+ `run(end_time: Optional[float]): Dict[str, Any]` - Thực thi vòng lặp DES chính cho đến khi đạt `end_time` hoặc hàng đợi rỗng.

- **DistributedEventDispatcher**: Kế thừa từ `SequentialEventDispatcher`, hiện thực mô hình phân vùng hàng đợi cục bộ (*Partitioned Queues*).
  - *Thuộc tính:*
    + `_lp_queues: Dict[Any, LadderQueue]` - Bảng ánh xạ mỗi LP tới một hàng đợi `LadderQueue` cục bộ độc lập.
  - *Phương thức:*
    + `register_lp(lp: BkSysLogicalProcess): void` - Đăng ký LP và cấp phát một hàng đợi `LadderQueue` riêng biệt.
    + `schedule_event(event: Event): void` - Định tuyến sự kiện trực tiếp vào hàng đợi cục bộ của trạm đích `receiver_id`.
    + `run(end_time: Optional[float]): Dict[str, Any]` - Tại mỗi bước lặp, quét tìm LP có sự kiện sớm nhất trên toàn mạng để trích xuất và xử lý.

- **Event**: Đối tượng giá trị (*Value Object*) đóng gói thông điệp và ngữ cảnh trao đổi giữa các thành phần mô phỏng.
  - *Thuộc tính:*
    + `timestamp: float` - Mốc thời gian ảo kích hoạt sự kiện (tiêu chí so sánh tự nhiên duy nhất).
    + `sender_id: Any` (alias `source_lp`, `lp_source`) - Định danh trạm logic phát sự kiện (`compare=False`).
    + `receiver_id: Any` (alias `dest_lp`, `lp_target`) - Định danh trạm logic tiếp nhận sự kiện (`compare=False`).
    + `event_type: Any` - Kiểu sự kiện (`ARRIVAL`, `SERVICE_START`, `DEPARTURE`, `TERMINATE`) (`compare=False`).
    + `payload: Any` (alias `data`) - Tải trọng dữ liệu đi kèm (thường chứa thực thể `Job`) (`compare=False`).
    + `event_id: Any` - Mã định danh duy nhất của sự kiện dạng UUID4 tự sinh (`compare=False`).

+ `_priority: int` - Giá trị độ ưu tiên nghiệp vụ mặc định phục vụ tương thích ngược.

– *Phương thức:*

+ `__lt__(other: Event): bool` - So sánh thứ tự ưu tiên trích xuất dựa trên mốc thời gian ảo `timestamp` (`@dataclass(order=True)`).

- **Job**: Thực thể đại diện cho một tác vụ, khách hàng hoặc gói tin luân chuyển trong mạng hàng đợi.

  – *Thuộc tính:*

  + `job_id: int` - Mã định danh duy nhất của yêu cầu trong toàn bộ kịch bản.
  + `orig_arrival_time: float` - Mốc thời gian ảo yêu cầu gia nhập mạng lần đầu tiên (phục vụ tính trễ toàn trình).
  + `arrival_time: float` - Mốc thời gian ảo yêu cầu xuất hiện tại trạm logic hiện tại.
  + `start_service_time: Optional[float]` - Thời điểm yêu cầu bắt đầu được máy chủ xử lý.
  + `departure_time: Optional[float]` - Thời điểm yêu cầu hoàn tất và rời khỏi trạm.

Phân hệ Quản lý Hàng đợi Ưu tiên

- **BaseEventQueue**: Là giao diện trừu tượng (*Interface/Abstract Base Class*) định nghĩa hợp đồng chung cho các cấu trúc hàng đợi sự kiện.

  – *Phương thức:*

  + `push(event: Event): void` - Thêm một sự kiện mới vào cấu trúc hàng đợi.
  + `pop(): Event` - Trích xuất và loại bỏ sự kiện có mốc thời gian ảo nhỏ nhất.
  + `peek(): Optional[Event]` - Xem sự kiện sớm nhất ở đầu hàng mà không loại bỏ.
  + `is_empty(): bool` - Kiểm tra xem hàng đợi có rỗng hay không.
  + `__len__(): int` - Trả về kích thước hiện tại của hàng đợi.

- **MinHeapPriorityQueue**: Hiện thực cây nhị phân Min-Heap chuẩn dựa trên module `heapq` với chi phí thời gian $\mathcal{O}(\log N)$, phù hợp cho các kịch bản thực nghiệm quy mô vừa và nhỏ.

  – *Thuộc tính:*

  + `_heap: List[Event]` - Mảng lưu trữ các nút cây nhị phân heap theo mốc thời gian.

  – *Phương thức:*

  + `push(event: Event): void` - Đẩy sự kiện vào heap qua thao tác `heappush`.
  + `pop(): Event` - Lấy phần tử gốc có timestamp nhỏ nhất qua `heappop`.
  + `peek(): Optional[Event]` - Trả về phần tử tại vị trí gốc `_heap[0]`.
  + `is_empty(): bool` - Kiểm tra danh sách `_heap` có rỗng hay không.

- **LadderQueue**: Hiện thực thuật toán hàng đợi phân tầng hai cấp (`_bottom` và `_top`) theo Hình 3 đề bài, tối ưu hóa chi phí trích xuất trung bình đạt $\mathcal{O}(1)$ cho các kịch bản tải bùng nổ lên đến hàng triệu sự kiện.

  – *Thuộc tính:*

  + `bucket_width: float` (alias `rung_width`) - Bề rộng nấc thang thời gian phục vụ việc hạ sự kiện.
  + `_bottom: List[Event]` - Hàng đợi Min-Heap nhỏ chứa các sự kiện cận kề ($t \leq$ _boundary).
  + `_top: List[Event]` - Hàng đợi Min-Heap chứa các sự kiện xa trong tương lai ($t >$ _boundary).
  + `_boundary: float` - Mốc thời gian ảo phân tầng giữa tầng đáy và tầng đỉnh.
  + `_count: int` - Tổng số lượng sự kiện đang lưu giữ trong cấu trúc.

  – *Phương thức:*

  + `push(event: Event): void` - Đặt sự kiện vào `_bottom` nếu $t \leq$ _boundary, ngược lại vào `_top`.
  + `pop(): Event` - Rút sự kiện từ `_bottom`; nếu rỗng thì kích hoạt `_refill_bottom()`.
  + `peek(): Optional[Event]` - Xem sự kiện sớm nhất ở đầu `_bottom` hoặc đỉnh `_top`.
  + `_refill_bottom(): void` - Hạ một nấc thang sự kiện từ `_top` xuống `_bottom` và cập nhật `_boundary`.

Phân hệ Tiến trình Logic PDES

- **BkSysLogicalProcessState**: Lớp bao đóng trạng thái khả biến của tiến trình logic, hỗ trợ chụp ảnh nhanh và lưu vết.

  – *Thuộc tính:*

  + `_data: Dict[str, Any]` - Từ điển lưu trữ các biến trạng thái nội tại.
  + `history: List[Dict[str, Any]]` - Lịch sử các bước chuyển dịch trạng thái theo thời gian ảo.

  – *Phương thức:*

  + `get(key, default): Any` / `set(key, value): void` - Đọc và ghi biến trạng thái.
  + `checkpoint(): Dict[str, Any]` - Sao chép sâu (*Deep-copy*) toàn bộ trạng thái tại mốc kiểm tra.
  + `restore(checkpoint): void` - Khôi phục trạng thái từ bản chụp nhanh.
  + `record_step(timestamp, description, data): void` - Ghi nhận bước chuyển dịch trạng thái phục vụ debug.

- **BkSysLogicalProcess (alias LogicalProcess)**: Lớp cơ sở trừu tượng cho mọi thực thể mô phỏng PDES.

  - *Thuộc tính:*
    + `lp_id: int | str` - Mã định danh duy nhất của tiến trình logic.
    + `local_virt_time: float` (alias `local_virtual_time`) - Đồng hồ ảo cục bộ (LVT) của trạm.
    + `state: BkSysLogicalProcessState` - Thực thể trạng thái nội tại của trạm.
    + `_output_events: List[Event]` - Danh sách vùng đệm chứa các sự kiện chuẩn bị xuất xưởng.
    + `_dispatcher: Optional[Any]` - Tham chiếu ngược đến bộ điều phối sự kiện.
    + `_handlers: Dict[str, Callable]` - Bảng tra cứu hàm xử lý nghiệp vụ theo loại sự kiện.
  - *Phương thức:*
    + `initialize(): void` / `finalize(): void` - Hook khởi tạo và chốt sổ vòng đời trạm logic.
    + `get_local_virt_time(): float` - Lấy mốc thời gian ảo cục bộ hiện tại.
    + `set_local_virt_time(t: float): void` - Cập nhật LVT, kiểm soát nghiêm ngặt ràng buộc nhân quả LCC ($t \geq LVT$).
    + `elapse_virt_time(delta: float): float` - Tịnh tiến LVT thêm một khoảng trễ $\Delta t \geq 0$.
    + `register_handler(event_type, handler): void` - Đăng ký hàm callback xử lý theo loại sự kiện.
    + `send_event(event: Event): void` - Gửi bản tin sự kiện tới trạm khác hoặc đẩy vào bộ điều phối.
    + `get_output_events(): List[Event]` - Trích xuất và dọn sạch danh sách sự kiện xuất xưởng.
    + `handle_event(event: Event): void` - Tiếp nhận và kích hoạt xử lý biến cố.
    + `reset(): void` - Tái lập đồng hồ LVT và hàng đợi xuất xưởng về trạng thái khởi đầu.

- **SourceLP**: Tiến trình nguồn chịu trách nhiệm tự phát sinh dòng yêu cầu Poisson ngoại vi vào hệ thống.
  - *Thuộc tính:*
    + `dest_lp_id: int | str` - Định danh trạm tiếp nhận kế tiếp (mặc định: `"Server"`).
    + `arrival_rate: float` - Tốc độ phát sinh yêu cầu trung bình λ (yêu cầu/giây).
    + `burst_prob: float` / `burst_size: int` - Xác suất và kích thước chùm sự kiện tải bùng nổ.
    + `_job_counter: int` - Bộ đếm tổng số yêu cầu đã sinh ra từ đầu kịch bản.
    + `_started: bool` - Cờ chống phát sinh trùng lặp sự kiện mồi đầu tiên.
  - *Phương thức:*

+ `initialize(): void` / `start(): void` - Kích hoạt tự lên lịch sự kiện đến đầu tiên $\Delta t \sim \text{Exp}(\lambda)$.
+ `handle_event(event: Event): void` - Đóng gói thực thể `Job`, chuyển tiếp sang trạm sau và tự lên lịch khách kế tiếp.

- **QueueServerLP**: Trạm phục vụ tích hợp hàng đợi và cụm máy chủ song song theo chuẩn Kendall $A/S/c/K$.
  - *Thuộc tính:*
    + `dest_lp_id: int | str` - Định danh trạm tiếp nhận kế tiếp (mặc định: `"Sink"`).
    + `service_rate: float` - Năng lực phục vụ của mỗi máy chủ μ (yêu cầu/giây).
    + `num_servers: float` - Số lượng máy chủ song song c ($1 \leq c \leq \infty$).
    + `capacity: Optional[int]` - Sức chứa tối đa của hệ thống K ($K =$ None nếu đệm vô hạn).
    + `metrics: MetricsCollector` - Bộ tích lũy mẫu đo hiệu năng tích hợp sẵn tại trạm.
    + `waiting_queue: Deque[Job]` - Hàng ghế chờ FIFO lưu giữ các yêu cầu đang đợi máy chủ rảnh.
    + `busy_servers: int` - Số lượng máy chủ đang bận xử lý tại mốc giờ ảo hiện tại.
    + `total_in_system: int` (property) - Tổng số khách đang có mặt trong trạm (busy + |queue|).
  - *Phương thức:*
    + `handle_event(event: Event): void` - Phân luồng sự kiện `ARRIVAL` hoặc `DEPARTURE`.
    + `_handle_arrival(event: Event): void` - Kiểm tra tràn đệm K, loại bỏ gói tin nếu đầy, phân bổ server rảnh hoặc xếp hàng.
    + `_start_service(job: Job, current_time: float): void` - Đặt mốc bắt đầu phục vụ, bốc thời gian phục vụ $S \sim \text{Exp}(\mu)$ và lên lịch hoàn tất.
    + `_handle_departure(event: Event): void` - Ghi nhận số đo hoàn tất, chuyển Job sang trạm sau và đón khách kế tiếp từ hàng đợi.
    + `_notify_state(current_time: float): void` - Cập nhật trạng thái tích phân diện tích theo thời gian.
    + `finalize(): void` / `get_metrics(): Dict[str, Any]` - Chốt sổ và trích xuất vector chỉ số hiệu năng.

- **BranchingServerLP**: Kế thừa từ `QueueServerLP`, chuyên biệt hóa cơ chế rẽ nhánh ngẫu nhiên Bernoulli cho mạng hàng đợi Jackson.
  - *Thuộc tính:*
    + `printer_lp_id: int | str` / `disk_lp_id: int | str` - Định danh trạm Máy in và Ổ đĩa.
    + `p_printer: float` - Xác suất rẽ nhánh tới Máy in ($p = 0.3$, còn lại $1 - p = 0.7$ tới Ổ đĩa).

  - *Phương thức:*
    + `_handle_departure(event: Event): void` - Ghi đè xử lý hoàn tất để thực hiện phép thử Bernoulli định tuyến ngẫu nhiên.

- **SinkLP**: Điểm thu gom cuối cùng của dòng xử lý, tính toán độ trễ toàn trình và giải phóng các lời hứa bất đồng bộ.
  - *Thuộc tính:*
    + `future_manager: Optional[Any]` - Cầu nối giải phóng Future cho phía ứng dụng coroutine.
    + `completed_jobs: List[Job]` - Danh sách toàn bộ các thực thể `Job` đã hoàn tất.
    + `total_response_times: List[float]` - Danh sách mảng thời gian lưu trú toàn trình của từng yêu cầu.
    + `total_completed: int` (property) - Đếm tổng số yêu cầu đã hoàn tất.
  - *Phương thức:*
    + `handle_event(event: Event): void` - Ghi nhận thời gian lưu trú $W = t_{\text{dep}} - t_{\text{orig_arr}}$ và kích hoạt `set_result()` cho Future tương ứng.
    + `get_metrics(): Dict[str, Any]` - Tính toán thời gian phản hồi trung bình và tổng số yêu cầu hoàn thành.

- **NetworkSinkLP**: Kế thừa từ `SinkLP`, chuyên biệt hóa tính toán độ trễ toàn trình cho mạng đa chặng Jackson.
  - *Thuộc tính:*
    + `network_latencies: List[float]` - Mảng lưu trữ thời gian trễ toàn trình mạng từ trạm nguồn đến trạm đích.
  - *Phương thức:*
    + `handle_event(event: Event): void` - Ghi nhận thời gian trễ $T = t_{\text{Sink}} -$ job.orig_arrival_time.
    + `get_metrics(): Dict[str, Any]` - Xuất chỉ số độ trễ mạng trung bình `mean_network_latency`.

- **QueuingNetwork**: Lớp hỗ trợ đóng gói cấu hình và kết nối mạng hàng đợi 4 trạm máy tính (Hình 1 đề bài).
  - *Thuộc tính:*
    + `source: SourceLP`, `io: QueueServerLP`, `cpu: BranchingServerLP`.
    + `printer: QueueServerLP`, `disk: QueueServerLP`, `sink: NetworkSinkLP`.
  - *Phương thức:*
    + `register_all(dispatcher: Any): void` - Tự động nạp toàn bộ 6 trạm mạng vào bộ điều phối.

Phân hệ Cầu nối Bất đồng bộ & Thu thập Dữ liệu

- **AsyncFutureManager**: Cầu nối phi nghẽn giữa môi trường mô phỏng thời gian ảo và cơ chế `asyncio.Future` của Python.
  - *Thuộc tính:*
    + `loop: Optional[asyncio.AbstractEventLoop]` - Vòng lặp sự kiện bất đồng bộ thực tế.
    + `_pending_futures: Dict[int, asyncio.Future]` - Bản đồ lưu trữ các Future đang chờ xử lý theo `request_id`.
  - *Phương thức:*
    + `_get_loop(): asyncio.AbstractEventLoop` - Lấy Event Loop đang chạy hoặc tạo mới an toàn trên Python 3.10+.
    + `create_future(request_id: int): asyncio.Future` - Tạo mới một lời hứa Future gắn với `request_id` và trao cho người dùng `await`.
    + `set_result(request_id: int, result: Any): bool` - Gán kết quả hoàn tất cho Future, đánh thức coroutine tương ứng mà không phong tỏa luồng điều phối.
    + `set_exception(request_id: int, exception: Exception): bool` - Báo lỗi vào Future khi gặp sự cố.
    + `cancel_all(): void` - Hủy bỏ toàn bộ Future đang treo khi dừng hệ thống.
    + `has_pending(request_id: int): bool` / `pending_count(): int` - Kiểm tra số lượng yêu cầu đang chờ giải phóng.
- **AsyncTaskCoordinator**: Hiện thực các mẫu phối hợp tác vụ đồng quy chuẩn hóa của Python `asyncio`.
  - *Phương thức tĩnh:*
    + `run_with_task_group(coroutines: Sequence): List[Any]` - Quản lý tác vụ cấu trúc (*Structured Concurrency*) qua `asyncio.TaskGroup`.
    + `run_with_gather(coroutines: Sequence): List[Any]` - Thu gom kết quả song song hàng loạt qua `asyncio.gather`.
    + `process_as_completed(coroutines, callback)`: Trả về kết quả ngay khi từng tác vụ riêng lẻ hoàn tất theo nguyên tắc FCFS qua `asyncio.as_completed`.
    + `run_in_runner(main_coro: Coroutine): Any` - Điểm khởi động chương trình với bộ thực thi `asyncio.Runner` cô lập.
- **BkSysCoLogicalProcess & BkSysFuture**: Động cơ PDES hướng Coroutine hỗ trợ cú pháp `async/await` bản địa.
  - *Thuộc tính & Phương thức:*

  + `await self.wait_for(event_type: str): Event` - Tạm dừng coroutine cho đến khi bản tin sự kiện phù hợp xuất hiện.
  + `await self.advance_time(delta: float): float` - Tịnh tiến thời gian ảo cục bộ phi nghẽn luồng.
  + `BkSysFuture`: Kế thừa từ `asyncio.Future`, cung cấp các kiểm tra trạng thái (`is_pending`, `is_resolved`, `is_rejected`) và hook callback xử lý lỗi.

- **MetricsCollector**: Bộ tích lũy mẫu đo hiệu năng và tính tích phân thời gian thực.
  - *Thuộc tính:*
    + `arrived_count: int` / `completed_count: int` / `dropped_count: int` - Bộ đếm số khách đến, hoàn tất và bị loại bỏ do tràn đệm.
    + `waiting_times: List[float]` / `service_times: List[float]` / `response_times: List[float]` - Mảng lưu trữ thời gian chờ, phục vụ và lưu trú.
    + `_area_q_len: float` / `_area_in_sys: float` / `_area_busy_servers: float` - Tích lũy diện tích hình chữ nhật tích phân thời gian $\sum \text{State} \cdot \Delta t$.
  - *Phương thức:*
    + `update_state(current_time, q_len, in_sys, busy_servers): void` - Cập nhật diện tích tích phân theo bước nhảy thời gian thực Δt.
    + `record_arrival(): void` / `record_drop(): void` - Ghi nhận lượt đến và rớt gói do quá tải K.
    + `record_completion(wait_time, serv_time): void` - Ghi nhận mẫu thời gian khi hoàn tất phục vụ.
    + `finalize(total_sim_time, num_servers): Dict[str, float]` - Tính toán toàn bộ chỉ số hiệu năng chuẩn: $W, W_q, S, L, L_q, \rho, P_b, \lambda_{\text{eff}}$.

3.1.2 Mối quan hệ giữa các lớp trong hệ thống

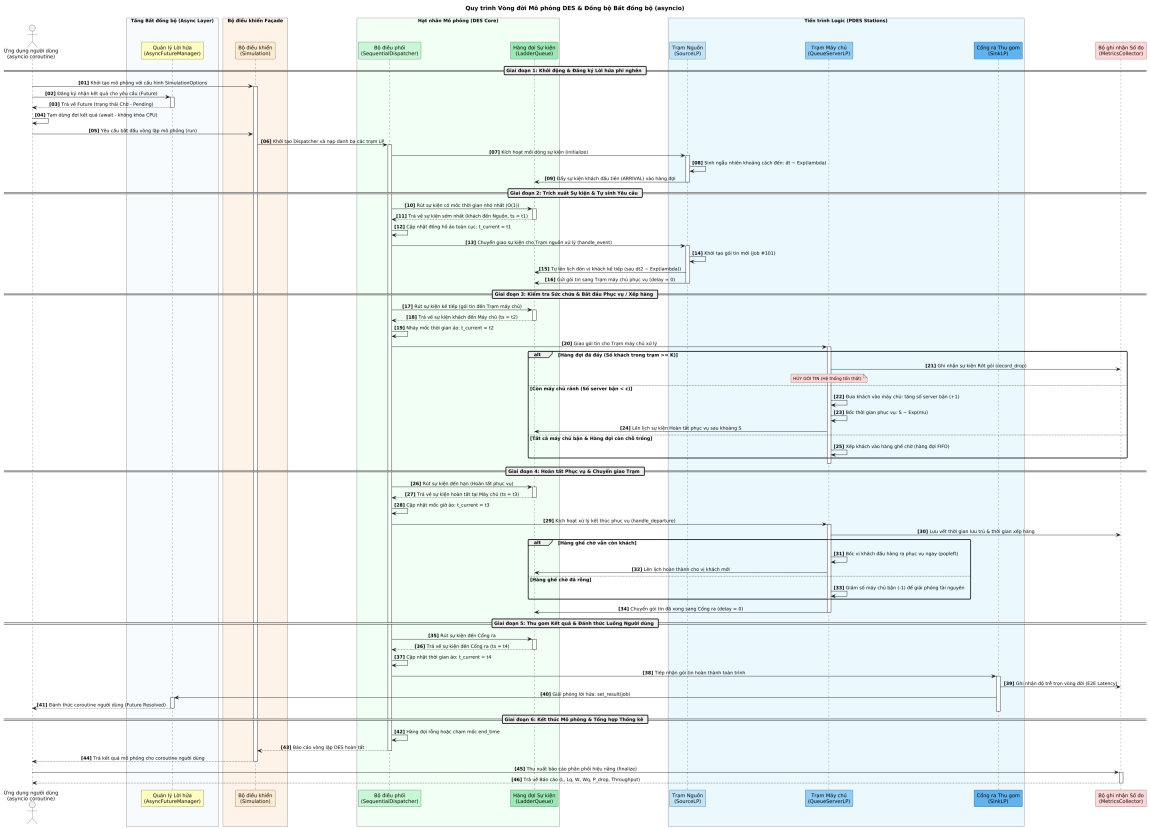
Các thành phần đối tượng phối hợp chặt chẽ với nhau thông qua các cơ chế điều phối thời gian ảo, luân chuyển thông điệp sự kiện và giải phóng lời hứa bất đồng bộ:

- **Quan hệ Điều khiển Facade cấp cao (*Facade Orchestration*):** Lớp mặt tiền `Simulation` đóng vai trò là nhạc trưởng cấp cao nhất, kết tập (*Aggregation*) đối tượng cấu hình `SimulationOptions`, quản lý khởi tạo hạt nhân điều phối `SequentialEventDispatcher` (hoặc mô hình phân vùng `DistributedEventDispatcher`) và sở hữu danh bạ tập hợp các thực thể `BkSysLogicalProcess`. Nhờ đó, ứng dụng người dùng chỉ cần giao tiếp qua giao diện duy nhất của Facade mà không phải can thiệp thủ công vào các tầng bên dưới.
- **Quan hệ Quản lý & Điều phối (Management):** `SequentialEventDispatcher` nắm giữ một tham chiếu duy nhất tới giao diện `BaseEventQueue` theo mẫu thiết kế *Strategy Pattern*, cho phép hoán đổi cấu trúc hàng đợi linh hoạt tại thời điểm chạy. Đồng thời,

`SequentialEventDispatcher` quản lý danh bạ kết tập gồm nhiều `BkSysLogicalProcess` để chuyển giao sự kiện qua phương thức `handle_event()`.

- **Quan hệ Kế thừa (Inheritance):**
  - `MinHeapPriorityQueue` và `LadderQueue` kế thừa và hiện thực các phương thức trừu tượng của `BaseEventQueue`.
  - `SourceLP`, `QueueServerLP` và `SinkLP` cùng kế thừa từ lớp cơ sở `LogicalProcess`, thừa hưởng đồng hồ ảo LVT và năng lực `send_event()`.
  - `BranchingServerLP` chuyên biệt hóa từ `QueueServerLP`, tái sử dụng toàn bộ hàng đợi c máy chủ và chỉ ghi đè logic rẽ nhánh ngẫu nhiên.
- **Quan hệ Sở hữu & Đóng gói (Composition / Aggregation):** `Event` đóng gói thực thể `Job` trong trường `payload`. `QueueServerLP` sở hữu hàng ghế chờ FIFO `waiting_queue` kiểu `deque[Job]`. `SinkLP` kết tập đối tượng `MetricsCollector` để tích lũy dữ liệu.
- **Quan hệ Cầu nối Bất đồng bộ (Async Bridging):** Khi một yêu cầu hoàn tất tại trạm cuối, `SinkLP` kích hoạt phương thức `set_result()` trên `AsyncFutureManager`, đánh thức coroutine phía người dùng mà không làm nghẽn tiến trình mô phỏng rời rạc.
- **Cơ chế Tuân thủ 5 Nguyên tắc SOLID:** Đảm bảo tính mở rộng cao: khi cần bổ sung một kỷ luật phục vụ mới hay một thuật toán hàng đợi mới, chỉ cần mở rộng lớp tương ứng mà không phải can thiệp vào hạt nhân điều phối `Dispatcher`.

3.1.3 Sơ đồ tuần tự tương tác sự kiện (UML Sequence Diagram)



Hình 4: *Sơ đồ tuần tự luân chuyển sự kiện (UML Sequence Diagram) trong vòng lặp DES*

Tiến trình điều phối và xử lý sự kiện trong sơ đồ tuần tự (Hình 4) thiết lập cầu nối đồng bộ chặt chẽ giữa tầng ứng dụng bất đồng bộ và hạt nhân mô phỏng rời rạc. Nhằm làm rõ trách nhiệm kiến trúc và giải tỏa sự nhập nhằng giữa các không gian lưu trữ, vai trò cùng cấu trúc dữ liệu nội bộ của từng thực thể tham gia được chú thích chi tiết tại Bảng 5.

Thành phần	Nơi lưu trữ nội bộ	Vai trò & Hành vi nghiệp vụ tương tác
User (Script)	`options: SimulationOptions`	Điểm khởi tạo cấu hình, nạp mô hình vào Facade và kích hoạt điều phối.
App (Coroutine)	-	Tác vụ coroutine phía người dùng; phát yêu cầu và tạm dừng bằng `await future` phi phong tỏa luồng.
Simulation (Sim)	`options, models, lps, dispatcher`	Lớp mặt tiền (Facade) quản lý chu trình sống, nạp tự động các LP và kết nối Dispatcher.
AsyncFutureManager	`_pending_futures: Dict[int, Future]`	Cầu nối bất đồng bộ; lưu trữ danh bạ lời hứa (*Future*) để đánh thức coroutine khi có kết quả.
SequentialEvent-Dispatcher	`current_time, lps, event_queue, trace_log`	Bộ máy điều phối trung tâm; duy trì mốc LVT toàn cục t, trích xuất sự kiện sớm nhất thúc đẩy bước nhảy thời gian.
LadderQueue	`_bottom, _top, _boundary, bucket_width`	**Sổ lịch hẹn sự kiện**; hàng đợi ưu tiên phân tầng đạt chi phí $\mathcal{O}(1)$ lưu trữ các bản tin `Event`.
SourceLP	`local_virt_time, _started, _job_counter`	Cổng phát sinh tải Poisson; khi mỗi sự kiện sẽ tự động lên lịch đón vị khách kế tiếp sau $\Delta t \sim \text{Exp}(\lambda)$.
QueueServerLP	`waiting_queue (FIFO), busy_servers, metrics`	Trạm máy chủ kèm bộ đệm; quản lý **hàng ghế chờ FIFO** lưu `Job` khi cả c máy chủ đều bận.
SinkLP	`completed_jobs, total_response_times`	Cổng ra thu gom; điểm kết thúc hành trình của `Job`, tính trễ toàn trình W và kích hoạt giải phóng `Future`.
MetricsCollector	`waiting_times, _area_q_len, _area_in_sys`	Bộ tích lũy số đo; tính tích phân thời gian $\sum \text{State} \cdot \Delta t$ để xác định $W, W_q, L, L_q, \rho, P_b$.

Bảng 5: *Bảng chú thích vai trò và cấu trúc lưu trữ của các thực thể tham gia sơ đồ tuần tự*

Dựa trên sự phân công trách nhiệm rành mạch trên, toàn bộ tiến trình điều phối và luân chuyển sự kiện được phân tầng thành 6 giai đoạn kỹ thuật rõ rệt, gắn kết trực tiếp với các bước tương tác từ **[01]** đến **[46]**:

1. **Giai đoạn 1: Khởi động & Đăng ký Lời hứa phi nghẽn (Bước [01] – [09]):**
   - *Đăng ký Promise:* Coroutine phía người dùng gọi phương thức `register_promise()` tới `AsyncFutureManager`. Trình quản lý khởi tạo đối tượng `asyncio.Future`, lưu trữ vào bảng tra cứu `_pending_futures` và trao lại đối tượng này cho người dùng. Ứng dụng thực hiện `await future` để tạm dừng luồng gọi mà hoàn toàn không phong tỏa luồng điều phối chính.

- *Mồi luồng sự kiện:* Hệ thống kích hoạt phương thức `start()` tại trạm `SourceLP`. Trạm nguồn bốc mẫu ngẫu nhiên khoảng thời gian đến đầu tiên $\Delta t_1 \sim \text{Exp}(\lambda)$, đóng gói thành bản tin `Event(type=ARRIVAL, dest='Source', timestamp=`Δt_1`)` và đẩy trực tiếp vào hàng đợi ưu tiên `EventQueue`.

2. **Giai đoạn 2: Trích xuất Sự kiện & Tự sinh Yêu cầu (Bước [10] – [16]):**

- *Bước nhảy thời gian ảo:* `Dispatcher` thực hiện thao tác `queue.pop()` lấy sự kiện có nhãn thời gian sớm nhất ra khỏi `EventQueue` và cập nhật trục thời gian ảo toàn cục: $t_{\text{current}} = t_1$. Mọi khoảng thời gian trống từ 0.00 đến t_1 được bỏ qua tức thời.
- *Tự sinh dòng Poisson vô hạn:* Sự kiện được chuyển giao cho `SourceLP` để khởi tạo một thực thể `Job` mới (chứa mã định danh `job_id` và mốc giờ đến `arrival_time` $= t_1$). Đồng thời, trạm tự lên lịch cho chính mình sự kiện khách đến kế tiếp sau độ trễ $\Delta t_2 \sim \text{Exp}(\lambda)$ và ném vào `EventQueue` nhằm duy trì dòng khách liên tục. Ngay sau đó, trạm chuyển tiếp gói tin sang `QueueServerLP` với độ trễ chuyển mạch bằng 0 (tức thời tại mốc t_1).

3. **Giai đoạn 3: Phân phối & Kiểm tra Sức chứa tại Máy chủ (Bước [17] – [25]):**

- `Dispatcher` tiếp tục lấy sự kiện đến chuyển cho `QueueServerLP.handle_event(e)`.
- *Cơ chế kiểm soát đệm K:* Máy chủ kiểm tra ngưỡng dung lượng trạm: nếu số khách hiện hữu trong trạm (bao gồm số khách đang được phục vụ cộng với số khách đang ngồi chờ) đã đạt trần K ($N(t) = \text{busy} + \text{len(waiting_queue)} \geq K$), gói tin bị hủy bỏ ngay lập tức (*Packet Dropped*) và cập nhật tổn thất vào `MetricsCollector`.
- *Phân bổ phục vụ hoặc xếp hàng chờ:* Nếu trạm còn chỗ:
  - Khi có ít nhất một máy chủ rảnh rỗi (busy_servers $< c$): Yêu cầu `Job` được đưa thẳng vào phục vụ ngay (thời gian chờ $W_q = 0$). Trạm tăng bộ đếm busy_servers $\leftarrow$ busy_servers $+ 1$, bốc ngẫu nhiên thời gian phục vụ $S \sim \text{Exp}(\mu)$ và tạo sự kiện hoàn tất trong tương lai `Event(type=DEPARTURE, timestamp=`$t + S$`)` đẩy vào `EventQueue`.
  - Khi tất cả c máy chủ đều đang bận (busy_servers $= c$): Yêu cầu `Job` được đưa vào **hàng ghế chờ FIFO** `waiting_queue.append(job)` nằm bên trong trạm để kiên nhẫn đợi đến lượt.

4. **Giai đoạn 4: Hoàn tất Phục vụ & Chuyển giao Trạm (Bước [26] – [34]):**

- Khi trục thời gian ảo tiến tới mốc hoàn thành $t_{\text{current}} = t + S$, `Dispatcher` kích hoạt sự kiện `DEPARTURE` tại `QueueServerLP`.
- Máy chủ ghi nhận thời gian lưu trú và thời gian chờ vào `MetricsCollector`. Sau đó kiểm tra hàng ghế chờ:
  - Nếu `waiting_queue` còn khách: Trạm lập tức bốc vị khách đầu hàng ra xử lý (`waiting_queue.popleft()`), bốc thời gian phục vụ mới S' và lên lịch sự kiện `DEPARTURE` tiếp theo vào `EventQueue`.
  - Nếu `waiting_queue` rỗng: Trạm giảm bộ đếm máy chủ bận (busy_servers $\leftarrow \max(0, \text{busy} - 1)$) để hoàn trả tài nguyên máy chủ về trạng thái rảnh rỗi.

- Gói tin đã xử lý xong được chuyển tiếp sang cổng ra `SinkLP` với độ trễ bằng 0.

5. **Giai đoạn 5: Thu gom Kết quả & Giải phóng Lời hứa (Bước [35] – [41]):**
   - `SinkLP` tiếp nhận gói tin hoàn tất, tính toán độ trễ toàn trình $T = t_{\text{dep}} - t_{\text{arr}}$ và ghi nhận vào `MetricsCollector`.
   - Trạm gọi phương thức `set_result(job_id, job)` trên `AsyncFutureManager`. Trình quản lý tra cứu mã yêu cầu trong bảng `_pending_futures`, gán giá trị kết quả và giải phóng `Future`. Lập tức coroutine phía người dùng đang dừng ở lệnh `await` được đánh thức để nhận kết quả trong thời gian thực mà không làm phong tỏa tiến trình mô phỏng rời rạc.

6. **Giai đoạn 6: Kết thúc Mô phỏng & Tổng hợp Thống kê (Bước [42] – [46]):**
   - Khi chạm mốc thời gian kết thúc T_{end} hoặc `EventQueue` cạn kiệt phần tử, `Dispatcher` dừng vòng lặp điều phối và phát thông báo hoàn tất tới ứng dụng.
   - Ứng dụng người dùng kích hoạt phương thức trích xuất báo cáo từ `MetricsCollector` để thu được toàn bộ vector chỉ số hiệu năng chuẩn xác $(W, W_q, L, L_q, P_b, \lambda_{\text{eff}})$.

Kiểu sự kiện	Thực thể phát	Thực thể nhận	Độ trễ Δt	Hành vi nghiệp vụ kích hoạt
ARRIVAL	`SourceLP`	`SourceLP`	$\text{Exp}(\lambda)$	Tự sinh khách kế tiếp và duy trì dòng Poisson
ARRIVAL	`SourceLP`	`QueueServerLP`	0.0	Chuyển Job mới đến kiểm tra dung lượng bộ đệm K
DEPARTURE	`QueueServerLP`	`QueueServerLP`	$\text{Exp}(\mu)$	Hoàn tất phục vụ, giải phóng server, đón khách chờ
ARRIVAL	`BranchingServerLP`	`Printer / Disk`	0.0	Rẽ nhánh xác suất ($p = 0.3$ Máy in, $p = 0.7$ Ổ đĩa)
ARRIVAL	`Server / Disk`	`SinkLP`	0.0	Ghi nhận trễ toàn trình và kích hoạt `set_result()`
TERMINATE	`Dispatcher`	`Dispatcher`	T_{end}	Dừng vòng lặp điều phối và tổng kết kết quả

Bảng 6: *Ma trận thông điệp sự kiện luân chuyển giữa các thành phần*

3.2 Kiến trúc điều phối cốt lõi của bộ mô phỏng DES (Core DES Dispatching Architecture)

3.2.1 Cấu trúc Lớp Bản ghi Sự kiện (`Event`) và Cơ chế So sánh theo Thời gian ảo (Timestamp Ordering)

Trong mô phỏng DES, toàn bộ động thái của hệ thống được kích hoạt bằng các bản tin sự kiện (`Event`). Để tối ưu hóa hiệu năng trích xuất và bảo đảm tính tiền định (*Determinism*), hệ thống áp dụng cơ chế so sánh tự nhiên dựa trên mốc thời gian ảo `timestamp`:

$$e_1 < e_2 \iff e_1.\text{timestamp} < e_2.\text{timestamp} \tag{73}$$

trong đó:

- e.timestamp: Mốc thời gian ảo mà sự kiện sẽ nổ ra (tiêu chí so sánh tự nhiên duy nhất theo khai báo `@dataclass(order=True)`).
- e.sender_id: Mã định danh tiến trình logic phát sinh sự kiện (hỗ trợ alias property `source_lp`).
- e.receiver_id: Mã định danh tiến trình logic tiếp nhận sự kiện (hỗ trợ alias property `dest_lp`).
- e.event_type: Kiểu sự kiện trong mô hình rời rạc (`ARRIVAL`, `SERVICE_START`, `DEPARTURE`,...).
- e.payload: Tải trọng dữ liệu đi kèm gói tin (chứa đối tượng `Job` hoặc từ điển dữ liệu, hỗ trợ alias property `data`).
- e.event_id: Chuỗi định danh duy nhất của sự kiện dạng UUID4 tự sinh.

Listing 1: Cấu trúc sự kiện Event và cơ chế so sánh theo thời gian ảo (bkitevsim/core/event.py)

```python
from __future__ import annotations
from dataclasses import dataclass, field
from enum import Enum, auto
import uuid
from typing import Any, Optional

class EventType(Enum):
    """Event types supported in discrete-event queuing models."""
    ARRIVAL = auto()         # Customer or packet arrives at a station
    SERVICE_START = auto()   # Server starts servicing a waiting customer
    DEPARTURE = auto()       # Service complete; packet departs station
    IO_COMPLETE = auto()     # I/O peripheral data transfer completed
    MESSAGE = auto()         # Inter-LP asynchronous coordination message
    TERMINATE = auto()       # Triggers simulation engine termination

@dataclass(order=True)
class Event:
    """Represents a discrete event in the simulation (PDES model).
```

```
19
20      Events are ordered by timestamp for priority queue operations.
21      The order=True parameter generates natural comparison based on timestamp.
22      """
23      # --- 1st key: Earliest virtual time (LVT) ---
24      timestamp: float
25      # --- Domain attributes (excluded from heap comparisons via compare=False)
26      sender_id: Any = field(compare=False, default=0)
27      receiver_id: Any = field(compare=False, default=0)
28      event_type: Any = field(compare=False, default="EVENT")
29      payload: Any = field(compare=False, default=None)
30      event_id: Any = field(compare=False, default_factory=lambda: str(uuid.uuid4
            ()))
31
32      def __init__(
33          self,
34          timestamp: float = 0.0,
35          sender_id: Any = None,
36          receiver_id: Any = None,
37          event_type: Any = "EVENT",
38          payload: Any = None,
39          event_id: Any = None,
40          **kwargs: Any
41      ) -> None:
42          self.timestamp = float(timestamp)
43          # Handle source / sender aliases
44          self.sender_id = sender_id if sender_id is not None else kwargs.get("
                source_lp", 0)
45          # Handle destination / receiver aliases
46          self.receiver_id = receiver_id if receiver_id is not None else kwargs.
                get("dest_lp", 0)
47          self.event_type = event_type.name if hasattr(event_type, "name") else
                str(event_type)
48          self.payload = payload if payload is not None else kwargs.get("data",
                None)
49          self.event_id = str(event_id) if event_id is not None else str(uuid.
                uuid4())
50          self._priority = kwargs.get("priority", 0)
51
52          # Validation: Enforce non-negative timestamps
53          if self.timestamp < 0:
54              raise ValueError(f"Event timestamp cannot be negative: {self.
                    timestamp}")
55
56      @property
57      def source_lp(self) -> Any:
58          return self.sender_id
59
60      @property
61      def dest_lp(self) -> Any:
62          return self.receiver_id
```

Phân tích kỹ thuật chuyên sâu về Phòng vệ kiểu dữ liệu và Cơ chế so sánh trong `bkitevsim`:

Để hiểu rõ tại sao kiến trúc lớp `Event` trong gói `bkitevsim` mang tính quyết định đến độ ổn định của toàn bộ hệ thống mô phỏng, cần phân tích nguy cơ kỹ thuật tiềm ẩn trong các hệ thống DES hướng đối tượng:

- **Cơ chế định thời tự nhiên qua `@dataclass(order=True)`:** Tiêu chí cốt lõi của DES là nguyên lý nhân quả: sự kiện có mốc thời gian ảo nhỏ hơn bắt buộc phải được trích xuất và kích hoạt trước. Bằng cách khai báo `timestamp: float` là trường so sánh duy nhất và kích hoạt `order=True`, Python tự động sinh các phương thức so sánh quan hệ (`__lt__`, `__gt__`,...) thuần túy dựa trên biến số thực `timestamp`, giúp các cấu trúc hàng đợi ưu tiên (`MinHeapPriorityQueue` và `LadderQueue`) đạt tốc độ so sánh cực đại $\mathcal{O}(1)$ trên thanh ghi CPU.
- **Phòng vệ ngoại lệ thời gian chạy (*Runtime Exception Prevention*):** Nhờ thiết lập `compare=False`, trong ngôn ngữ Python, nếu hai đối tượng dataclass có trường so sánh đầu tiên bằng nhau (e_1.timestamp == e_2.timestamp), module `heapq` sẽ tự động so sánh trường tiếp theo được định nghĩa trong lớp. Nếu không thiết lập tường minh, bộ so sánh sẽ chạm vào trường `payload` chứa kiểu dữ liệu từ điển (`dict`) hoặc đối tượng nghiệp vụ phức tạp. Do Python không hỗ trợ toán tử so sánh $<$ giữa hai từ điển, chương trình sẽ lập tức ném ngoại lệ:

  ```
  TypeError: '<' not supported between instances of 'dict'
  ```

  làm sập toàn bộ tiến trình mô phỏng ngay tại thời gian chạy. Trong gói `bkitevsim`, việc cấu hình tường minh `compare=False` cho toàn bộ các trường còn lại (`sender_id`, `receiver_id`, `event_type`, `payload`, `event_id`) là giải pháp phòng vệ kiến trúc mang tính sống còn, cô lập hoàn toàn phép so sánh heap chỉ vào biến số thực `timestamp`.
- **Hệ thống thuộc tính tương thích ngược (*Backward Compatibility Properties*):** Nhằm dung hòa giữa chuẩn thiết kế mới của gói `bkitevsim` (sử dụng danh pháp chuẩn PDES `sender_id`, `receiver_id`, `payload`) và các kịch bản mô phỏng kiểm thử cũ, lớp `Event` cung cấp sẵn hệ thống getter property (`source_lp`, `dest_lp`, `data`, `priority`), cho phép mã nguồn ngoài truy cập nhất quán mà không làm thay đổi cấu trúc lưu trữ nội bộ.

3.2.2 Cấu trúc hàng đợi sự kiện phân tầng hai cấp (LadderQueue)

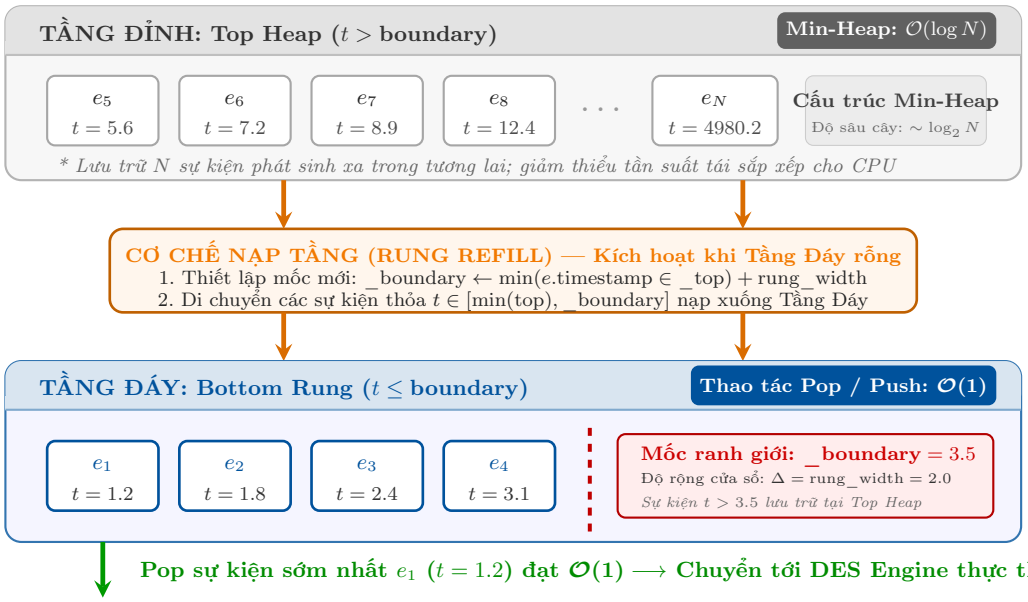
Cấu trúc Min-Heap nhị phân chuẩn sở hữu chi phí chèn và xóa là $O(\log N)$. Khi số lượng sự kiện mô phỏng vượt ngưỡng 10^6, độ sâu của cây Heap làm suy giảm nghiêm trọng băng thông xử lý của CPU.

Để giải quyết vấn đề này, nhóm đã hiện thực thuật toán `LadderQueue` phân tầng hai cấp (Hình 5):

- **Tầng dưới (`_bottom`):** Lưu trữ tập hợp các sự kiện sắp xảy ra trong một cửa sổ thời gian gần $[t_{\text{current}}, _\text{boundary}]$. Do số lượng phần tử nhỏ, thao tác trích xuất đạt hiệu năng tối ưu $O(1)$.
- **Tầng trên (`_top`):** Lưu trữ các sự kiện thuộc tương lai xa ($t > _\text{boundary}$).
- **Bất biến phân tầng (*Tiered Invariant*):** Duy trì đẳng thức:

$$\max_{e \in _\text{bottom}} (e.\text{timestamp}) \leq _\text{boundary} < \min_{e \in _\text{top}} (e.\text{timestamp}) \tag{74}$$

- **Cơ chế nạp tầng (*Rung Refill*):** Khi tầng `_bottom` cạn kiệt, thuật toán xác định ranh giới mới $_\text{boundary} = \min(_\text{top}) + \text{rung_width}$ và trút các sự kiện thỏa điều kiện từ tầng trên xuống tầng dưới.



Hình 5: *Kiến trúc phân tầng hai cấp và cơ chế nạp tầng (Rung Refill) của thuật toán LadderQueue*

Listing 2: Hiện thực cấu trúc phân tầng LadderQueue (bkitevsim/core/priority_queue.py)

```
class LadderQueue(BaseEventQueue):
    def __init__(self, bucket_width: float = 1.0, rung_width: Optional[float] =
        None) -> None:
```

```
        self.bucket_width = rung_width if rung_width is not None else
            bucket_width
        self.rung_width = self.bucket_width
        self._bottom: List[Event] = []
        self._top: List[Event] = []
        self._boundary: float = -1.0
        self._count: int = 0

    def push(self, event: Event) -> None:
        self._count += 1
        if not self._bottom:
            if not self._top:
                self._boundary = event.timestamp + self.bucket_width
                heapq.heappush(self._bottom, event)
                return
            else:
                if event.timestamp <= self._boundary:
                    heapq.heappush(self._bottom, event)
                else:
                    heapq.heappush(self._top, event)
        else:
            if event.timestamp <= self._boundary:
                heapq.heappush(self._bottom, event)
            else:
                heapq.heappush(self._top, event)

    def pop(self) -> Event:
        if self._count == 0:
            raise IndexError("pop from empty LadderQueue")

        if not self._bottom:
            self._refill_bottom()

        event = heapq.heappop(self._bottom)
        self._count -= 1
        return event

    def _refill_bottom(self) -> None:
        if not self._top:
            return

        min_future_time = self._top[0].timestamp
        self._boundary = min_future_time + self.bucket_width

        remaining_top: List[Event] = []
        for ev in self._top:
            if ev.timestamp <= self._boundary:
                self._bottom.append(ev)
            else:
                remaining_top.append(ev)

```

```
53          heapq.heapify(self._bottom)
54          self._top = remaining_top
55          heapq.heapify(self._top)
```

Bảng 7 tổng hợp sự so sánh định lượng về độ phức tạp giữa hai cấu trúc hàng đợi sự kiện.

Cấu trúc hàng đợi	Thao tác Push	Thao tác Pop	Bộ nhớ	Quy mô mô phỏng
`MinHeapPriorityQueue`	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$	Kịch bản nhỏ đến trung bình ($N < 50.000$ sự kiện)
`LadderQueue` (**Hình 5**)	$\boldsymbol{\mathcal{O}(1)}$ khấu hao	$\boldsymbol{\mathcal{O}(1)}$ khấu hao	$\mathcal{O}(N)$	Kịch bản tải cực hạn ($N > 10^6$ sự kiện)

Bảng 7: *So sánh độ phức tạp tính toán giữa Min-Heap và LadderQueue*

Phân tích chi phí khấu hao và Độ nhạy tham số rung_width (Δ):

Hiệu năng vượt trội của thuật toán `LadderQueue` được chứng minh qua hai khía cạnh lý thuyết và thực nghiệm:

- **Chứng minh chi phí khấu hao $\mathcal{O}(1)$ (*Amortized Complexity*):** Xét vòng đời của N sự kiện mô phỏng:
  - Mỗi sự kiện khi sinh ra chỉ được đẩy vào `_top` tối đa 1 lần (chi phí $\mathcal{O}(\log N)$) hoặc đẩy thẳng vào `_bottom` (chi phí $\mathcal{O}(\log k)$ với $k \ll N$).
  - Trong toàn bộ quá trình mô phỏng, mỗi phần tử từ `_top` chỉ được trút xuống `_bottom` đúng 1 lần duy nhất thông qua cơ chế nạp tầng `_refill_bottom`.
  - Tại tầng đáy `_bottom`, số lượng sự kiện luôn bị chặn trên bởi kích thước cửa sổ Δ = rung_width, do đó thao tác trích xuất `heappop` diễn ra trên tập phần tử rất nhỏ, tiệm cận $\mathcal{O}(1)$.
  - Tổng khối lượng tính toán phân bổ cho N phần tử đạt $\mathcal{O}(N)$, dẫn đến chi phí trung bình trên mỗi thao tác là $\mathcal{O}(1)$ khấu hao.
- **Phân tích độ nhạy của tham số độ rộng bước thang Δ (`rung_width`):**
  - Nếu $\Delta \to \infty$: Cửa sổ thời gian quá rộng, toàn bộ sự kiện đều đổ dồn vào tầng đáy, khiến `_bottom` bị quá tải và cấu trúc suy biến ngược trở lại thành một Min-Heap nhị phân thông thường ($\mathcal{O}(\log N)$).
  - Nếu $\Delta \to 0$: Cửa sổ thời gian quá hẹp, tầng đáy liên tục rỗng, hệ thống phải gọi hàm `_refill_bottom` với tần suất dày đặc, làm tiêu tốn tài nguyên CPU vào các chu kỳ dịch chuyển phần tử giữa hai tầng.
  - *Quy tắc cấu hình tối ưu:* Giá trị Δ cần thỏa mãn $\Delta \approx \frac{k}{\lambda}$ (với λ là tốc độ khách đến trung bình và $k \in [2, 5]$). Với tốc độ lưu lượng chuẩn $\lambda = 1.0$, nhóm thiết lập rung_width $= 2.0$, bảo đảm mỗi lần nạp tầng chứa từ 2 đến 4 sự kiện, tối đa hóa thông lượng xử lý của CPU.

3.2.3 Vòng lặp điều phối sự kiện trung tâm (Dispatcher Event Loop)

Lớp `Dispatcher` đóng vai trò là nhạc trưởng trung tâm, duy trì vòng lặp thực thi của hệ thống mô phỏng. Toàn bộ tiến trình điều phối được chuẩn hóa trong Thuật toán 1.

Thuật toán 1: Vòng lặp điều phối sự kiện rời rạc (`SequentialEventDispatcher.run`)

Đầu vào: Hàng đợi sự kiện $\mathcal{Q}$ (`LadderQueue`), bảng ánh xạ tiến trình logic $\mathcal{L}$ (`self.lps`), mốc kết thúc T_{end}.
Đầu ra: Thống kê thời gian mô phỏng ảo (`simulation_time`) và thời gian thực máy tính (`wall_time`).

while $\mathcal{Q} \neq \emptyset$ **do**
  $e \leftarrow \mathcal{Q}$.pop() *// Lấy sự kiện có timestamp sớm nhất từ tầng đáy $\mathcal{O}(1)$*
  if e.timestamp $> T_{end}$ **then**
    $\mathcal{Q}$.push(e); **break** *// Vượt ngưỡng thời gian mô phỏng, bảo toàn biên và dừng*
  end if
  $t_{\text{current}} \leftarrow e$.timestamp ***// Bước nhảy thời gian ảo: Không dùng sleep!***
  Ghi vết sự kiện e vào `trace_log` *// Kiểm chứng vết thực thi đơn điệu*
  $LP_{dest} \leftarrow \mathcal{L}[e.\text{receiver_id}]$ *// Tra cứu trạm đích (hoặc qua thuộc tính e.dest_lp)*
  LP_{dest}.set_local_virt_time(e.timestamp) *// Cập nhật mốc LVT nhân quả cho trạm*
  LP_{dest}.handle_event(e) *// Chuyển giao sự kiện cho trạm đích xử lý*
  for each $e_{out} \in LP_{dest}$.get_output_events() **do** $\mathcal{Q}$.push(e_{out}) *// Thu gom sự kiện xuất xưởng*
  events_processed $\leftarrow$ events_processed $+ 1$
end while
return Báo cáo thống kê hiệu năng mô phỏng `self.stats`.

Listing 3: Vòng lặp điều phối chính trong bkitevsim/dispatchers/sequential_eventdispatcher.py

```
    def run(self, end_time: Optional[float] = None) -> Dict[str, Any]:
        """Run the simulation until end_time or no more events."""
        if end_time is not None:
            self.end_time = float(end_time)
        wall_start = time.perf_counter()
        self.schedule_initial_events()

        while not self.event_queue.is_empty():
            event = self.event_queue.pop()

            # [Phase 1]: Boundary check - preserve boundary event when simulation horizon
```

```
                reached
            if event.timestamp > self.end_time:
                self.event_queue.push(event)
                break

            # [Phase 2]: Instantaneous Virtual Clock Jump and audit trace logging
            self.current_time = event.timestamp

            if len(self.trace_log) < self.max_trace_entries:
                self.trace_log.append({
                    "seq": self.events_processed + 1,
                    "event_id": event.event_id,
                    "timestamp": event.timestamp,
                    "type": str(event.event_type),
                    "src": event.sender_id,
                    "dst": event.receiver_id
                })

            # [Phase 3]: Message Routing via destination LP lookup and LVT advance
            receiver = self.lps.get(event.receiver_id)
            if receiver is not None:
                receiver.set_local_virt_time(event.timestamp)
                receiver.handle_event(event)
                for ev in receiver.get_output_events():
                    self.schedule_event(ev)

            self.events_processed += 1

        for lp in self.lps.values():
            lp.finalize()

        wall_duration = time.perf_counter() - wall_start
        self.stats = {
            "simulation_time": self.current_time,
            "events_processed": self.events_processed,
            "wall_time": wall_duration,
            "end_time": self.end_time,
            "dispatcher": "sequential",
        }
        return self.stats
```

Phân tích 3 pha vận hành cốt lõi của vòng lặp Dispatcher:

Vòng lặp thực thi của lớp `Dispatcher` được cấu trúc hóa thành 3 pha nghiệp vụ khép kín:

1. **Pha 1: Trích xuất sự kiện và Bảo toàn ranh giới biên (*Boundary Invariant Preservation*):**
   - *Trích xuất đỉnh hàng đợi:* Lấy sự kiện sớm nhất (`self.event_queue.pop()`) từ tầng đáy của `LadderQueue` với độ phức tạp khấu hao $\mathcal{O}(1)$.
   - *Kiểm tra ngưỡng dừng:* So sánh mốc thời gian sự kiện với cận biên kết thúc (e.timestamp > self.end_time).
   - *Bảo toàn trạng thái biên:* Đẩy ngược sự kiện lại hàng đợi qua `event_queue.push(event)` trước khi ngắt vòng lặp (`break`), cho phép chạy nối tiếp kịch bản (*Simulation Resumption*) mà không mất mát dữ liệu biên.

2. **Pha 2: Nhảy thời gian ảo và Ghi nhận nhật ký vết (*Audit Logging*):**
   - *Bước nhảy thời gian ảo:* Cập nhật đồng hồ toàn cục `self.current_time` = e.timestamp tức thời trong $\mathcal{O}(1)$, hoàn toàn không tiêu tốn chu kỳ chờ CPU thực tế.
   - *Ghi nhận nhật ký vết:* Lưu bản ghi cấu trúc `{"seq", "event_id", "timestamp", "type", "src", "dst"}` vào `trace_log` (với chặn trên `max_trace_entries = 1000` bảo đảm an toàn bộ nhớ).
   - *Kiểm chứng nhân quả:* Cung cấp bằng chứng số học phục vụ xác minh tính đơn điệu ngặt của trục thời gian ảo trong các bài kiểm thử tự động.
3. **Pha 3: Điều phối thông điệp tức thời $\mathcal{O}(1)$ (*Message Routing*):**
   - *Tra cứu trạm đích:* Truy xuất thực thể trạm nhận qua bảng băm `self.lps` với khóa `event.receiver_id` (hoặc alias `dest_lp`) trong $\mathcal{O}(1)$.
   - *Cập nhật LVT trạm nhận:* Gọi `receiver.set_local_virt_time(event.timestamp)` để đồng bộ mốc thời gian ảo cục bộ và phòng vệ luật nhân quả trước khi xử lý.
   - *Chuyển giao quyền xử lý:* Kích hoạt phương thức xử lý `receiver.handle_event(event)` theo chuẩn PDES.
   - *Thu gom sự kiện xuất xưởng:* Thu thập sự kiện mới phát sinh qua `receiver.get_output_events()` và nạp lại vào hàng đợi qua `self.schedule_event(ev)`.
   - *Cập nhật chỉ số:* Tăng bộ đếm sự kiện (`self.events_processed += 1`) phục vụ đo đạc thông lượng và tốc độ xử lý.

3.2.4 Bộ điều khiển mô phỏng Facade (Simulation & SimulationOptions)

Nhằm tuân thủ trọn vẹn mô hình tương tác hệ thống được đặc tả trong Hình 4 và Hình 5 của đề bài *The Queuing Systems-based Performance Evaluation Project (hk261_BKsyspeEvsim.pdf)*, hệ thống không yêu cầu người dùng phải tự tay kết nối và khởi tạo chi tiết từng thành phần rời rạc. Thay vào đó, gói thư viện `bkitevsim` cung cấp một lớp mặt tiền điều phối cấp cao nhất mang tên `Simulation` kết hợp với đối tượng cấu hình `SimulationOptions` (Listing 4):

Listing 4: Hiện thực lớp điều khiển Facade Simulation và cấu hình SimulationOptions (bkitevsim/core/simulation.py)

```
class SimulationOptions:
    # Encapsulates simulation configuration options for bkitevsim
    def __init__(
        self,
        end_time: float = 100.0,
        dispatcher: str = "sequential",
        bucket_width: float = 1.0,
        log_level: str = "INFO",
        log_file: Optional[str] = None,
        stats_file: Optional[str] = None,
        verbose: bool = False,
        model_params: Optional[Dict[str, Any]] = None
    ) -> None:
        self.end_time: float = end_time
```

```
        self.dispatcher: str = dispatcher
        self.bucket_width: float = bucket_width
        self.log_level: str = log_level
        self.log_file: Optional[str] = log_file
        self.stats_file: Optional[str] = stats_file
        self.verbose: bool = verbose
        self.model_params: Dict[str, Any] = model_params or {}

    def to_dict(self) -> Dict[str, Any]:
        return {
            "end_time": self.end_time, "dispatcher": self.dispatcher,
            "bucket_width": self.bucket_width, "log_level": self.log_level,
            "log_file": self.log_file, "stats_file": self.stats_file,
            "verbose": self.verbose, "model_params": self.model_params,
        }

    @classmethod
    def from_dict(cls, data: Dict[str, Any]) -> SimulationOptions:
        return cls(**data)


class Simulation:
    # High-level simulation controller (Facade Pattern according to Figure 4)
    def __init__(
        self,
        options: Optional[SimulationOptions] = None,
        models: Optional[Dict[str, Type[BkSysLogicalProcess]]] = None,
        factories: Optional[Dict[str, Callable[..., BkSysLogicalProcess]]] = None
    ) -> None:
        self.options: SimulationOptions = options if options is not None else
            SimulationOptions()
        self.models: Dict[str, Type[BkSysLogicalProcess]] = dict(models) if models else
            {}
        self.factories: Dict[str, Callable[..., BkSysLogicalProcess]] = dict(factories)
            if factories else {}
        self.lps: Dict[Any, BkSysLogicalProcess] = {}
        self.dispatcher: Optional[SequentialEventDispatcher] = None

    def register_model(self, name: str, model_cls: Type[BkSysLogicalProcess]) -> None:
        self.models[name] = model_cls

    def create_lp(self, lp_id: int | str, model_name: str, **params: Any) ->
        BkSysLogicalProcess:
        if model_name in self.factories:
            lp = self.factories[model_name](lp_id, **params)
        elif model_name in self.models:
            lp = self.models[model_name](lp_id, **params)
        else:
            raise ValueError(f"Model '{model_name}' is not registered.")

        self.lps[lp_id] = lp
        if self.dispatcher is not None:
            self.dispatcher.register_lp(lp)
        return lp

    def _create_dispatcher(self) -> SequentialEventDispatcher:
        disp_type = self.options.dispatcher.lower()
        if disp_type in ("distributed", "distributed_sequential"):
            return DistributedEventDispatcher(
```

```
                end_time=self.options.end_time, bucket_width=self.options.bucket_width
            )
        return SequentialEventDispatcher(
            end_time=self.options.end_time, bucket_width=self.options.bucket_width
        )

    def run(self) -> Dict[str, Any]:
        if self.dispatcher is None:
            self.dispatcher = self._create_dispatcher()
            for lp in self.lps.values():
                if lp.lp_id not in self.dispatcher.lps:
                    self.dispatcher.register_lp(lp)
        return self.dispatcher.run()

    def get_statistics(self) -> Dict[str, Any]:
        stats: Dict[str, Any] = dict(self.dispatcher.stats if self.dispatcher else {})
        per_lp: Dict[str, Any] = {}
        for lp_id, lp in self.lps.items():
            if hasattr(lp, "get_metrics") and callable(getattr(lp, "get_metrics")):
                per_lp[str(lp_id)] = lp.get_metrics()
            elif hasattr(lp, "metrics"):
                metrics_obj = getattr(lp, "metrics")
                if hasattr(metrics_obj, "finalize") and callable(getattr(metrics_obj, "
                    finalize")):
                    per_lp[str(lp_id)] = metrics_obj.finalize(self.options.end_time)
        stats["per_lp_stats"] = per_lp
        return stats
```

Phân tích kiến trúc và 4 ưu thế cốt lõi của Bộ điều khiển Facade `Simulation`:

1. **Đóng gói tham số vận hành chuẩn hóa (`SimulationOptions`):** Mọi tham số thực nghiệm (thời gian mô phỏng `end_time`, kiểu điều phối `dispatcher`, cấu hình bậc thang LadderQueue `bucket_width`, mức nhật ký `log_level`) được tập trung hóa vào một dataclass có kiểu dữ liệu tường minh. Hai phương thức `to_dict()` và `from_dict()` cho phép hệ thống nạp kịch bản thực nghiệm trực tiếp từ tệp JSON hoặc dòng lệnh CLI một cách tự động, bảo đảm tính tái lập (*Reproducibility*) tuyệt đối.

2. **Cơ chế đăng ký mô hình khai báo (*Declarative Model Registration*):** Thông qua `register_model()` và `register_factory()`, người dùng có thể định nghĩa các lớp trạm logic tùy biến hoặc hàm sinh thực thể (Factory). Phương thức `create_lp(lp_id, model_name, **params)` áp dụng mẫu thiết kế *Factory Pattern*, cho phép khởi tạo hàng loạt trạm logic với các tham số khác nhau (như tốc độ đến λ, tốc độ phục vụ μ, số server c, sức chứa K) mà không cần quản lý thủ công các con trỏ bộ nhớ.

3. **Tự động hóa gắn kết bộ điều phối (`_create_dispatcher`):** Dựa trên giá trị `options.dispatcher`, lớp `Simulation` tự động lựa chọn và khởi tạo đúng loại Dispatcher:

   - Nếu `options.dispatcher == "sequential"`: Cấp phát `SequentialEventDispatcher` với một hàng đợi `LadderQueue` tập trung toàn cục.
   - Nếu `options.dispatcher == "distributed_sequential"`:
     Cấp phát `DistributedEventDispatcher` với cấu trúc phân vùng hàng đợi cục bộ cho từng LP (`_lp_queues[lp_id] = LadderQueue()`).

Toàn bộ danh bạ các trạm logic trong `self.lps` được tự động đăng ký vào Dispatcher qua phương thức `dispatcher.register_lp(lp)`.

4. **Tích hợp hợp nhất số liệu hiệu năng (`get_statistics`):** Sau khi vòng lặp DES kết thúc, phương thức `get_statistics()` thực hiện duyệt qua tất cả các tiến trình logic đã đăng ký, tự động gọi `get_metrics()` hoặc `metrics.finalize(sim_time)` để kết xuất bảng số đo hiệu năng toàn diện ($W, W_q, L, L_q, \rho, P_b$) dưới dạng từ điển hợp nhất `per_lp_stats`, phục vụ trực tiếp cho việc đối chiếu giải tích và xuất biểu đồ khoa học.

3.3 Hiện thực mô hình Tiến trình logic và Các trạm hàng đợi (PDES)

3.3.1 Lớp cơ sở Tiến trình logic và Cơ chế giao tiếp hướng thông điệp

Theo mô hình mô phỏng phân tán PDES của Fujimoto, hệ thống không duy trì một trạng thái khổng lồ tập trung mà phân rã thành các **Tiến trình logic (*Logical Processes* – *LPs*)** độc lập thông qua lớp cơ sở `BkSysLogicalProcess` (alias `LogicalProcess`). Mỗi LP đóng vai trò là một thực thể tự trị có các thuộc tính kỹ thuật sau:

- **Khép kín trạng thái (`BkSysLogicalProcessState`):** Đóng gói toàn bộ dữ liệu trạng thái nội bộ, hỗ trợ chụp ảnh nhanh (`checkpoint()`), khôi phục trạng thái (`restore()`) và lưu vết từng bước chuyển dịch (`record_step()`).
- **Đồng hồ ảo nội bộ (`LVT`):** Mỗi trạm tự quản lý mốc thời gian ảo cục bộ `local_virt_time` (truy xuất qua property `local_virtual_time`). Phương thức `set_local_virt_time(t)` bắt buộc kiểm tra điều kiện $t \geq$ local_virt_time, bảo đảm không có hiện tượng thời gian thụt lùi ngược về quá khứ (*Local Causality Constraint - LCC*).
- **Cơ chế giao tiếp không phong tỏa (`send_event`):** Các trạm không gọi hàm chéo trực tiếp của nhau mà đóng gói thông điệp thành đối tượng `Event` và chuyển cho bộ điều phối xử lý qua `self.dispatcher.schedule_event(event)` hoặc lưu tạm vào danh sách đệm xuất xưởng `_output_events`.

Listing 5: Lớp cơ sở LogicalProcess và kiểm soát LVT (bkitevsim/core/logical_process.py)

```
class BkSysLogicalProcessState:
    """Encapsulates the mutable state of a Logical Process.

    Supports checkpointing, cloning, and state restoration for PDES.
    """
    def __init__(self, **kwargs: Any) -> None:
        self._data: Dict[str, Any] = dict(kwargs)
        self.history: List[Dict[str, Any]] = []

    def get(self, key: str, default: Any = None) -> Any:
        return self._data.get(key, default)

    def set(self, key: str, value: Any) -> None:
```

```
        self._data[key] = value

    def checkpoint(self) -> Dict[str, Any]:
        """Create a deep-copy checkpoint of the state."""
        return copy.deepcopy(self._data)

    def restore(self, checkpoint: Dict[str, Any]) -> None:
        """Restore state from a previous checkpoint."""
        self._data = copy.deepcopy(checkpoint)


class BkSysLogicalProcess(abc.ABC):
    """Abstract base class for a BkSys Logical Process in PDES.

    Each LP has a unique ID and maintains its own Local Virtual Time (LVT).
    """
    def __init__(
        self,
        lp_id: int | str,
        initial_state: Optional[BkSysLogicalProcessState] = None
    ) -> None:
        self.lp_id = lp_id
        self.local_virt_time: float = 0.0
        self.state = initial_state if initial_state is not None else
            BkSysLogicalProcessState()
        self._output_events: List[Event] = []
        self._dispatcher: Optional[Any] = None

    def set_local_virt_time(self, local_virt_time: float) -> None:
        """Enforces Local Causality Constraint (LCC): time cannot travel
            backward."""
        if local_virt_time < self.local_virt_time:
            raise ValueError(
                f"Causality violation in LP {self.lp_id}: cannot travel backward
                    "
                f"from {self.local_virt_time:.4f} to {local_virt_time:.4f}"
            )
        self.local_virt_time = local_virt_time

    def send_event(self, event: Event) -> None:
        """Send an event to another LP or buffer it for dispatcher."""
        if self._dispatcher is not None and hasattr(self._dispatcher, "
            schedule_event"):
            self._dispatcher.schedule_event(event)
        else:
            self._output_events.append(event)
```

3.3.2 Kiến trúc trạm phục vụ hợp nhất 6 mô hình hàng đợi Kendall

Thay vì hiện thực rời rạc từng lớp phần mềm riêng lẻ cho mỗi bài toán, lớp `QueueServerLP` được xây dựng theo kiến trúc hợp nhất dựa trên ký hiệu chuẩn Kendall $A/S/c/K$. Bằng cách tham số hóa hai biến quyết định:

- `num_servers` (c): Số lượng máy chủ song song ($1 \leq c \leq \infty$).
- `capacity` (K): Tổng sức chứa tối đa của hệ thống (phục vụ + ghế chờ).

Hệ thống hợp nhất toàn bộ 6 mô hình hàng đợi lý thuyết một cách nhất quán:

1. **Mô hình $M/M/1$:** `num_servers = 1, capacity = None` (1 server, hàng đợi vô hạn).
2. **Mô hình $M/M/1/K$:** `num_servers = 1, capacity = K` (1 server, bộ đệm giới hạn K).
3. **Mô hình $M/M/\infty$:** `num_servers = float('inf'), capacity = None` (Vô hạn máy chủ).
4. **Mô hình $M/M/n/n$:** `num_servers = n, capacity = n` (Mô hình Erlang loss, 0 ghế chờ).
5. **Mô hình $M/M/c$:** `num_servers = c, capacity = None` (c máy chủ, hàng đợi vô hạn).
6. **Mô hình $M/M/c/K$:** `num_servers = c, capacity = K` (c máy chủ, bộ đệm giới hạn K).

Listing 6: Hai nhánh xử lý nghiệp vụ cốt lõi trong QueueServerLP

```
    def _handle_arrival(self, event: Event) -> None:
        """Process ARRIVAL event: checks buffer capacity, allocates server or enqueues."""
        job: Job = event.payload["job"] if isinstance(event.payload, dict) and "job" in event.payload else event.payload
        job.arrival_time = event.timestamp
        self.metrics.record_arrival()

        # Branch 1: Finite buffer capacity check (M/M/1/K, M/M/c/K, M/M/n/n)
        if self.capacity is not None and self.total_in_system >= self.capacity:
            self.metrics.record_drop()  # Drop packet when buffer is full
            self._notify_state(event.timestamp)
            return

        # Branch 2: Server dispatching and waiting queue logic
        if self.busy_servers < self.num_servers:
            self.busy_servers += 1
            self._start_service(job, event.timestamp)
        else:
            self.waiting_queue.append(job)  # Enqueue into FIFO waiting buffer

        self._notify_state(event.timestamp)

    def _handle_departure(self, event: Event) -> None:
        """Process DEPARTURE event: records metrics, routes job downstream, and serves next."""
        payload = event.payload if isinstance(event.payload, dict) else {}
        job: Job = payload.get("job")
        service_duration = payload.get("duration", 0.0)

```

```
        if job and job.start_service_time is not None:
            # Compute waiting time in queue: Wq = start_service - arrival_time
            wait_time = job.start_service_time - job.arrival_time
            self.metrics.record_completion(wait_time, service_duration)
            job.departure_time = event.timestamp

            out_ev = create_event(
                timestamp=event.timestamp,
                sender_id=self.lp_id,
                receiver_id=self.dest_lp_id,
                event_type="ARRIVAL",
                payload={"job": job, "wait_time": wait_time, "service_time":
                    service_duration}
            )
            self.send_event(out_ev)

        # Serve next waiting customer from queue or mark server idle
        if self.waiting_queue:
            next_job = self.waiting_queue.popleft()
            self._start_service(next_job, event.timestamp)
        else:
            self.busy_servers = max(0, self.busy_servers - 1)

        self._notify_state(event.timestamp)
```

Bảng đối chiếu tham số hóa hợp nhất cho 6 mô hình hàng đợi Kendall:

Bảng 8 tổng hợp sự ánh xạ giữa 6 mô hình lý thuyết Kendall và các tham số cấu hình cụ thể trong lớp `QueueServerLP`.

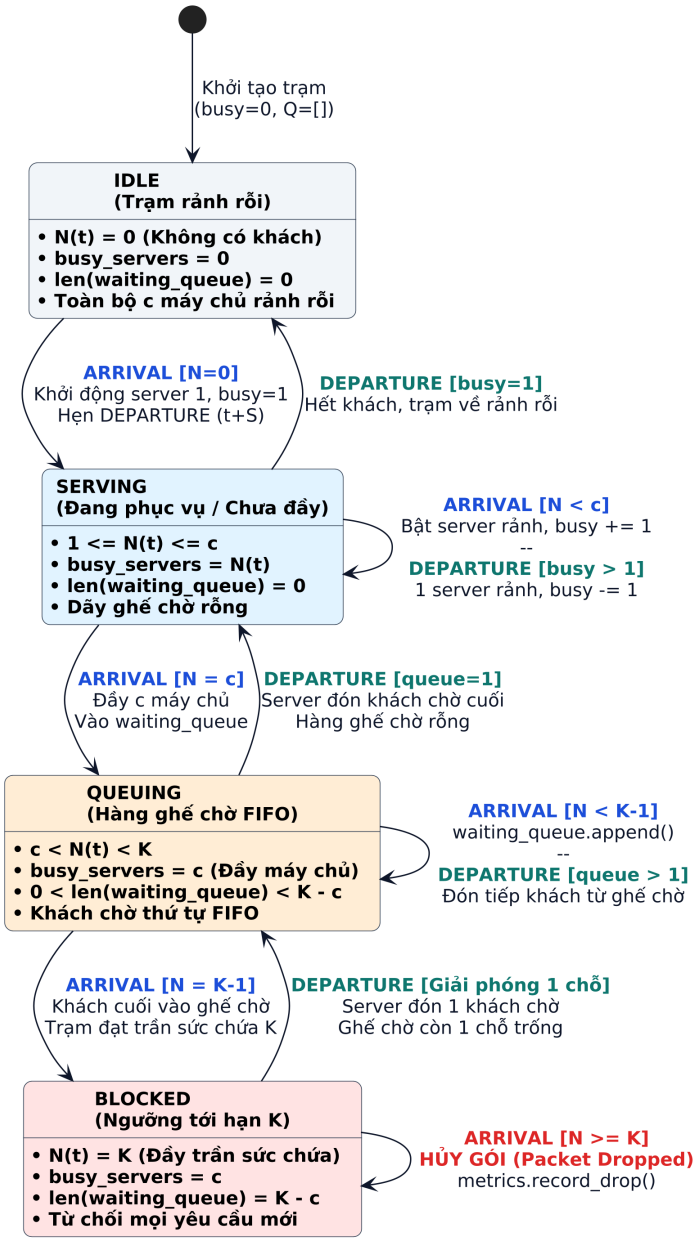
Mô hình	`num_servers` (c)	`capacity` (K)	Ghế chờ	Cơ chế tổn thất / rớt gói
$M/M/1$	1	`None`	Vô hạn (∞)	Không có rớt gói ($P_b = 0$, bộ đệm vô hạn)
$M/M/1/K$	1	K	$K-1$ chỗ	Hủy gói khi tổng số khách trong trạm $\geq K$
$M/M/\infty$	∞ (`inf`)	`None`	Không cần ghế ($W_q = 0$)	Không rớt gói; mỗi khách có máy chủ phục vụ riêng
$M/M/n/n$	n	n	Không có ghế (0 chỗ)	Rớt gói Erlang-B khi toàn bộ n máy chủ đều bận
$M/M/c$	c	`None`	Vô hạn (∞)	Không rớt gói; xếp hàng Erlang-C khi bận cả c máy chủ
$M/M/c/K$	c	K	$K-c$ chỗ	Hủy gói khi tổng số khách đang phục vụ + chờ $\geq K$

Bảng 8: *Ánh xạ tham số cấu hình hợp nhất cho 6 mô hình hàng đợi Kendall*

Máy trạng thái hữu hạn xử lý sự kiện trong QueueServerLP:

Hai phương thức `_handle_arrival` và `_handle_departure` trong Listing 6 hiện thực hóa

hai máy trạng thái khép kín được đặc tả qua sơ đồ chuyển trạng thái FSM (Hình 6):



Hình 6: *Máy trạng thái hữu hạn (FSM) mô hình hóa vòng đời phục vụ của* `QueueServerLP`

- **Máy trạng thái tiếp nhận (`_handle_arrival`):**

  a) *Kiểm tra sức chứa:* Tính tổng số khách trong trạm thông qua thuộc tính `total_in_system`, xác định bởi $N(t) = \text{busy} + |\mathcal{Q}_{\text{wait}}|$. Nếu $N(t) \geq K$ (với cấu hình có bộ đệm hữu hạn), trạm lập tức kích hoạt `self.metrics.record_drop()` và hủy gói tin.

  b) *Điều phối máy chủ rảnh:* Nếu busy_servers $< c$, trạm tăng số máy chủ bận thêm 1 (`self.busy_servers += 1`), bốc thời gian phục vụ $S \sim \text{Exp}(\mu)$ và phát sự kiện `DEPARTURE` trong tương lai qua `create_event` với độ trễ S.

  c) *Xếp hàng chờ FIFO:* Nếu cả c máy chủ đều bận, yêu cầu được đẩy vào hàng ghế chờ

`waiting_queue` bằng thao tác `append` đạt $\mathcal{O}(1)$.

- **Máy trạng thái hoàn tất (`_handle_departure`):**

a) *Ghi nhận số liệu thống kê:* Tính thời gian chờ thực tế $W_q = t_{\text{start_service}} - t_{\text{arrival}}$ và ghi nhận vào bộ tích lũy `MetricsCollector` qua phương thức `record_completion(wait_time, service_duration)`.

b) *Chuyển tiếp tác vụ:* Đóng gói đối tượng `Job` đã hoàn tất và phát sự kiện `ARRIVAL` tới trạm kế tiếp (`dest_lp_id`) qua `create_event` với độ trễ $\Delta t = 0.0$.

c) *Đón khách tiếp theo:* Nếu hàng chờ `waiting_queue` còn khách, lấy phần tử đầu hàng qua `popleft()` ($\mathcal{O}(1)$) và phục vụ ngay. Nếu hàng rỗng, giảm biến đếm máy chủ bận: busy_servers $\leftarrow \max(0, \text{busy_servers} - 1)$.

3.3.3 Hiện thực mạng hàng đợi Jackson và Cơ chế rẽ nhánh xác suất

Nhằm mô hình hóa mạng máy tính 4 nút trạm phức tạp theo Hình 1 của đề bài (Source → I/O → CPU → Printer/Disk → Sink), lớp `BranchingServerLP` kế thừa từ `QueueServerLP` và ghi đè logic hoàn tất phục vụ:

Listing 7: Hiện thực rẽ nhánh xác suất trong BranchingServerLP (bkitevsim/models/network.py)

```
class BranchingServerLP(QueueServerLP):
    """Probabilistic routing server modeling CPU departure branch in Jackson
        network."""

    def __init__(
        self,
        lp_id: int | str = "CPU",
        printer_lp_id: int | str = "Printer",
        disk_lp_id: int | str = "Disk",
        p_printer: float = 0.3,
        service_rate: float = 5.0,
        num_servers: float = 1,
        capacity: Optional[int] = None,
        metrics: Optional[MetricsCollector] = None
    ) -> None:
        super().__init__(
            lp_id=lp_id,
            dest_lp_id=printer_lp_id,
            service_rate=service_rate,
            num_servers=num_servers,
            capacity=capacity,
            metrics=metrics
        )
        self.printer_lp_id = printer_lp_id
        self.disk_lp_id = disk_lp_id
        self.p_printer = p_printer  # Probability of routing to Printer (0.3)

    def _handle_departure(self, event: Event) -> None:
```

```
        """Route departing job probabilistically via Bernoulli trial to Printer
            or Disk."""
        payload = event.payload if isinstance(event.payload, dict) else {}
        job: Job = payload.get("job")
        service_duration: float = payload.get("duration", 0.0)

        if job and job.start_service_time is not None:
            wait_time = job.start_service_time - job.arrival_time
            self.metrics.record_completion(wait_time, service_duration)
            job.departure_time = event.timestamp

            # Probabilistic routing: Bernoulli trial with probability p_printer
            dest = self.printer_lp_id if (random.random() < self.p_printer) else
                self.disk_lp_id
            if not hasattr(job, "orig_arrival_time"):
                job.orig_arrival_time = job.arrival_time
            job.arrival_time = event.timestamp

            out_ev = create_event(
                timestamp=event.timestamp,
                sender_id=self.lp_id,
                receiver_id=dest,
                event_type="ARRIVAL",
                payload={"job": job}
            )
            self.send_event(out_ev)

        if self.waiting_queue:
            next_job = self.waiting_queue.popleft()
            self._start_service(next_job, event.timestamp)
        else:
            self.busy_servers = max(0, self.busy_servers - 1)

        self._notify_state(event.timestamp)
```

- **Cơ chế rẽ nhánh Bernoulli (`p_printer`):** Tại ngã ba xuất phát từ CPU, bộ điều phối trạm thực hiện phép thử ngẫu nhiên: nếu random() $< p_{\text{printer}}$ (mặc định 0.3) thì chuyển gói tin sang trạm `PrinterLP`, ngược lại $(1 - p = 0.7)$ chuyển sang trạm `DiskLP`.
- **Cơ chế bảo toàn mốc thời gian gốc (`orig_arrival_time`):** Khi gói tin chuyển tiếp qua các trạm I/O → CPU → Printer/Disk, thuộc tính `arrival_time` được cập nhật tại chỗ để tính cục bộ $W_{q,\text{node}}$, trong khi `orig_arrival_time` được giữ nguyên vẹn giúp trạm đích `NetworkSinkLP` tính toán chính xác độ trễ toàn trình (*End-to-End Latency*):

$$T_{\text{End-to-End}} = t_{\text{Sink}} - \text{job.orig_arrival_time} \tag{75}$$

giúp đối chiếu trực tiếp với định lý Jackson giải tích: $W_{\text{network}} = W_{\text{IO}} + W_{\text{CPU}} + 0.3W_{\text{Printer}} + 0.7W_{\text{Disk}}$.

3.4 Cơ chế giao tiếp phi nghẽn và Điều phối bất đồng bộ (Python Asyncio)

3.4.1 Cơ chế giao tiếp phi nghẽn luồng theo mô hình Promise / Future

Để kết nối giữa luồng gọi người dùng và bộ máy mô phỏng mà không gây nghẽn luồng chính (*Non-blocking I/O*), lớp `AsyncFutureManager` hiện thực mẫu thiết kế Promise/Future:

- **Khởi tạo và Quản lý Event Loop an toàn (`_get_loop`):** Tự động phát hiện vòng lặp sự kiện `asyncio` đang chạy hoặc tạo mới an toàn trên các phiên bản Python 3.10+, 3.11+ và 3.12+, tránh hoàn toàn lỗi `RuntimeError: no running event loop`.
- **Tạo lời hứa (`create_future`):** Khởi tạo một đối tượng `asyncio.Future` được gán nhãn duy nhất theo `request_id` và lưu vào bảng tra cứu `_pending_futures`. Luồng gọi coroutine phía người dùng có thể thực hiện toán tử `await future` để tạm dừng tác vụ một cách hòa nhã mà không chiếm dụng CPU.
- **Giải phóng lời hứa (`set_result`):** Khi trạm thu gom `SinkLP` tiếp nhận kết quả cuối cùng của yêu cầu, phương thức `set_result(request_id, result)` lập tức hoàn tất Future và đánh thức coroutine đang chờ để nhận kết quả ngay tức thời.
- **Báo lỗi và Hủy an toàn (`set_exception`, `cancel_all`):** Cho phép truyền trực tiếp đối tượng ngoại lệ vào Future khi gặp sự cố, hoặc hủy bỏ hàng loạt các lời hứa đang treo khi dừng hệ thống.

Listing 8: Cơ chế Promise / Future phi nghẽn (bkitevsim/async_engine/future_manager.py)

```
class AsyncFutureManager:
    """Manages asynchronous Request-to-Future lifecycle without thread blocking.
    """

    def __init__(self, loop: Optional[asyncio.AbstractEventLoop] = None) -> None
        :
        self.loop = loop
        # Pending future registry: mapping request_id -> asyncio.Future
        self._pending_futures: Dict[int, asyncio.Future] = {}

    def _get_loop(self) -> asyncio.AbstractEventLoop:
        """Retrieve active running event loop or create a fresh loop safely."""
        if self.loop is not None and not self.loop.is_closed():
            return self.loop
        try:
            return asyncio.get_running_loop()
        except RuntimeError:
            loop = asyncio.new_event_loop()
            asyncio.set_event_loop(loop)
            self.loop = loop
            return loop

```

```
    def create_future(self, request_id: int) -> asyncio.Future:
        """Create and register an asyncio.Future attached to a unique request ID."""
        loop = self._get_loop()
        future = loop.create_future()
        self._pending_futures[request_id] = future
        return future

    def set_result(self, request_id: int, result: Any) -> bool:
        """Resolve promise with result, awakening awaiting coroutine."""
        future = self._pending_futures.pop(request_id, None)
        if future and not future.done():
            future.set_result(result)
            return True
        return False
```

3.4.2 Các mẫu điều phối đồng quy có cấu trúc (Structured Concurrency)

Từ phiên bản Python 3.11+, cơ chế **Structured Concurrency** qua `asyncio.TaskGroup` được áp dụng để quản lý an toàn vòng đời coroutine. Lớp `AsyncTaskCoordinator` chuẩn hóa 4 mẫu điều phối:

1. **Mẫu TaskGroup (`run_with_task_group`):** Quản lý các coroutine con trong ngữ cảnh `async with asyncio.TaskGroup()`. Khi có một tác vụ con gặp ngoại lệ, toàn bộ các tác vụ còn lại sẽ được hủy tự động, loại trừ nguy cơ rò rỉ tác vụ (*Task Leak*).
2. **Mẫu Gom nhóm song hành (`run_with_gather`):** Kích hoạt đồng thời danh sách các coroutine và đợi kết quả tổng thể với `asyncio.gather`.
3. **Mẫu Tiêu thụ FCFS (`process_as_completed`):** Xử lý kết quả theo nguyên tắc Đến trước – Phục vụ trước với `asyncio.as_completed`. Bất kỳ tác vụ nào xong trước sẽ được chuyển ngay cho hàm callback.
4. **Mẫu Runner cô lập (`run_in_runner`):** Sử dụng `asyncio.Runner()` để thiết lập Event Loop độc lập cho mỗi đợt mô phỏng, bảo đảm dọn dẹp sạch tài nguyên hệ điều hành sau khi hoàn thành.

Listing 9: 4 mẫu điều phối bất đồng bộ chuẩn mực (bkitevsim/async_engine/task_coordinator.py)

```
class AsyncTaskCoordinator:
    """Orchestrates concurrent simulation execution (Section 4 specifications)."""

    @staticmethod
    async def run_with_task_group(
        coroutines: Sequence[Coroutine[Any, Any, Any]]
    ) -> List[Any]:
```

```
        """Listing 2b: Structured concurrency via asyncio.TaskGroup (Python
            3.11+)."""
        tasks = []
        async with asyncio.TaskGroup() as tg:
            for coro in coroutines:
                task = tg.create_task(coro)
                tasks.append(task)
        return [task.result() for task in tasks]

    @staticmethod
    async def run_with_gather(
        coroutines: Sequence[Coroutine[Any, Any, Any]]
    ) -> List[Any]:
        """Listing 2a: Unscoped concurrent execution using asyncio.gather."""
        return await asyncio.gather(*coroutines)

    @staticmethod
    async def process_as_completed(
        coroutines: Sequence[Coroutine[Any, Any, Any]],
        callback: Callable[[Any], None]
    ) -> List[Any]:
        """Listing 2c: Streaming results in FCFS order using as_completed."""
        results = []
        for finished_task in asyncio.as_completed(coroutines):
            result = await finished_task
            callback(result)
            results.append(result)
        return results

    @staticmethod
    def run_in_runner(main_coro: Coroutine[Any, Any, Any]) -> Any:
        """Listing 3a(bis): Program entry point with isolated asyncio.Runner."""
        with asyncio.Runner() as runner:
            return runner.run(main_coro)
```

Mẫu điều phối	Phạm vi	Cơ chế quản lý ngoại lệ	Kịch bản ứng dụng tối ưu
`TaskGroup`	Scoped	Tự động hủy tác vụ còn lại nếu xảy ra lỗi (*Fail-fast*)	Chạy nhóm mô phỏng ràng buộc nghiệp vụ liên đới
`gather`	Unscoped	Trả về danh sách ngoại lệ hoặc hủy thủ công qua task handle	Chạy song song các bộ tham số hoàn toàn độc lập
`as_completed`	Streaming	Xử lý ngay khi tác vụ hoàn tất theo thứ tự FCFS	Bơm trực tiếp dữ liệu ra dashboard theo thời gian thực
`Runner`	Isolated	Khởi tạo và đóng Event Loop độc lập tuyệt đối	Chạy các bài kiểm thử tự động (*Unit / Integration Test*)

Bảng 9: *So sánh đặc tính kỹ thuật của 4 mẫu điều phối bất đồng bộ*

3.5 Quản lý thời gian ảo và Kiểm chứng tính nhân quả cục bộ

3.5.1 Nguyên lý bước nhảy thời gian ảo và Phân tách với thời gian thực

Sự phân tách tuyệt đối giữa **Thời gian ảo mô phỏng (*Virtual Simulation Time*)** và **Thời gian thực máy tính (*Wall-clock Time*)** là nguyên lý cốt lõi của công nghệ DES.

Cảnh báo sai lầm nghiêm trọng trong mô phỏng DES

Tuyệt đối không được dùng lệnh `time.sleep()` hoặc `asyncio.sleep()` để mô phỏng độ trễ phục vụ. Việc dùng lệnh sleep sẽ trói buộc thời gian ảo vào chu kỳ xung nhịp vật lý của CPU, khiến cuộc thử nghiệm 10.000 giây ảo đòi hỏi máy tính thực tế phải chờ gần 3 giờ đồng hồ.

Trong bộ mô phỏng của nhóm:

- Đồng hồ `dispatcher.current_time` thực hiện **bước nhảy gián đoạn (jump)** thẳng tới timestamp của sự kiện kế tiếp trong hàng đợi.
- Các khoảng thời gian hệ thống rảnh rỗi (*idle*) hoặc khách ngồi đợi trong hàng ghế không tiêu tốn chu kỳ CPU thực tế.
- Một kịch bản thử nghiệm quy mô lớn với 50.000 sự kiện kéo dài 5000 giây ảo được hoàn tất chỉ trong khoảng $0.2 \rightarrow 0.4$ giây thực tế máy tính.

Bảng 10 phân định rạch ròi sự khác biệt bản chất giữa mô phỏng sự kiện rời rạc và mô phỏng theo nhịp xung thời gian thực.

Tiêu chí kỹ thuật	Mô phỏng DES (Hệ thống đề tài)	Mô phỏng thời gian thực (Real-time)
Bước nhảy thời gian	Gián đoạn theo sự kiện: $t_{k+1} = e_{\text{next}}.\text{timestamp}$	Tuyến tính theo chu kỳ cố định Δt hoặc lệnh `sleep()`
Hiệu suất CPU	Tối đa 100%, bỏ qua hoàn toàn các khoảng thời gian rảnh rỗi	Lãng phí chu kỳ xung nhịp chờ lệnh `sleep()` nhả khóa
Tốc độ mô phỏng	Nhanh hơn thực tế hàng nghìn lần (5000s ảo $\approx$ 0.3s thực)	Bị neo chặt vào thời gian vật lý thực (5000s ảo $\approx$ 5000s thực)
Độ chính xác thời gian	Chính xác số học tuyệt đối theo phân phối ngẫu nhiên	Bị sai lệch do độ trễ điều phối luồng (*OS Thread Jitter*)

Bảng 10: *So sánh bản chất giữa Mô phỏng DES và Mô phỏng theo thời gian thực*

3.5.2 Ràng buộc nhân quả cục bộ và Kiểm chứng vết tính toán

Để kết quả mô phỏng có giá trị khoa học, chuỗi tiến trình phải tuân thủ nghiêm ngặt **Ràng buộc nhân quả cục bộ (*Local Causality Constraint*)**:

$$t_{\text{current}}^{(1)} \leq t_{\text{current}}^{(2)} \leq \cdots \leq t_{\text{current}}^{(M)} \tag{76}$$

Chứng minh tính bảo toàn nhân quả trong hạt nhân điều phối: Tính bất biến nhân quả $t_{\text{current}}^{(n+1)} \geq t_{\text{current}}^{(n)}$ được bảo đảm tuyệt đối nhờ hai tiền đề toán học:

- **Tiền đề trật tự Min-Heap:** Hàng đợi sự kiện $\mathcal{Q}$ luôn trả về phần tử có timestamp nhỏ nhất: $e_{\text{next}} = \min_{e \in \mathcal{Q}}(e.\text{timestamp})$. Do đó, $e_{\text{next}}.\text{timestamp} \geq t_{\text{current}}$.
- **Tiền đề độ trễ không âm:** Mọi sự kiện phát sinh mới trong tương lai đều có độ trễ $\Delta t \geq 0$, tức là $t_{\text{new}} = t_{\text{current}} + \Delta t \geq t_{\text{current}}$. Không một tiến trình nào có thể gửi thông điệp về quá khứ ($\Delta t < 0$).

Từ hai tiền đề trên, theo phương pháp quy nạp toán học, chuỗi thời gian ảo của vòng lặp điều phối luôn là một dãy đơn điệu tăng ngặt trên trục thời gian, loại trừ hoàn toàn nguy cơ vi phạm nhân quả (*Causality Violation*).

Hệ thống bảo đảm tính đơn điệu này qua 2 lớp phòng vệ thực tiễn:

1. **Phòng vệ nội tại tại từng LP (`set_local_virt_time`):** Nếu một sự kiện nổ ra có timestamp nhỏ hơn LVT hiện tại của trạm, phương thức lập tức ném ngoại lệ `ValueError("Causality violation in LP...")` và ngắt tiến trình để bảo toàn tính toàn vẹn của thời gian ảo.
2. **Kiểm chứng tự động trên vết thực thi (`trace_log`):** Toàn bộ các sự kiện được ghi nhận vào nhật ký vết tính toán và được kiểm chứng tự động qua bộ test suite chính quy trong tệp `tests/test_verification.py`:
   - `test_causality_trace_analysis()`: Kiểm tra tính đơn điệu tăng ngặt của trục thời gian ảo trên toàn bộ vết sự kiện được ghi nhận bởi Dispatcher.
   - `test_priority_queue_monotonicity()`: Kiểm tra thứ tự trích xuất sự kiện luôn đơn điệu tăng trên cả `MinHeapPriorityQueue` và `LadderQueue`.
   - `test_event_conservation_law()`: Kiểm chứng định luật bảo toàn biến cố: tổng số gói tin đến bằng tổng số gói tin phục vụ xong cộng số gói tin bị rớt do tràn bộ đệm.

TÀI LIỆU THAM KHẢO

[1] Raj Jain, *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*, John Wiley & Sons, 1991.

[2] Mor Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*, Cambridge University Press, 2013.

[3] Leonard Kleinrock, *Queueing Systems, Volume 1: Theory*, Wiley-Interscience, 1975.

[4] Richard M. Fujimoto, *Parallel and Distributed Simulation Systems*, John Wiley & Sons, 2000.

[5] Python Software Foundation, *asyncio — Asynchronous I/O Documentation (Python 3.11+)*, https://docs.python.org/3/library/asyncio.html.

[6] Khoa KH&KT Máy tính – ĐH Bách Khoa ĐHQG-HCM, *Course: System Performance Evaluation – The Queuing Systems-based Performance Evaluation Project (hk261_dasyn_asspe)*, Tháng 10/2026.